DIALITE: Discover, Align and Integrate Open Data Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We demonstrate a novel table discovery pipeline called DIALITE that allows users to discover, integrate and analyze open data tables. DIALITE has three main stages. First, it allows users to discover tables from open data platforms using state-of-the-art table discovery techniques. Second, DIALITE integrates the discovered tables to produce an integrated table. Finally, it allows users to analyze the integration result by applying different downstreaming tasks over it. Our pipeline is flexible such that the user can easily add and compare additional discovery and integration algorithms.

CCS CONCEPTS

• **Information systems → Information integration**.

KEYWORDS

Data Lakes, Data Discovery, Data Integration, Full Disjunction

ACM Reference Format:
Aamod Khatiwada, Roee Shraga, and Renée J. Miller. 2023. DIALITE: Discover, Align and Integrate Open Data Tables. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, Seattle, WA, USA, 4 pages. https://doi.org/10.1145/3555041.3589732

1 INTRODUCTION

Data discovery has become an important component in data science pipeline. The discovery process uses techniques such as keyword search [13] and table search [4, 7, 9] to discover a set of tables (datasets). Data scientists use such tables to support decision-making processes, train machine learning models, perform statistical analysis, and so on. After discovery, a natural step is to integrate the discovered tables. An integrated table provides a unified view of the data and allows users to run queries and analyses that go beyond a single table.

While integrating tables, we intend to combine tuples from different tables in a maximal way such that the integrated tuples carry as much information as possible. This enriches the analysis and decision-making process after integration and also improves the quality of downstream applications. The widely known outer-join [12] operator is not associative and does not aim to maximize the connections among the integrated tuples [6]. Accordingly, *Full Disjunction (FD)* [6] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [11]. FD can be viewed as an associative version of outer join [2] and has been used to integrate information across relational tables [2, 10] and web tables [10]. In a recent paper [8], we proposed ALITE, a new algorithm based on Full Disjunction to integrate tables discovered in data lakes. ALITE was shown to be correct and faster than the existing FD algorithms [2, 10] while integrating real open data lake tables in practice [8]. Additional details on other integration operators and their comparison against FD are available in the full ALITE paper [8] where FD was shown to be a better semantics for integration because it produces a result over which a downstream task like entity-resolution performs more accurately.

In this demonstration, we propose DIALITE, a novel system that discovers, aligns, and integrates open data tables. It extends the aforementioned ALITE [8], which does not perform discovery but instead takes a set of tables as input, aligns the matching columns using holistic schema matching, and applies the FD to get an integrated table. DIALITE offers state-of-the-art systems for different table discovery tasks [7, 14] before applying ALITE to integrate them. Furthermore, DIALITE also offers new downstream analytics (that have not been previously considered) to evaluate the quality of integration. Specifically, DIALITE allows users to

(1) Upload a table or randomly generate one using GPT-3 as a query and discover related tables from a given data lake that are unionable [7] or joinable [14] with the query table. Apart from the available table discovery algorithms, DIALITE also allows users to add new algorithms for table search based on their preference.
(2) Integrate the discovered tables (or upload a set of tables to be integrated) using a novel table integration system called ALITE [8]. Besides ALITE, we allow users to add new integration operators within our extendible architecture.
(3) Analyze and compare the table integrated using ALITE against alternative integration techniques by performing downstream applications. In our demo, we consider outer join as an alternative integration technique (or other queries/methods added by the user) and present data analytics (common aggregations and statistics) and entity-resolution as downstream applications. Both, when applied over real tables (which can be incomplete) will show the dramatic difference between maximally integrating information using FD vs. using outer joins.

Related Work. To the best of our knowledge, DIALITE is the first system that enables a full table search pipeline starting from

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589732

table discovery, followed by table integration and downstream applications over the integrated table. An earlier system called COCOA [3] focuses on finding joinable tables with correlated attributes that expand the query table. After finding the tables, COCOA applies LEFT JOIN between the query table and the discovered table as the integration operator. DIALITE, on the other hand, finds a set of related tables that can be integrated with the query table (including unionable and joinable tables). Auctus [1] also discovers related tables and integrates table pairs by applying inner join or union. Unlike Auctus, DIALITE does not limit the number of tables to be integrated. In addition, DIALITE uses FD to integrate the tables, which has been shown in theory to maximize the connections among the tuples in the tables [11].

Existing table discovery techniques search for joinable, unionable, or related tables. **JOSIE** [14] and **LSH Ensemble** [15], for instance, take a query table as input with a query column marked by the user and returns a set of top-k tables that are joinable with the query table. **SANTOS** [7] takes a query table and discovers a set of top-k semantically unionable tables as output. Other table search techniques [4, 9] also focus on table discovery from large repositories. But the important question of how to integrate the discovered tables is not addressed. **ALITE** [8] provides a solution to this question. As the data lake tables may lack consistent and meaningful column headers, ALITE applies holistic schema matching over the set of searched tables and assigns a dummy column header called an Integration ID to the set of matching columns. Then, it applies a natural FD over the integration IDs to integrate the tables. The integration process outputs an integrated table with maximally integrated tuples [6, 11].

2 SYSTEM DESCRIPTION

DIALITE, outlined in Fig. 1, has three stages (discover, align & integrate, and analyze) which are included in our demo plan (Sec. 3).

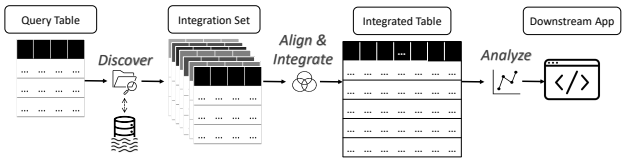


Figure 1: An overview of DIALITE.

DIALITE offers various options for discovery, alignment, integration, and analysis, including the ability for users to implement their own methods. The pipeline begins with a user-provided query table Q. However, we also allow the user to randomly generate a table using GPT-3 [5]. We now detail the components.

2.1 Discover

Given a query table Q, DIALITE uses a data discovery method to find tables in a data lake (table repository) $\mathcal{D}$ that are unionable [7], joinable [14], or simply semantically similar. DIALITE allows the user to choose among existing discovery algorithms including SANTOS [7] and LSH Ensemble [15]. Alternatively, a user can use their own discovery algorithm by implementing a similarity method between two tables. As we apply ALITE [8] for alignment and integration, the discovery phase is agnostic to the type of search. ALITE, aiming to maximize the connections among the tables, decides how the tables should be integrated. The output of discovery is an *integration set* $D \subseteq \mathcal{D}$, a set of tables to be integrated, including the query table. Our discovery techniques allow users to control the number of tables returned, and our system permits users to select a subset of the discovered tables to be integrated.

2.2 Align and Integrate

Given an integration set D, DIALITE uses ALITE [8] to integrate the tables. The integration set can be derived from the discovery part (Sec. 2.1) or provided as input by the user. The latter represents a traditional data integration scenario where the integration set is given. As shown in Fig. 1, this stage outputs an integrated table.

ALITE is composed of two main parts, namely align and integrate. Note that we do not require reliable column headers in the integration set. The first part of ALITE (Align) applies holistic schema matching to identify common columns in the integration set. The matched columns are given the same integration ID. Using these ids as attribute names, the integrate part applies (natural) FD. Specifically, ALITE uses a new holistic schema matching algorithm that was shown to outperform state-of-the-art matchers. ALITE uses a novel algorithm to compute the FD shown to be correct and efficient on real tables (including tables having nulls) [8]. DIALITE also allows users to add alternative integration operators, e.g., outer join, which is included for demonstration.

2.3 Analyze

Given an integrated table (that can also be uploaded as a file by the user), DIALITE allows the user to explore the benefits of integration. Specifically, the user can choose a downstream application to apply over the integrated table. A simple application is an aggregation query that can be applied over the integrated table as we illustrate in Sec. 3.1. In Sec. 3.2 we also present a more complex downstream application, entity-resolution (ER).

2.4 Implementation

DIALITE is implemented in Python 3.8 and the demonstration uses a web application.[1] Within our pipeline, we use SANTOS [2], LSH Ensemble [3] and ALITE [4] using their publicly available code. Also, we use the *py_entitymatching* package to show ER as a downstream application.[5] We allow users to interact with the system after each step so that they can validate the intermediate results.

3 DEMONSTRATION PLAN

Next, we illustrate our demonstration. The link to a demonstration video is available in the github repository.[1] Our demo contains two parts. First, we present the use case of the pipeline as described in Sec. 2. This part is actually composed of three demonstration items, that can be demonstrated independently. Then, we demonstrate DIALITE's extensibility to new algorithms for discovery, integration, and analysis.

[1] https://github.com/northeastern-datalab/dialite
[2] https://github.com/northeastern-datalab/santos
[3] https://github.com/ekzhu/datasketch
[4] https://github.com/northeastern-datalab/alite
[5] https://github.com/anhaidgroup/py_entitymatching

3.1 DIALITE Use case

Discovery. The users of DIALITE would be able to upload a query table in CSV format from an existing pool of tables.[6] Note that we will provide a data lake for the users to use in the demonstration. The tables in this data lake are real tables from open data and are currently preprocessed for our discovery algorithms [7, 15]. Specifically, the indexes used in SANTOS [7] and LSH Ensemble [15] are built offline, i.e., they are already available for the user to use. The users can easily preprocess and link their own data lake.

The set of discovered tables by all the table discovery systems is stored for the next step. As there may be an overlap in unionable and joinable search results, we persist *the set of tables* found by all techniques to form an integration set.

T_1 (Query Table)

TID	Country	City	Vaccination Rate (1+ dose)
t_1	Germany	Berlin	63%
t_2	England	Manchester	78%
t_3	Spain	Barcelona	82%

T_2 (Retrieved unionable table)

TID	Country	City	Vaccination Rate (1+ dose)
t_4	Canada	Toronto	83%
t_5	Mexico	Mexico City	±
t_6	USA	Boston	62%

T_3 (Retrieved joinable table)

TID	City	Total Cases	Death Rate (per 100k residents)
t_7	Berlin	1.4M	147
t_8	Barcelona	2.68M	275
t_9	Boston	263k	335
t_{10}	New Delhi	2M	158

Figure 2: Tables detailing COVID-19 cases in different places. The symbol ± represents null values present in the input tables ("*missing nulls*").

Example 1. *Consider tables T_1, T_2 and T_3 about COVID-19 cases in different places shown in Fig. 2. Here, TID (Tuple ID) is not a real data column and it is added only to refer to the tuples. Let, T_1 be the query table, and tables T_2 and T_3 reside in the data lake repository. In our demo, a user selects* `City` *as an intent column and query column to search for a unionable table using SANTOS [7] and a joinable table using LSH Ensemble [15] respectively. Please see respective papers for additional details on the search algorithms. Let, T_2 and T_3 be results of the unionable and joinable searches respectively. So the result of the discovery step is an integration set of tables T_1, T_2, and T_3. Note that the names of columns are presented for simplicity and are not used by the discovery techinques, which are designed for the ambiguty of data lakes, i.e., unreliable/missing metadata.*

Align and Integrate. Once the integration set is formed, DIALITE allows user to apply ALITE's holistic schema matching to generate integration IDs. Over such IDs, we apply ALITE's FD.

Example 2. *Consider the integration set of tables T_1, T_2 and T_3 shown in Fig. 2 is formed after table discovery step as illustrated in Ex. 1. DIALITE applies ALITE's integration algorithm over these tables that returns an integrated table as shown in Fig. 3. The integration semantics is explained in ALITE paper [8].*

[6]For the demonstration itself, we also allow users to randomly generate a query table (see Sec. 3.2). Also, a user may choose to upload their query table noting that it may be off-topic wrt the data lake, which may yield no results.

FD(T_1, T_2, T_3) (Integrated Table by ALITE)

OID	TIDs	Country	City	Vaccination Rate (1+ dose)	Total Cases	Death Rate (per 100k residents)
f_1	{t_1, t_7}	Germany	Berlin	63%	1.4M	147
f_2	{t_2}	England	Manchester	78%	⊥	⊥
f_3	{t_3, t_8}	Spain	Barcelona	82%	2.68M	275
f_4	{t_4}	Canada	Toronto	83%	⊥	⊥
f_5	{t_5}	Mexico	Mexico City	±	⊥	⊥
f_6	{t_6, t_9}	USA	Boston	62%	263k	335
f_7	{t_{10}}	⊥	New Delhi	⊥	2M	158

Figure 3: Result of applying ALITE over the tables in Fig. 2. The symbol ⊥ represents the null values produced during the integration due to missing information ("*produced nulls*").

Analyze. After integration, the next feature that ALITE offers is the analysis of the integrated table. We show the use of an aggregation query over the integrated table.

Example 3. *With the integrated table, we now allow the user to use queries that go beyond the single tables. For example, over the integrated table (Fig. 3), the user can find that Boston is the city with the lowest vaccination rate and Toronto has the highest. Trying to understand the reason for that, the user may explore the relationship between vaccination rates (given in T_1 and T_2), number of cases and death rates (given in T_3). For example, the user can compute the correlation between vaccination and death rates that shows a positive (pearson) correlation of 0.16 and (somewhat surprising) correlation of 0.9 between case numbers and vaccination rates. While a bit counter-intuitive, the analysis reveals an interesting insight about the nature of vaccinations, suggesting that in cities with higher death rates and more cases, the government is focusing on vaccination programs and the people are more willing to vaccinate.*

3.2 DIALITE Extendibility

As described in Sec. 2, the user can extend DIALITE by implementing their own alternative components in the pipeline. Specifically, we aim to demonstrate the ability of users to implement (using python code) new discovery algorithms, integration methods and perform their required analysis.

Example 4. *For illustration, a sample code snippet to add a new joinable table discovery algorithm is provided in Fig. 4. The user basically implements a similarity function between two datasets (df1 and df2) that is used by DIALITE for table discovery.*

```
def new_joinability_discovery_algorithm(df1, df2):
    join_df = pd.merge(df1, df2, how ='inner')
    return len(join_df)/max(len(df1), len(df2))
```

Figure 4: Implementing user-defined discovery algorithm based on inner join.

We also consider a scenario where the user may not have a query table to start the analysis. So, DIALITE allows users to use simple prompts to generate the query table for the analysis. We demonstrate this feature using GPT-3 based implementation [5] and allow the user to generate a query table based on prompt as

Integrating Data Lake Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Wolfgang Gatterbauer
Northeastern University
Boston, Massachusetts, USA
w.gatterbauer@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We have made tremendous strides in providing tools for data scientists to discover new tables useful for their analyses. But despite these advances, the proper integration of discovered tables has been under-explored. An interesting semantics for integration, called Full Disjunction, was proposed in the 1980's, but there has been little progress in using it for data science to integrate tables culled from data lakes. We provide ALITE, the first proposal for scalable integration of tables that may have been discovered using join, union or related table search. We empirically show that ALITE can outperform previous algorithms for computing the Full Disjunction. ALITE relaxes previous assumptions that tables share common attribute names (which completely determine the join columns), are complete (without null values), and have acyclic join patterns. To evaluate ALITE, we develop and share three new benchmarks for integration that use real data lake tables.

PVLDB Reference Format:
Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. Integrating Data Lake Tables. PVLDB, 16(4): 932 - 945, 2022.
doi:10.14778/3574245.3574274

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/northeastern-datalab/alite.

1 INTRODUCTION

The number of public datasets has grown immensely in open data platforms [56, 57, 78]. Also, individual corporations have a wealth of data stored in their own data lakes. Analyzing and integrating such datasets can help governments and enterprises in making decisions and plans. Data scientists, as the main users of data, use different techniques to discover datasets such as keyword search [11, 12, 58, 75] and table search (using the data within their table as a query) [10, 28, 48, 50, 57, 80]. Such a process usually outputs a collection of data lake tables that may enrich their analysis [56]. Existing techniques usually discover unionable [12, 41, 48, 57], joinable [26, 55, 75, 77, 78], and related tables [10, 19, 77].

Example 1. *Consider the data lake tables about football stadiums shown in Fig. 1. We have added a* `TID` *(Tuple ID) column in each table to permit us to refer to tuples. Assume that a data scientist uses table* T_1 *as a* query table *to search for the top-2 unionable tables [57] and the top-2 joinable tables [78] from a data lake. Let* T_2 *and* T_3 *be the union search results and* T_4 *and* T_5 *be the join search results. Join search finds tables that join on an indicated column (in this case* `Location`*), but does not discover if there are other common (integratable) columns. For simplicity, assume that the common columns on these tables are already detected and have identical column headers. Note that in practice this will not be the case.*

After discovery, data scientists would often integrate the discovered tables before analyzing and applying statistical tools. Such integration not only extends their data but also allows them to answer queries that go beyond a single table. Consider Tables T_2, T_3 and T_4 in Fig. 1 and assume a football team has data scientists assisting in finding a new coach. Specifically, the team looks for an experienced coach who has handled teams playing in front of large crowds in new stadiums. So, they may use queries such as *"coaches who coach teams having stadiums established after 2000, that accommodate at least 50 thousand spectators"*. The information required here goes beyond a single table. In our example, one needs (at least) to integrate T_2, T_3 and T_4 to obtain such facts, for example, `Dan Campbell` who coaches the `Detroit Lions` that uses `Ford Field` Stadium established in `2002` and having a capacity of `65k` (f_7 in Fig. 2). Prior search methods do not address this "post-discovery" phase and do not answer the important question of how to integrate tables (relations) obtained by table search technique(s).

Example 2. *The standard relational union operator needs all tables to have exactly the same schema. However, this is not the case even for union search results (where tables that union on a subset of attributes can be retrieved) [57]. So to integrate the tables in Fig. 1, one can project out non-common columns and union on only the common columns. For* T_1, T_2, *and* T_3, *this would just leave* `Location`*. For the joinable tables, a join on only* `Location` *of* T_1 *with* T_4 *leads to tuples like* t_{11} *being omitted and the result has two* `Stadium` *attributes. Worse, the natural join operator, i.e.* $T_1 \bowtie T_4 \bowtie T_5$*, returns an empty set because* T_4 *and* T_5 *do not have joining tuples. The problem gets more complicated if we try to integrate all five tables using these operators.*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574274

T_1

TID	Stadium	Location	Team
t_1	NRG Stadium	Texas	Houston Texans
t_2	AT&T Stadium	Texas	Dallas Cowboys
t_3	Paul Brown	Ohio	±
t_4	Sofi Stadium	California	Angeles Chargers

T_2

TID	Stadium	Location	Opened
t_5	Soldier Field	Chicago	1924
t_6	Ford Field	Michigan	2002

T_3

TID	Team	Location	Coach
t_7	Houston Texans	Texas	Lovie Smith
t_8	Green Bay Packers	Wisconsin	Matt LaFleur
t_9	Detroit Lions	Michigan	Dan Campbell

T_4

TID	Stadium	Location	Capacity
t_{10}	NRG Stadium	Texas	±
t_{11}	Ford Field	Michigan	65k

T_5

TID	Stadium	Location	Team
t_{12}	Lambeau Field	Wisconsin	Green Bay Packers
t_{13}	±	Ohio	Cleveland
t_{14}	Sofi Stadium	California	±

Figure 1: Tables about football stadiums, their locations and home teams. The objective is to integrate the five tables. TID is not a real data column and metadata like column headers may not be available in real data lake tables, but are used for illustration purposes. The symbol ± represents null values present in the input tables ("*missing nulls*").

Within the data integration literature, *Full Disjunction (FD)* [32] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [65]. Indeed, Rajamaran and Ullman describe FD as a relation with nulls (denoted by ⊥) such that every set of join-consistent tuples appears within an FD tuple, with a concrete value or ⊥ in each attribute not found within the set of tuples [65]. Here, join-consistent is defined as common attributes (attributes with the same name), so this is effectively a *natural FD*. The widely known outer-join [46, 66] is not associative (hence, the result may depend on the order in which tables are integrated) and does not aim to maximize the connections among the integrated tuples [32, 52].

Example 3. *Outer-join and outer-union keep all tuples and columns and pad non-matching tuples (respectively, columns) with nulls [16, 46]. The outer-union of the tables from Fig. 1 is depicted in Fig. 2(a). It does not maximally connect the facts in the original tables. Here, ± indicates a missing value (missing null) in the original tables and ⊥ represents a null introduced by the outer-union operator (produced null). In particular, outer union includes partial facts like t_{10} that are made redundant by more complete facts like t_1. Similar observations can be made of the outer-join results. So, Galindo-Legario defined the Full Disjunction (FD) [32]. Informally, it removes redundant facts and produces, in this case, the first 8 tuples (mustard colored) of Fig. 2(b). FD can be viewed as an associative version of outer join [17] and has been used to integrate information across relational tables [17, 59] and web tables [59]. Notice in this example, FD uses information from all five tables so it is important to be able to compute FD over (possibly large) sets of tables.*

But using FD in data lakes poses several important challenges: 1) We cannot rely on common attributes having the same name as in our example. Instead, we must discover what the common attributes are [56]. We will use schema matching for this purpose [63]. Notice that we are not given just two schemas that need to be matched, rather we have a set of tables all of which could potentially share attributes with some or many of the other tables. Hence, we will use *holistic schema matching* [64]. 2) We cannot assume that integrated datasets are complete (that is, they may actually contain null values or partial facts). 3) Prior work on Full Disjunction has been done on relatively small relations (with only 1000 or so tuples per relation [17]) or assumes the common attributes form graphs with specific acyclic structures [65]. To the best of our knowledge, the only work on using FD on larger datasets requires that all joins be done on attributes having key-to-foreign-key relationships [59], a strict requirement that makes the technique only applicable within well-designed enterprise scenarios, not the possibly messy tables retrieved from data lakes commonly used in data science.

We assume data scientists use table discovery algorithms to identify a set of tables that they wish to integrate. Regardless of the search technique, we wish to find the best way to integrate the tables. Specifically, we propose a table integration technique ALITE (Align and Integrate) that first applies schema matching to identify common columns in a set of tables to be integrated. Matched columns are given the same integration ID. We then apply a natural FD over the tables using integration IDs as attribute names.

Contributions. (1) To the best of our knowledge, we introduce the problem of integrating data lake tables obtained using table discovery algorithms. (2) We propose a new holistic schema matching algorithm for sets of tables that outperforms the state-of-the-art matchers on the real data lake tables in our integration benchmarks. (3) We compare the FD used as the integration semantics in ALITE to several other semantics and show the difference and superiority of FD. Empirically, we show FD's superiority for a downstreaming task of entity resolution [44]. (4) We propose a novel algorithm to compute the FD by using complementation and subsumption operators in a novel way. We show that the use of these operators permits optimizations that make the computation faster than the state-of-the-art techniques, in practice. Specifically, ALITE scales better than the state-of-the-art FD algorithms on data lake tables, which are typically large and may have complex join graphs. (5) We introduce and share several open data integration benchmarks.

2 PRELIMINARIES

We now provide the building blocks for integrating tables in a data lake setting, namely, specifying the notation and the basic integration operators, after which we formally define the problem.

Table 1: Symbols used in this paper and their definitions

Symbol	Definition	Symbol	Definition
$\mathcal{T}$ (n=$\|\mathcal{T}\|$)	Set of Tables	r	Set of tuples
T (T_i)	Table (the i_{th} table in $\mathcal{T}$)	$t[A]$	Value of t on the column A
A ($T.A$)	Column (Column A in table T)	$\mathcal{S}$ (s=$\|\mathcal{S}\|$)	Set of all input tuples
m_i	Arity of Table T_i	$\mathcal{F}$ (f=$\|\mathcal{F}\|$)	Set of output tuples
$\mathcal{A}(T)$	Schema (set of columns) of T	±	Null denoting a missing value
t	Tuple	⊥	Null produced by an operator

Notation. Table 1 summarizes the used notation. For any operator, let $\mathcal{S}$ (and its size s) and $\mathcal{F}$ (and its size f) denote the collective set of all input and output tuples, respectively. We use two types of nulls: ± denotes a *missing null* i.e., missing value from an incomplete input relation to be integrated and ⊥ denotes a *produced null*, a null value that is introduced by an operator during integration.

TID	Stadium	Location	Team	Opened	Coach	Capacity
t_1	NRG Stadium	Texas	Houston Texans	⊥	⊥	⊥
t_2	AT&T Stadium	Texas	Dallas Cowboys	⊥	⊥	⊥
t_3	Paul Brown	Ohio	±	⊥	⊥	⊥
t_4	Sofi Stadium	California	Angeles Chargers	⊥	⊥	⊥
t_5	Soldier Field	Chicago	⊥	1924	⊥	⊥
t_6	Ford Field	Michigan	⊥	2002	⊥	⊥
t_7	⊥	Texas	Houston Texans	⊥	Lovie Smith	⊥
t_8	⊥	Wisconsin	Green Bay Packers	⊥	Matt LaFleur	⊥
t_9	⊥	Michigan	Detroit Lions	⊥	Dan Campbell	⊥
t_{10}	NRG Stadium	Texas	⊥	⊥	⊥	±
t_{11}	Ford Field	Michigan	⊥	⊥	⊥	65k
t_{12}	Lambeau Field	Wisconsin	Green Bay Packers	⊥	⊥	⊥
t_{13}	±	Ohio	Cleveland	⊥	⊥	⊥
t_{14}	Sofi Stadium	California	±	⊥	⊥	⊥

(a) T1 ⊎ T2 ⊎ T3 ⊎ T4 ⊎ T5

OID	TIDs	Stadium	Location	Team	Opened	Coach	Capacity
f_1	$\{t_1, t_7, t_{10}\}$	NRG Stadium	Texas	Houston Texans	⊥	Lovie Smith	±
f_2	$\{t_2\}$	AT&T Stadium	Texas	Dallas Cowboys	⊥	⊥	⊥
f_3	$\{t_3\}$	Paul Brown	Ohio	±	⊥	⊥	⊥
f_4	$\{t_{13}\}$	±	Ohio	Cleveland	⊥	⊥	⊥
f_5	$\{t_4\}$	Sofi Stadium	California	Angeles Chargers	⊥	⊥	⊥
f_6	$\{t_5\}$	Soldier Field	Chicago	⊥	1924	⊥	⊥
f_7	$\{t_6, t_9, t_{11}\}$	Ford Field	Michigan	Detroit Lions	2002	Dan Campbell	65k
f_8	$\{t_8, t_{12}\}$	Lambeau Field	Wisconsin	Green Bay Packers	⊥	Matt LaFleur	⊥
f_9	$\{t_3, t_{13}\}$	Paul Brown	Ohio	Cleveland	⊥	⊥	⊥
f_{10}	t_{14}	Sofi Stadium	California	±	⊥	⊥	⊥

- $FD(T_1, T_2, T_3, T_4, T_5) = \{f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8\}$
- $FD_{tuple\text{-}set}(T_1, T_2, T_3, T_4, T_5) = FD(T_1, T_2, T_3, T_4, T_5) \cup \{f_{10}\}$
- $T_1 \boxplus T_2 \boxplus T_3 \boxplus T_4 \boxplus T_5 = FD(T_1, T_2, T_3, T_4, T_5) - \{f_3, f_4\} \cup \{f_9, f_{10}\}$

(b) Output tuples generated using different operators

Figure 2: Result of integrating the tables in Fig. 1 using different techniques. The table in (a) is the result of outer unioning the five tables. The table in (b) is the union of tuples obtained using FD (first eight tuples in mustard), a variant called tuple-set FD (which is the FD plus f_{10}), and Complement Union (⊞). A unique Output ID (OID) is provided for each output tuple for clarity.

Example 4. *Consider Table T_1 of Fig. 1. The schema of T_1 is $\mathcal{A}(T_1) = \{Stadium, Location, Team\}$. Tuple t_3 = {Paul Brown, Ohio, ±} has attribute value $t_3[Stadium]$ = Paul Brown and a missing null on the Team column, i.e., $t_3[Team] = \pm$.*

2.1 Finding Common Columns

Our running example is unrealistic as common (or in relational terms join-consistent) columns from different tables have the same name and columns that are not common have different names. This is not the case in most realistic examples. Hence, we begin by using schema matching to assign *integration ids* to columns such that two matched columns will have the same id and two columns that are not matched will have different ids. We will ensure no two columns in the same table share an integration id. Accordingly, we will set $\mathcal{A}(T)$ to be the set of integration ids of T's columns (Section 4).

2.2 Integration Operators

We assume that the reader is familiar with the elementary relational algebra operators like union (∪), join (⋈) and outer join(⟗) [66] based on which, we now introduce some (less well known) operators that we use as components of an integration solution.

Outer Union (⊎) is an extension to the union operator. It unions tables even if they do not have the same schema [16]. The outer union between T_1 and T_2 is denoted by $T_1 \uplus T_2$. For each $A \in \mathcal{A}(T_1) - \mathcal{A}(T_2)$, we pad T_2 with a new column A containing nulls (specifically ⊥). Similarly, for each $A \in \mathcal{A}(T_2) - \mathcal{A}(T_1)$, we pad T_1 with a new column A containing nulls. We then union the padded relations.

Example 5. *The outer union of the tables in Fig. 1 is shown in Fig. 2(a). Here, the input size ($|\mathcal{S}| = 14$) is the same as the output size ($|\mathcal{F}| = 14$) but the output may be smaller if there are duplicates.*

Subsumption (β). Given two different tuples t_1 and t_2 having the same schema, the tuple t_1 (subsuming tuple) *subsumes* t_2 (subsumed tuple), denoted by $t_1 \sqsupset t_2$, if all the non-null values of t_2 are equal to that of t_1 on the respective columns and t_1 has fewer null values (either missing or produced) than t_2 [7, 32]. We denote the subsumption operation using β where $\beta(r)$ contains all tuples of r that are not subsumed by another tuple in r. Applying subsumption to the outer union result is called the *minimum union* (⨁) [32]. An example of minimal union (⨁) of tables in Fig. 1 is the set $\{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{11}, t_{12}, t_{13}\}$ in Fig. 2. This is because the tuple t_{10} is subsumed by t_1 and t_{14} is subsumed by t_4. Here, the size of the input ($|\mathcal{S}| = 14$) is larger than the output ($|\mathcal{F}| = 12$).

Complementation (κ). Two different tuples t_1 and t_2 having the same schema *complement* each other if: 1) there is at least one column A on which they have equal and non-null values; 2) for every column A where both tuples are non-null, the tuples must have the same value on A; 3) there is at least one column A on which t_1 is non-null and t_2 is null (missing or produced); and 4) there is at least one column A on which t_2 is non-null and t_1 is null (missing or produced) [7, 9]. The complementation of t_1 and t_2 is a tuple t_3 where for any column A, $t_3[A] = t_1[A]$ if either $t_1[A]$ is non-null or both $t_1[A]$ and $t_2[A]$ are non-null (hence, equal). Otherwise, if $t_2[A]$ is non-null $t_3[A] = t_2[A]$. For the case where both values are null, if $t_1[A] = t_2[A] = \perp$ then $t_3[A] = \perp$ otherwise (at least one of the nulls is missing) $t_3[A] = \pm$. The complementation operator (κ) replaces all complemented pairs of tuples with their complementation. Note that a tuple that results from complementation could be complemented by other tuples so the complementation operator is the iterative result of applying complementation to a relation until it contains no further complementing tuples. Applying complementation over a set of outer unioned tuples, is known as *complement union* (⊞).

Example 6. *In Table(a) of Fig. 2, tuples t_3 and t_{13} complement each other. Their complementation is denoted as f_9 in Fig. 2(b), i.e., $\kappa(t_3, t_{13}) = f_9$. So complementation can* overcombine *tuples that do not agree on all their common attributes. In this example, T_5 asserts that* `Cleveland` *is a team in* `Ohio` *with an unknown stadium while T_1 asserts that* `Paul Brown` *is a stadium in* `Ohio`*. But we do not definitively know that* `Paul Brown` *is the stadium of* `Cleveland`*. The complement union of the tables in Fig. 1, i.e., $T_1 \boxplus T_2 \boxplus \cdots \boxplus T_5$ is the set of tuples in Table (b) of Fig. 2 excluding tuples $\{f_3, f_4\}$. Note that complementation union may not remove all subsumable tuples (e.g. f_{10}, which can be subsumed by f_5, is not complemented by f_5 since the fourth condition of complementation is not met).*

2.3 Full Disjunction

The operators of the previous section offer possible semantics for integrating tables. In 1994, Galindo-Legario proposed a different semantics called Full Disjunction (FD) [32]. His proposal is essentially



Magneto: Combining Small and Large Language Models for Schema Matching

Yurong Liu*
New York University
yurong.liu@nyu.edu

Eduardo H. M. Pena*†
Federal University of Technology Paraná
eduardopena@utfpr.edu.br

Aécio Santos
New York University
aecio.santos@nyu.edu

Eden Wu
New York University
eden.wu@nyu.edu

Juliana Freire
New York University
juliana.freire@nyu.edu

ABSTRACT

Recent advances in language models (LMs) open new opportunities for schema matching (SM). Recent approaches have shown their potential and key limitations: while small LMs (SLMs) require costly, difficult-to-obtain training data, large LMs (LLMs) demand significant computational resources and face context window constraints. We present `Magneto`, a cost-effective and accurate solution for SM that combines the advantages of SLMs and LLMs to address their limitations. By structuring the SM pipeline in two phases, retrieval and reranking, `Magneto` can use computationally efficient SLM-based strategies to derive candidate matches which can then be reranked by LLMs, thus making it possible to reduce runtime while improving matching accuracy. We propose (1) a self-supervised approach to fine-tune SLMs which uses LLMs to generate syntactically diverse training data, and (2) prompting strategies that are effective for reranking. We also introduce a new benchmark, developed in collaboration with domain experts, which includes real biomedical datasets and presents new challenges for SM methods. Through a detailed experimental evaluation, using both our new and existing benchmarks, we show that `Magneto` is scalable and attains high accuracy for datasets from different domains.

PVLDB Reference Format:
Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. `Magneto`: Combining Small and Large Language Models for Schema Matching. PVLDB, 18(8): 2681 - 2694, 2025.
doi:10.14778/3742728.3742757

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/VIDA-NYU/magneto-matcher.

*Equal contribution.
†Work done as a visiting researcher at New York University.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 8 ISSN 2150-8097.
doi:10.14778/3742728.3742757

1 INTRODUCTION

The rapid increase in the volume of structured data– from data published in scientific articles [63, 72] and repositories [29, 60, 66, 80] to open government portals [19, 64] – creates new opportunities to answer important questions through analytics and predictive modeling. But often, data from multiple sources must be integrated to answer these questions. Consider the following example.

Example 1.1. In the context of the National Cancer Institute's Clinical Proteomic Tumor Analysis Consortium (CPTAC) [15], Li et al. [52] carried out a comprehensive proteogenomic analysis of cancer tumors. They collected data from ten studies (published as supplementary material in research papers) that cover multiple patient cohorts and cancer types. To facilitate their analysis, they mapped each dataset into the GDC standard, a data model set by the National Cancer Institute's Genomic Data Commons (GDC) for cancer genomic data [38, 60]. Even though the studies had been carried out by members of the CPTAC effort, the datasets containing patient case and sample data used different representations for variable names and values. Integrating these data required a substantial effort to match variables from each dataset source schemata to the target GDC format, which encompasses over 700 attributes. □

Even though the schema matching problem has been extensively studied [8, 22, 48, 58], matching still requires a time-consuming, manual curation process for complex tasks, which like the one described above, involves ambiguity and heterogeneity in the representation of attributes and values (see Table 1). Schema matching approaches that rely on attribute names, data types and values for similarity assessments are likely to fail for such matches. As a point of reference, we assessed the effectiveness of state-of-the-art strategies for the biomedical datasets from Example 1.1. As shown in Figure 1, they perform poorly: the best technique achieves at most 0.45 of mean reciprocal rank (MRR) and incurs high computational costs. We discuss these results in detail in Section 6.

Determining correspondences between columns may require knowledge beyond the schema and contents of a table. Table 1 shows some possible matches for the proteogenomic analysis. Sometimes, selecting the correct match is difficult even for subject matter experts. For example, for the attribute `patient_age` from one of the datasets, there are at least three plausible matches in GDC: `age_at_diagnosis`, `days_to_birth`, and `age_at_index`. Without additional context, it is difficult to determine the correct match.

Table 1: Examples of candidate matches between source tables and the GDC that highlight schema heterogeneity.

Source Column and Values	Target Candidate Columns
Histologic_Grade_FIGO: FIGO grade 1, FIGO grade 2, FIGO grade 3	**tumor_grade**: G1, G2, G3 **who_nte_grade**: G1, G2, G3 **adverse_event_grade**: Grade 1, Grade 2, Grade 3
Path_Stage_Primary_Tumor-pT: pT1a (FIGO IA), pT2 (FIGO II), pT3b (FIGO IIIB)	**uicc_pathologic_t**: T1a, T2, T3b **ajcc_pathologic_t**: T1a, T2, T3b
Age: 65, 72, 49	**age_at_diagnosis**: n/a **days_to_birth**: n/a **age_at_index**: n/a

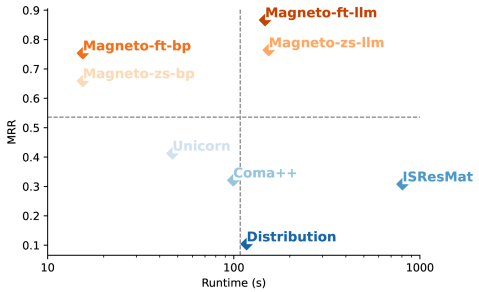


Figure 1: Trade-off between runtime and accuracy (using MRR) for different schema matchers in the task of Example 1.1. `Magneto` variants (orange) outperform traditional methods (blue) in MRR and accuracy-runtime trade-off.

Schema Matching and Language Models. Renewed interest in data integration has emerged due to the capabilities of language models [34, 59]. For schema matching, promising approaches have been proposed [10, 11, 27, 79]. A key challenge in schema matching is estimating the similarity between two columns. Pre-trained Language Models (PLMs), referred to as SLMs in this paper to distinguish them from large language models (LLMs), create column representations (or embeddings) enriched with semantic information. The similarity between two embeddings serve as a proxy for column-matching scores [10, 11]. SLMs have also been fine-tuned for schema matching [27] and general matching tasks [79].

LLMs, in contrast, are trained using large, generic data corpora and thus contain knowledge that can assist in obtaining additional semantics necessary to identify matches. Prompting strategies combined with fine-tuned models have been shown effective to improve table understanding and help in integration tasks [50].

While prior work has shown the usefulness of SLMs and LLMs for schema matching, they also present significant practical challenges:

- *Challenge 1*: Fine-tuning SLMs can lead to significant performance improvements, but this requires the availability of manually curated training data, which can be expensive to create.
- *Challenge 2*: LLMs usually do not need fine-tuning, but face constraints due to fixed context windows, requiring truncation of large prompts and potential loss crucial information. While some models offer larger windows, cost scales with input and output size. Furthermore, accuracy can decline with long prompts and API calls incur high latency, especially for large inputs [51].

Our Approach. We introduce `Magneto`, a framework that combines SLMs and LLMs to derive cost-effective and general schema matching solutions. As illustrated in Figure 2, `Magneto` is structured in two phases: *candidate retrieval* selects a subset of the possible matches; and reranker, which ranks the candidates to make it easier for users to examine and select matches. To address *Challenge 1*, `Magneto` leverages LLMs to automate SLMs fine-tuning. Instead of relying on manually created training data and structure-based augmentation (e.g., row shuffling and sampling [27, 31]), we use LLMs to derive data. Using LLMs, we can add syntactic diversity and capture different representations for column names and their values. `Magneto` addresses *Challenge 2* by using cheaper SLM-based methods for finding candidates thereby reducing the number of matches that need to be checked by more costly LLM-based rerankers. To ensure that all necessary details are included in the prompt while staying within the context window limit, we need to create representative samples of the columns in candidate matches to be ranked by the LLM. In addition, since LLMs are designed for textual data, we must serialize the column content. Selecting the right serialization strategy is still an open research problem that has attracted substantial attention [23, 32]. We explore different alternatives for both sampling and serialization.

Our goal with `Magneto` is to achieve high accuracy for schema matching at a low cost. Figure 1 illustrates the trade-offs between runtime and accuracy for existing schema matching algorithms and different variants of `Magneto`. Note that traditional methods tend to cluster at relatively high accuracy with long runtimes or relatively low runtimes with reduced accuracy. In contrast, `Magneto` variants strike a balance across both dimensions and have the advantage of requiring no training data.

We conduct an extensive experimental study of `Magneto` on various datasets from two benchmarks. The first is a new benchmark, that we developed in collaboration with domain experts, which includes real biomedical datasets. Our experiments show that many existing schema matching solutions struggle with this benchmark, particularly given the complexity and syntactic variability present in its datasets. In contrast, `Magneto` demonstrates superior accuracy and generally faster runtimes. We also evaluate `Magneto` on the Valentine benchmark [48] and observe that it also performs well for the Valentine datasets which cover a variety of domains.

Contributions. Our main contributions are summarized as follows:

- We introduce the `Magneto` framework that effectively combines SLMs and LLMs and allows the creation of schema matching strategies that attain a balance between accuracy and runtime for diverse data and tasks (Section 2).
- We propose an LLM-powered method to generate training data for fine-tuning SLMs for schema matching tasks and a contrastive learning pipeline using triplet loss and online triplet mining to enhance column embedding distinction (Section 3).
- Unlike previous approaches that use LLMs for schema matching, we show that reranking candidate matches with LLMs, derived from less costly methods, achieves high accuracy at a lower cost. (Section 4).
- In collaboration with biomedical researchers, we created a new benchmark that represents a real schema matching effort [52],

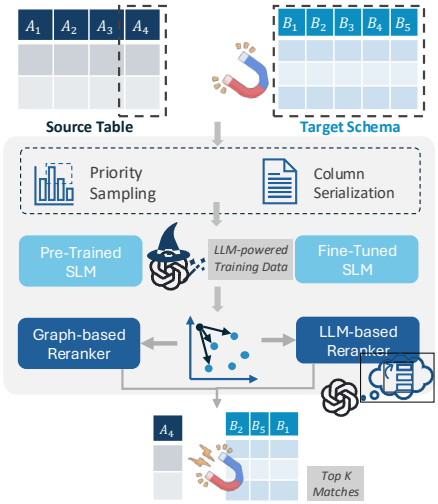


Figure 2: Magneto takes source and target tables and identifies matches in two phases: an SLM retrieves and ranks candidates, then an LLM assesses and reranks them. This enables efficient matching and cross-domain generalization, with customizable strategies based on method combinations.

encompassing characteristics not present in existing benchmarks and introducing new challenges for SM methods (Section 5).

- We perform an extensive experimental evaluation of different strategies derived using our framework, comparing them against SM methods over datasets that cover a wide range of domains. We also carry out ablation studies to assess the effectiveness of different choices for column sampling and serialization (Section 6).

2 MOTIVATION AND SOLUTION OVERVIEW

Biomedical Data Integration: Opportunities and Challenges. Due to substantial investments made in infrastructure to share biomedical data [62], many datasets are now stored in repositories with related research articles [60, 66]. Pooling these data can advance research across various diseases and populations, but current technologies are limited in their ability to integrate these data. Our research was motivated by this challenge as part of a collaboration with biomedical researchers in the context of the ARPA-H Biomedical Data Fabric (BDF) program [2], which aims to develop usable data integration methods and tools.

Our interviews with biomedical experts revealed that current data integration practices rely heavily on manual processes and dataset-specific scripts for schema matching [12, 52, 57]. As illustrated in Example 1.1, this approach is both error-prone and difficult to reproduce, creating significant barriers to biomedical research.

Based on these findings, we identified two critical requirements that guided the design of Magneto. First, researchers need approaches that generalize across the diverse landscape of biomedical data—spanning different data types (genomics, proteomics, electronic health records), disease categories (various cancers, autoimmune and rare diseases), and data sources (publications, hospitals, data commons). Second, given the complexity of biomedical schemas, these approaches must facilitate curation, as even subject matter experts often struggle to determine correct matches.

2.1 Definitions and Evaluation Metrics

Before describing our approach, we introduce the notation, schema matching definition, and evaluation metrics used in this paper.

Definition 2.1. (Schema Matching) Let $\mathbf{S}(A_1, \ldots, A_n)$ be a source table and $\mathbf{T}(B_1, \ldots, B_m)$ be a target table, where $A_i \in \mathbf{S}$ and $B_j \in \mathbf{T}$ are columns that define the schemata. Each column $A \in \mathbf{S} \cup \mathbf{T}$ has an associated *domain*, denoted $\mathcal{D}(A)$, representing the set of possible values that the column can take; note that $\mathcal{D}(A)$ may be empty. Schema matching focuses on aligning the table schemata by establishing correspondences between columns representing the same real-world concept or entity. A matching algorithm (or matcher) aims to identify pairs (A_i, B_j) that represent the same (or semantically equivalent) column based on various factors, such as their domains and names. Thus, a matcher $\mathcal{M}$ can be seen as a function that generates a schema mapping $M \subseteq \mathbf{S} \times \mathbf{T}$, where each element $(A, B) \in M$ represents a correspondence between a source column A and a target column B, where $\mathcal{D}(A) \approx \mathcal{D}(B)$, meaning that the domains of columns A and B are related or overlap. □

Matching algorithms often associate a score with each match they derive. These scores are used to generate ranked lists containing the derived matches, which help users explore matches by prioritizing the highest-scoring candidates. These lists provide a global ranking of best matches (A_i, B_j) among all pairs of possible matches of a given pair of tables $\mathbf{S}$ and $\mathbf{T}$ or a per-column ranking of the best matches B_j for a given source attribute $A_i \in \mathbf{S}$. Thus, a common approach to evaluate schema matching methods is to assess their ability to produce high-quality ranked lists of matches [27, 48]. We use two evaluation metrics: Mean Reciprocal Rank (MRR) and Recall at Ground-Truth Size (Recall@GT), as detailed next.

Definition 2.2. (Mean Reciprocal Rank) Let $matches[A]$ denote the ranked list of matches produced by a schema-matching algorithm for the source column $A \in \mathbf{S}$, and r_A be the position of the first correct target column B within the ordered list $matches[A]$. The reciprocal rank (RR) of an individual column A is the multiplicative inverse of the rank, i.e., $\frac{1}{r_A}$. The *mean reciprocal rank* (MRR) for a table $\mathbf{S}$ is the average RR over the subset of columns $\mathbf{S}'$ that contain a correct match in the ground truth:

$$\text{MRR} = \frac{1}{|\mathbf{S}'|} \sum_{A \in \mathbf{S}'} \frac{1}{r_A}. \tag{1}$$

Intuitively, MRR measures how long it takes to find the first relevant match when examining the ranked list of matches. □

MRR is a standard evaluation metric for ranked lists in search engines and question-answering systems [71, 83]. For schema matching, high-quality results correspond to ranking the most relevant match for each source column as highly as possible. A high MRR score therefore indicates that users can more easily identify correct matches when evaluating candidate matches for a given column.

We also use the recall at ground truth size (Recall@GT), a standard measure used in recent schema matching literature [48]. Unlike MRR, which evaluates rankings per source column, Recall@GT operates on a global ranking that merges all candidate matches across columns into a single list.

Definition 2.3. (Recall@GT) Let *matches* denote the global ranked list containing all matches produced by a matcher that considers all

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00123

Schema Matching using Pre-Trained Language Models

Yunjia Zhang
University of Wisconsin-Madison
yunjia@cs.wisc.edu

Avrilia Floratou
Microsoft
avflor@microsoft.com

Joyce Cahoon
Microsoft
jcahoon@microsoft.com

Subru Krishnan
Microsoft
subru@microsoft.com

Andreas C. Müller
Microsoft
amueller@microsoft.com

Dalitso Banda*
Sentry
dalitso.banda@sentry.io

Fotis Psallidas
Microsoft
fopsalli@microsoft.com

Jignesh M. Patel
University of Wisconsin-Madison
jignesh@cs.wisc.edu

***Abstract*—Schema matching over relational data has been studied for more than two decades. However, the state-of-the-art methods do not address key modern-day challenges encountered in real customer scenarios, namely: 1) no access to the source (customer) data due to privacy constraints, 2) target schema with a much larger number of entities and attributes compared to the source schema, and 3) different but semantically equivalent entity and attribute names in the source and target schemata. In this paper, we address these shortcomings. Using real-world customer schemata, we demonstrate that existing linguistic matching approaches have low accuracy. Next, we propose the Learned Schema Mapper (LSM), a novel linguistic schema matching system that leverages the natural language understanding capabilities of pre-trained language models to improve the overall accuracy. Combining this with active learning and a smart attribute selection strategy that selects the most informative attributes for users to label, LSM can significantly reduce the overall human labeling cost. Experimental results demonstrate that users can correctly match their full schema while saving as much as 81% of the labeling cost compared to manual labeling.**

I. Introduction

More and more organizations in various industries (retail, healthcare, manufacturing, etc.) are adopting a data-driven approach to discover non-trivial and strategic insights. At the same time, the amount of information collected has never been greater [1], highlighting the need for modern data analytics tools that can keep up with the volume, variety, and velocity of data associated with different industries. To that end, multiple companies are offering industry-specific analytics solutions [2]–[4] tailored to meet customers' domain-specific needs. Azure Industry solutions [2], for example, offers a variety of services that aim to optimize industry-specific processes, provide enhanced collaboration capabilities and reduce the time to actionable insights. Enabling these enhanced experiences often requires a substantial understanding of customer data. To tackle this problem, the service operator usually maps the customer's data to a known industry-specific schema (ISS) whose semantics are clearly defined, and then builds tools on top of it to streamline various processes.

*Work done while at Microsoft.

In this paper, we study the schema matching problem in the context of known ISSs and customer data in the form of a relational schema. While the problem has been studied for over two decades, we find that existing solutions are not adequate in our environment.

First, information about customer data is often limited due to privacy considerations. In particular, we find that customers are reluctant to grant access to individual data records [5], [6]. As a result, the service operator cannot leverage customer data records when performing schema matching. The same considerations do not constrain their access to the customer schemata. Indeed, since the outcome of schema matching reveals what entities and attributes in the known ISS are present in the customer data, further obfuscating the customer schema from the service operator is not valuable [1]. As a result, customers are generally willing to provide schema information to the operator to facilitate schema matching.

Second, ISSs are typically large (in terms of entities and attributes) as they try to capture a wide variety of concepts associated with a given industry. The customer schema, however, does not necessarily encapsulate all these concepts and is often much smaller than the target ISS. This discrepancy can complicate the schema matching process as it results in a large number of available (and potentially irrelevant) candidate matches for each attribute of the customer schema.

Finally, the entity and attribute names of the customer schema are often hard to understand due to abbreviations and/or customer-specific terminology, further complicating the automation of schema matching for the service operator.

The externally-imposed customer constraints along with the challenges associated with real customer schemata significantly limit the accuracy of existing schema matching approaches (see Section III). In this work, we propose the Learned Schema Matcher (LSM), a novel human-in-the-loop linguistic schema matching approach. LSM does not require access to the individual data records (data-free schema matching) but only to the source schema, respecting the customer

[1]There are other scenarios where both the source and target schemata need to be obfuscated from the service operator [7]. Extending our approach to meet these more stringent privacy requirements is left for future work.

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00123

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

requirements. At the core of our approach lies a fine-tuned language model that is able to scale to large ISSs and allows us to better handle noise in the source schema. Further, by combining this model with active learning and a smart interaction strategy to obtain targeted feedback from the user, we can fully map a source schema to an ISS while incurring significantly less human labeling cost than prior work.

Our contributions are the following:

- We study the state-of-the-art linguistic schema matching approaches on real-world schemata from Microsoft customers and demonstrate that they have low accuracy.
- We develop LSM, a novel matching approach that relies on a fine-tuned pre-trained language model to tackle data-free schema matching. To the best of our knowledge, this is the first work demonstrating that leveraging the natural language capabilities of pre-trained language models can improve the accuracy of data-free schema matching. Previous methods that are either not data-free [8]–[10] or do not leverage natural language understanding [11]–[15].
- The externally-imposed customer constraints render near-100% accuracy virtually impossible without humans-in-the-loop. We thus propose to combine our schema matching model with an active learning strategy. Our method employs a novel attribute selection strategy to obtain targeted feedback from the user with the goal of reducing the human labeling cost.
- We evaluate LSM on real customer schemata and publicly available datasets. We show that with LSM, users can correctly match their full schema to ISS with a 81% reduction in labeling cost compared to manual labeling.

The remainder of the paper is organized as follows: Section II contains the problem statement. Section III presents an evaluation of the state-of-the-art linguistic matching approaches on real customer schemata. Section IV describes the design of our proposed LSM method. Section V contains our experimental evaluation, and Section VI discusses our experiences in designing LSM. Finally, Section VII presents the related work, and our conclusions are stated in Section VIII.

II. Problem Statement

We now define the problem of schema matching in our environment. Given a source (customer) schema and a known target (ISS) schema that both follow the E/R model [16], our goal is to map each attribute of the source schema to an attribute at the target schema. Figure 1 shows an example of an ISS and a customer schema and their corresponding match.

More formally, we are given a source schema S_s consisting of a set of entities E_s, a set of attributes A_s, and a set of PK/FK relationships R_s. Similarly, we also have a known target schema S_t that consists of a set of entities E_t, a set of attributes A_t, and a set of PK/FK relationships R_t. We assume that each attribute in A_s (A_t) belongs to a single entity in E_s (E_t).

Each entity e ($e \in E_s \cup E_t$) has a name $e.name$, a primary key $e.pk$, and a set of foreign keys $e.fks$. Each attribute a ($a \in A_s \cup A_t$) has a name $a.name$, a data type $a.dtype$, and optionally a natural language description $a.desc$ associated with it.

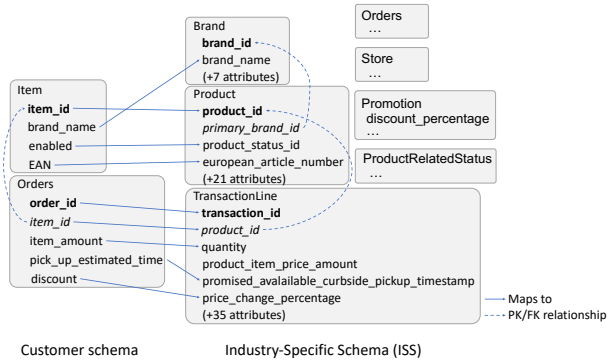


Fig. 1: An example customer schema and corresponding ISS.

We define an attribute correspondence as $r_{ij} = (a_i, a_j)$ where a_i is an attribute in the source schema S_s and a_j is an attribute at the target schema S_t. As discussed in [17], intuitively, an attribute correspondence specifies a relationship between a_i and a_j. The correspondence may specify that the two attributes are equal to each other or that there may be a transformation function involved (e.g., from dollars to euros). In this work, we focus on correspondences that denote equality and leave more complex transformations as part of future work.

Definition 1. *An entity match $\mathcal{M}$ is a triple (e_s, e_t, m), where e_s is an entity in E_s, e_t is an entity in E_t, m is a set of attribute correspondences between e_s and e_t, and each source and target attribute occurs in at most one correspondence in m.*

Based on the above definition, we now define the schema matching problem addressed in this paper.

Definition 2. *The result $\overline{\mathcal{M}}$ of the schema matching process is a set of entity matches between entities in E_s and in E_s, where each attribute in S_s and S_t appears in at most one entity match.*

As we discuss in Section IV, our approach generates $\overline{\mathcal{M}}$ by utilizing all the available information in the source and target schemata (i.e., entities, attributes, and PK/FK relationships).

III. Motivation

Given that in our setting, we do not have access to the customer data, we evaluated the applicability of the state-of-the-art schema matching approaches that rely solely on schema-level information. In this section, we demonstrate that prior approaches have poor accuracy on our real customer schemata. We then identify a set of challenges that motivate the design of the Learned Schema Matcher.

State-of-the-art Methods. We choose four representative schema-based matching methods: CUPID [11], COMA [18],

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

	# Entities	# Attr.	# Unique Attr. Names	# PK/FK	Desc.
Customer A	3	29	28	2	Y
Customer B	8	53	45	7	N
Customer C	3	84	82	2	N
Customer D	7	136	125	7	N
Customer E	25	530	475	24	Y

TABLE I: Statistics on the customers' (source) schemata.

		# Entities	# Attributes	# PK/FK
RDB-star	Source	13	65	12
	Target	5	34	4
IPFQR	Source	1	51	0
	Target	1	67	0
MovieLens-IMDB	Source	6	19	5
	Target	7	39	6

TABLE II: Statistics on publicly available schemata.

S-MATCH [12], and Similarity Flooding [15], which are all heuristic-based. To evaluate the effectiveness of machine learning-based approaches, we also experimented with the LSD [13] and MLM [14] methods. Note that these methods require access to both the schema and the data records, which is not possible in our environment. We thus had to adapt them to leverage schema-level information only. We implemented all the methods except COMA [18] from scratch according to the descriptions in [11]–[14] respectively. We use the latest community edition of COMA 3.0 [2]. Note that COMA and CUPID can optionally accept user feedback during the matching process. In this section, we present results without user feedback, but we do use the interactive mode in our experiments in Section V where we present a comparison with our approach. A summary of the methods is presented below:

- ***CUPID:*** CUPID tackles the schema matching problem for both XML and relational schemata by combining both linguistic (through a synonym dictionary) and structural information and computing the weighted sum of the associated similarities. In our implementation, we use the pre-trained word embedding from FastText [19] as the synonym dictionary and generate the similarity score using cosine similarity. For each customer schema, we search the best-performing weights for the weighted sum and report only the best results.
- ***COMA:*** COMA uses a set of matchers to compute the linguistic similarity between two attributes. The matchers cover a broad spectrum of similarity metrics such as affix, n-gram, Soundex, edit distance, etc. To combine the similarities, COMA can choose from various aggregation functions such as min, max, average, etc.
- ***S-MATCH:*** S-MATCH (SM) solves the semantic matching problem on two trees. It uses WordNet [20] to understand the meaning of the nodes in the trees and identify synonyms. It covers multiple semantic relations such as equivalence, mismatch, overlapping, etc. We use S-MATCH to tackle the schema matching problem as defined in Definition 2 and thus consider only attribute equivalence.

[2]https://sourceforge.net/projects/coma-ce/

	CUPID	COMA	SM	SF	LSD	MLM
RDB-Star	0.96	**1.00**	**1.00**	0.70	0.26	**1.00**
IPFQR	**1.00**	0.98	0.82	**1.00**	0.08	0.98
MovieLens-IMDB	0.64	0.54	**0.72**	0.71	0.00	0.64
Customer A	0.18	**0.21**	0.07	0.11	0.00	0.11
Customer B	**0.14**	0.02	0.06	0.02	0.00	0.08
Customer C	0.08	0.22	0.11	**0.23**	0.00	0.14
Customer D	0.27	**0.31**	0.13	0.14	0.00	0.14
Customer E	0.27	0.17	**0.28**	0.12	0.00	0.14

TABLE III: The top-3 accuracy of six state-of-the-art approaches on our schemata.

- ***Similarity Flooding:*** Similarity flooding (SF) [15] builds a pairwise connectivity graph based on the structure of the source and target schemata. The connectivity graph contains element pairs as nodes. Two nodes (x_1, y_1) and (x_2, y_2) are connected if x_1, x_2 and y_1, y_2 are neighbors in the original schema graphs respectively. SF initializes the similarity scores of the element pairs and propagates the scores for each element pair following the edges in the connectivity graph. In our implementation, we use embedding similarities as the initial scores.
- ***LSD:*** The LSD method is one of the first to leverage machine learning for schema matching. LSD learns to match between the source and target schemata based on a set of provided examples. LSD has four individual learners: 1) the `whirl` learner, which classifies the attributes based on nearest neighbors of TF-IDF encodings, 2) a naive Bayesian learner that classifies the descriptions, 3) a name matcher based on name similarities, and 4) a county-name recognizer that matches the names if the attribute is a county name. We randomly select 50% of the ground truth matches as the training set and measure the matching quality for the rest 50% of the attributes.
- ***MLM:*** MLM is another machine learning-based method. Different from LSD, MLM featurizes the candidate matches using both the schema specifications and the data records. Following the featurization, MLM generates matches with a clustering model (K-means or self-organizing map [14]). To apply MLM in our problem setting, we only use the features from the schema-level specification. Since MLM adopts unsupervised learning, all the attributes in the target (ISS) schema are treated as the training set.

Datasets. We use five real-world schemata from Microsoft customers in retail. The characteristics of these schemata are shown in Table I. As shown in the table, the five schemata are of different sizes: Customer A has the smallest schema with 29 attributes spread across three entities, while Customer E has the largest schema with 25 entities and 530 attributes. Among the five customer schemata, two of them contain attributes with natural language descriptions. The corresponding target ISS is also defined for the retail industry, which encapsulates information about customers, goods, stores, promotions, sales, and other assets that are typically tracked. The retail ISS we consider consists of 92 entities, 1218 attributes, and 184 PK/FK relationships. The ground truth matches are obtained from domain experts.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

FlexER: Flexible Entity Resolution for Multiple Intents

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Northeastern University, MA, USA
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel

Entity resolution, a longstanding problem of data cleaning and integration, aims at identifying data records that represent the same real-world entity. Existing approaches treat entity resolution as a universal task, assuming the existence of a **single** interpretation of a real-world entity and focusing only on finding matched records, separating corresponding from non-corresponding ones, with respect to this single interpretation. However, in real-world scenarios, where entity resolution is part of a more general data project, downstream applications may have varying interpretations of real-world entities relating, for example, to various user needs. In what follows, we introduce the problem of *multiple intents entity resolution* (MIER), an extension to the universal (single intent) entity resolution task. As a solution, we propose FlexER, utilizing contemporary solutions to universal entity resolution tasks to solve multiple intents entity resolution. FlexER addresses the problem as a multi-label classification problem. It combines intent-based representations of tuple pairs using a multiplex graph representation that serves as an input to a graph neural network (GNN). FlexER learns intent representations and improves the outcome to multiple resolution problems. A large-scale empirical evaluation introduces a new benchmark and, using also two well-known benchmarks, shows that FlexER effectively solves the MIER problem and outperforms the state-of-the-art for a universal entity resolution.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity resolution, entity matching, graph neural networks, supervised learning

ACM Reference Format:
Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. FlexER: Flexible Entity Resolution for Multiple Intents. *Proc. ACM Manag. Data* 1, 1, Article 42 (May 2023), 27 pages. https://doi.org/10.1145/3588722

1 INTRODUCTION

An essential component of any data science lifecycle involves data preparation. Accordingly, contemporary data projects intensively invest in large-scale data cleaning and integration techniques to combine data from multiple heterogeneous sources into meaningful and valuable information. Consider, for example, a data-intensive organization (*e.g.*, an online shopping company) that continuously collects data on user queries to better understand their needs. The (mostly implicit) intents of users may vary. While some users are interested in basketball shoes, regardless of their brand, others may be more interested in buying branded products, caring less whether shoes are designated as running shoes.

At the heart of the data preparation realm lies the *entity resolution* task (with variations of entity matching, record linkage, deduplication, and more), which has been extensively studied over the

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il; Roee Shraga, Northeastern University, Boston, MA, USA, r.shraga@northeastern.edu; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART42 $15.00
https://doi.org/10.1145/3588722

Table 1. Amazon Product Dataset Sample (D_{ex})

r_{id}	**Product title**
r_1	Nike Men's Lunar Force 1 Duckboot
r_2	NIKE Men Lunar Force 1 Duckboot, Black/Dark Loden-BROGHT Crimson
r_3	NIKE Men's Air Max Stutter Step Ankle-High Basketball Shoe
r_4	Nike Men's Air Max 2016 Running Shoe
r_5	adidas Performance Men's D Rose 6 Boost Primeknit Basketball
r_6	The Man Who Tried to Get Away

past decades (see books and surveys [8, 13, 17]). Entity resolution is a data integration task that aims at identifying same real-world entities that are represented by different data records (instances). Specifically, entity resolution can be used to "clean" a dataset from duplicate tuples referring to the same entity, offering a useful post processing tool for integrating multiple data sources.

Entity resolution techniques for resolving duplicates in a dataset were developed over years of research. Traditional methods focused on string similarity [22, 23, 32, 37] and rule-based methods [54, 55]. Learning-based approaches were also suggested [3, 31], followed by deep learning methods in recent years [15, 16, 25, 28, 34, 41, 64]. Following a common practice for text processing, the use of *pre-trained language models*, and specifically, BERT-based models [10], was also introduced for entity resolution [5, 33, 35, 46].

The aforementioned methods share a common assumption regarding the universality of the entity resolution problem, assuming the existence of a **single** interpretation to the notion of a real-world entity. In particular, most solutions aim to create a single clean view of a dataset, by separating corresponding from non-corresponding record pairs, with respect to this single interpretation.

The universal property of entity resolution solutions is being challenged by the development of applications that require the ability to provide a fine-grained (or personalized) analysis, offering services that are tailored to specific user needs and requirements [4, 62]. For example, online shops aim at personalizing shopping experience to the needs of individual users, having to infer such needs from mainly implicit feedback [29], such as clicks and mouse movements. In such a setting, having a universal resolution facility is likely to cripple the resolution process, failing to cater to the needs of all applications at all times. To illustrate the need for a non-universal (flexible) resolution, we use the *AmazonMI* dataset, a newly suggested benchmark based on amazon products data (see Section 5.1), in the following motivating example.

1.1 Motivating Example: From Record Duplication to Entity Resolution Interpretation

Table 1 depicts an excerpt of the AmazonMI dataset to illustrate the differences between the universal approach to entity resolution and the flexible entity resolution we present and address in this work. Figure 1 offers visual illustration to the multiplicity of entity interpretations in the example.

Record duplication is usually the result of discordant representations (*e.g.,* multi-lingual, synonyms, capitalizations), changes in the data over time, typos, *etc.* For example, records r_1 and r_2 in Table 1 refer to the same pair of Nike basketball shoes called "Men's Lunar Force 1 Duckboot." The difference between r_1 and r_2 originates from capitalization issues (Nike vs. NIKE) and additional specification (*e.g.,* color). Contrarily, r_1 and r_6 will most likely be conceived as different, as the latter refers to a book.

The pair (r_1, r_3) is an example of a pair that, under a certain interpretation, does not refer to the same entity, representing two different Nike basketball shoe variants. Yet, under a different interpretation, both r_1 and r_3 are Nike basketbsall shoes. Similarly, r_1 differs from r_4 and r_5 since they represent a different type of Nike shoes (basketball vs. running) and basketball shoes of different brands (Nike vs. Adidas), respectively, but may be determined to be representatives of the same entity (Nike shoes and basketball shoes, respectively) under a broader interpretation. Such different interpretation may be attributed to different user tastes or different contexts. For example, a pro basketball player would be more sensitive to differences between basketball shoes than a user that seeks shoes for a neighborhood afternoon fun game.

Over the years, entity resolution solutions resolved a single entity interpretation, detecting r_1 and r_2 as duplicates, to create a (single) clean dataset, *e.g.,* by only preserving r_1. The multiple intent phenomenon challenges any downstream application that requires entity matching and involves personalization to user's needs. Beyond online shopping, multiple intents can be frequently found in the domain of recommendation systems, where user's implicit feedback (*e.g.*, item selection and search queries) can serve in understanding a user's intent for better responding to her needs [29].

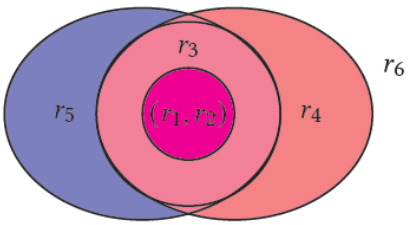


Fig. 1. Possible entity resolution solutions for different entity interpretations over the tuples from Table 1.

1.2 Main Contributions

The motivating example offers an intuitive description of a scenario where a single dataset may serve as a basis for multiple clean views to be generated by an entity resolution solution, suggesting different interpretations of an entity in each such view. We use the term *intent* (formally defined in Section 2.2) to reflect user preferences that, in the scope of this paper, relate to the interpretation of the output view. We are, in particular, interested in intents that are unknown apriori and therefore cannot be constructed using data that exists in the database. Rather, such intents are known only through the training set that labels tuple pairs to be matched under a given interpretation. A typical such scenario can be found in recommendation systems where feedback is collected from users, either implicitly or explicitly, to be used to indicate intent. Another possible scenario may be motivated using machine learning algorithm for entity matching, where different tuning parameters yield different outcomes even for the same algorithm. For example, see the use-case of Yad VaShem, the Jewish Holocaust museum in Jerusalem [51].

In this work, we define the problem of *multiple intents entity resolution* (MIER), an extension to the universal (single intent) entity resolution task. The MIER task considers multiple intents when creating a solution for potential downstream applications that involve entity resolution problems. In the absence of human interpretation to intents (recall the use of implicit feedback in recommendation systems to indicate intent), we aim at training a model to offer such interpretation based on training data and intent cross learning. We propose FlexER, a solution to the MIER problem that utilizes contemporary solutions to universal entity resolution tasks (*e.g.,* DITTO [35]). FlexER positions the problem as a multi-label classification problem and combines intent-based

Ground Truth Inference for Weakly Supervised Entity Matching

RENZHI WU, Georgia Institute of Technology, USA
ALEXANDER BENDECK, Georgia Institute of Technology, USA
XU CHU, Georgia Institute of Technology, USA
YEYE HE, Microsoft Research, USA

Entity matching (EM) refers to the problem of identifying pairs of data records in one or more relational tables that refer to the same entity in the real world. Supervised machine learning (ML) models currently achieve state-of-the-art matching performance; however, they require a large number of labeled examples, which are often expensive or infeasible to obtain. This has inspired us to approach data labeling for EM using *weak supervision*. In particular, we use the labeling function abstraction popularized by Snorkel, where each labeling function (LF) is a user-provided program that can generate many noisy match/non-match labels quickly and cheaply. Given a set of user-written LFs, the quality of data labeling depends on a *labeling model* to accurately infer the ground-truth labels. In this work, we first propose a simple but powerful labeling model for general weak supervision tasks. Then, we tailor the labeling model specifically to the task of entity matching by considering the EM-specific transitivity property.

The general form of our labeling model is simple while substantially outperforming the best existing method across ten general weak supervision datasets. To tailor the labeling model for EM, we formulate an approach to ensure that the final predictions of the labeling model satisfy the transitivity property required in EM, utilizing an exact solution where possible and an ML-based approximation in remaining cases. On two single-table and nine two-table real-world EM datasets, we show that our labeling model results in a 9% higher F1 score on average than the best existing method. We also show that a deep learning EM end model (DeepMatcher) trained on labels generated from our weak supervision approach is comparable to an end model trained using tens of thousands of ground-truth labels, demonstrating that our approach can significantly reduce the labeling efforts required in EM.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity matching, weak supervision, labeling model, transitivity

ACM Reference Format:
Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. https://doi.org/10.1145/3588712

1 INTRODUCTION

Entity matching (EM) refers to the process of determining if a pair of records from two data sources refer to the same real-world entity. EM has many applications, for example, in matching product listings for competitive pricing [2] and in building knowledge graphs [28]. As a long-standing problem, EM has been extensively studied (e.g., see surveys [18, 29, 30, 34, 36, 76]).

Authors' addresses: Renzhi Wu, Georgia Institute of Technology, Atlanta, USA, renzhiwu@gatech.edu; Alexander Bendeck, Georgia Institute of Technology, Atlanta, USA, abendeck3@gatech.edu; Xu Chu, Georgia Institute of Technology, Atlanta, USA, xu.chu@cc.gatech.edu; Yeye He, Microsoft Research, Redmond, USA, yeyehe@microsoft.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART32 $15.00
https://doi.org/10.1145/3588712

The Need for Weak Supervision in EM. The most important requirement for EM solutions is high matching quality as it directly affects downstream application performance. State-of-the-art EM solutions that report the highest matching quality [29, 50, 59, 60] are supervised machine learning (ML) approaches that train a binary classifier to predict the label (match or non-match) for any tuple pair. However, these models require large numbers of labeled tuple pairs, which are often not available or costly to obtain [29]. The high human cost in data labeling has become a main bottleneck in adopting high-quality EM solutions in practice.

To meet the needs of data-hungry supervised ML models, ML practitioners have increasingly turned to *weak supervision* methods, in which a larger volume of cheaply generated, but often noisier, labeled examples is used in lieu of hand-labeled examples. Different forms of weak supervision have been investigated, including the use of non-expert crowd workers [33], pretrained models [24], and rules/patterns/heuristics [75].

To unify different forms of supervision, the *data programming* paradigm [69] has been proposed, in which users write *labeling functions* (LFs) to programmatically label training data, rather than manually labeling each example by hand. Each LF is a small user-provided program (e.g., in Python) that leverages noisy signals in the data or domain knowledge to provide a label (or abstain) for an input example. In binary classification tasks such as EM, the output of each LF is +1 (positive class), -1 (negative class), or 0 (abstain). The data programming approach was first implemented in Snorkel [66], a weakly supervised data labeling system. Snorkel allows users to write LFs and then uses a generative model to combine all LFs to produce probabilistic labels. The data programming approach has been widely adopted in various ML tasks [31, 32, 53, 54, 73, 81, 85].

Example Labeling Functions. The LFs shown in Figure 1 are two user-written LFs developed using an existing tool [84] for matching electronic products [1]. The first LF, "name_overlap", encodes the intuition that matching products should have similar "name" attributes. Specifically, pairs of products with sufficiently high/low word overlap in their names are predicted as matches (+1)/non-matches (-1). When the amount of word overlap does not provide conclusive evidence either way, the LF abstains (0). The second LF, "size_unmatch", uses a regular expression to search the screen size in patterns like "... Samsung **40'** LCD ...". When the screen sizes of a pair of products are different, the LF predicts the pair to be non-match (-1); otherwise, the LF abstains (0).

```
@labeling_function
def name_overlap(tpl_l, tpl_r):
    x = lower(tpl_l.name)
    y = lower(tpl_r.name)
    x = set(splitBySpace(x))
    y = set(splitBySpace(y))
    itsect = x.intersection(y)
    max_len = max(len(x), len(y))
    score = len(itsect) / max_len
    if score < 0.1:
        return -1
    elif score > 0.8:
        return 1
    else:
        return 0
```

```
@labeling_function
def size_unmatch(tpl_l, tpl_r):
    l_name_des = str(tpl_l.name) + " " \
                 + str(tpl_l.description)
    r_name_des = str(tpl_r.name) + " " \
                 + str(tpl_r.description)
    x = re.findall(r"[\d\.]+\'", l_name_des)
    y = re.findall(r"[\d\.]+\'", r_name_des)
    if len(x) == 0 or len(y) == 0:
        return 0
    x, y = x[0], y[0]
    if x != y:
        return -1
    else:
        return 0
```

Fig. 1. Two user-written LFs for the abt-buy dataset [1].

The predictions of multiple LFs can then be used to construct a *labeling matrix* X, where each row corresponds to a tuple pair, and each column corresponds to predictions of one LF for all tuple pairs. After constructing the labeling matrix, a *labeling model* is used to consolidate the labeling function predictions into a final label for each tuple pair. Figure 2 shows one instance of a labeling matrix with votes from LFs (LF1, LF2, ...) and inferred labels from a naive labeling model (+1 for match, -1 for non-match) as well as the unknown ground truth (GT) labels for each tuple pair.

Given a set of user-written LFs, the quality of data labeling depends on the *labeling model* that infers the ground-truth labels from the LFs. In this paper, we first present an embarrassingly simple

	LF1	LF2	LF3	...	Naïve Inferred Label	GT
(t_1, t_2)	1	1	1	...	1	1
(t_1, t_3)	1	1	-1	...	1	-1
(t_1, t_4)	1	1	0	...	1	1
...	...	...	...	...	...	...
(t_2, t_3)	0	-1	-1	...	-1	-1
(t_2, t_4)	0	0	-1	...	-1	1

Fig. 2. Example of a labeling matrix

yet highly effective labeling model (SIMPLE) for general weak supervision tasks. We then specialize the labeling model (SIMPLE-EM) for the task of EM by considering the transitivity property.

(1) Labeling Model for General Weak Supervision Tasks: Existing methods are mostly generative models, i.e., they model the process of generating the predictions of each LF from the hidden ground-truth labels [26, 32, 52, 68]. Under an assumed generative process, one could measure the likelihood of observing all LFs (i.e., the labeling matrix) and an estimation of the ground-truth labels can be obtained by maximizing the likelihood [26, 32, 52, 68]. A fundamental limitation of these approaches is the need to assume that the generative process takes certain simplified forms for ease of modeling or to make the model mathematically solvable. Such assumptions include the Markov assumption [26, 32, 68] and the mixture of independent sub-types assumption [52]. Evidently, these assumptions can often break in practice. Recent benchmark studies [86] suggest that the best existing labeling model for general weak supervision tasks is only 1.5% better than majority vote, and the second best method is already worse than majority vote. In addition, even with these various assumptions to simplify the generative process, the existing approaches are typically quite complicated and difficult to implement.

We take a fundamentally different approach that is a significant departure from the conventional wisdom that generative models should be used in labeling models. The core intuition of our method is that *labeling models are functions*. Conceptually, every labeling model takes in the labeling matrix X and predicts the label vector $\hat{\boldsymbol{y}}$. For example, X could be the sub-matrix in Figure 2 with columns of all LFs and $\hat{\boldsymbol{y}}$ could be the naive inferred label column in Figure 2. Designing a labeling model is to design a function G parameterized by Θ, and $\hat{\boldsymbol{y}} = G(X, \Theta)$ would be the predicted labels of the model. The prediction of the whole matrix X can be made by independently predicting each individual row/data point; i.e., $\hat{y_i} = g(X[i, :], \theta)$ where θ is the parameter of g. Different designs of existing labeling models can be seen as the function g taking different forms.

As discussed, existing labeling models typically build complicated models (e.g., probabilistic graphical models [27, 41, 55] and matrix completion models [38, 67]) by making various assumptions to handcraft g. In this paper, we ask a fundamental question that is contrary to the conventional wisdom: *Can we use a generic classifier (e.g., random forest) as g to be the labeling model and avoid all the complicated designs in the existing approaches?* We provide an affirmative answer to this question by presenting an embarrassingly simple method (in Section 3) that achieves better performance than the existing approaches across ten datasets.

(2) EM-specific Labeling Model with Transitivity: The *transitivity property of EM* states that for any three tuples t_i, t_j, and t_k, if t_i matches t_j and also matches t_k, then t_j has to match t_k. Concretely, consider the scenario in Figure 2 – a naive model would likely predict the tuple pair (t_2, t_4) as non-match, despite strong evidence that both (t_1, t_2) and (t_1, t_4) should be matches, which results in inconsistent label assignments that violate transitivity. Our goal is to explicitly model transitivity using constraints, so that matching decisions can be made in a holistic manner.

A naive approach to enforce transitivity is to do a post-processing step, which is sub-optimal. Intuitively, the additional signals from the transitivity constraints should be made an integral part of the labeling model to improve its accuracy. We thus attempt to incorporate transitivity directly in the labeling model. However, this makes the labeling model complex and difficult to solve.

Progressive Entity Matching: A Design Space Exploration

JAKUB MACIEJEWSKI, National and Kapodistrian University of Athens, Greece
KONSTANTINOS NIKOLETOS, National and Kapodistrian University of Athens, Greece
GEORGE PAPADAKIS, National and Kapodistrian University of Athens, Greece
YANNIS VELEGRAKIS, University of Trento & Utrecht University, The Netherlands

Entity Resolution (ER) is typically implemented as a batch task that processes all available data before identifying duplicate records. However, applications with time or computational constraints, e.g., those running in the cloud, require a progressive approach that produces results in a pay-as-you-go fashion. Numerous algorithms have been proposed for Progressive ER in the literature. In this work, we propose a novel framework for Progressive Entity Matching that organizes relevant techniques into four consecutive steps: (i) filtering, which reduces the search space to the most likely candidate matches, (ii) weighting, which associates every pair of candidate matches with a similarity score, (iii) scheduling, which prioritizes the execution of the candidate matches so that the real duplicates precede the non-matching pairs, and (iv) matching, which applies a complex, matching function to the pairs in the order defined by the previous step. We associate each step with existing and novel techniques, illustrating that our framework overall generates a superset of the main existing works in the field. We select the most representative combinations resulting from our framework and fine-tune them over 10 established datasets for Record Linkage and 8 for Deduplication, with our results indicating that our taxonomy yields a wide range of high performing progressive techniques both in terms of effectiveness and time efficiency.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: Progressive Entity Resolution, Nearest Neighbor Search, Blocking, Sorted Neighborhood

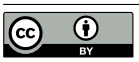
ACM Reference Format:
Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. 2025. Progressive Entity Matching: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 65 (February 2025), 25 pages. https://doi.org/10.1145/3709715

1 Introduction

Entity Resolution (ER), often also referred to as Record Linkage (a.k.a. Clean-clean ER) or Deduplication (a.k.a. Dirty ER), is a fundamental task in data management [24]. It deals with the challenge of identifying and linking data structures, typically referred to as *entity profiles*, that represent the same real-world object [7] The linked structures are referred to as duplicates. Detecting the duplicates is crucial for boosting the performance of a wide range of data management tasks, from recommendation to question answering.

An ER application in order to be successful, needs to overcome a series of challenges [3, 7, 13]. The first is *Volume*. ER algorithms should scale to thousands or even millions of entity profiles. This isn't straightforward, due to the the inherently quadratic nature of ER, which is why it is

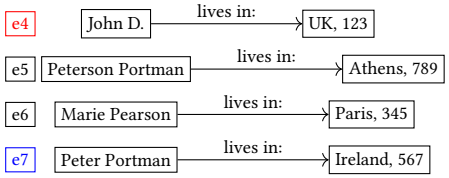
Authors' Contact Information: Jakub Maciejewski, National and Kapodistrian University of Athens, Greece, sdi1700080@di.uoa.gr; Konstantinos Nikoletos, National and Kapodistrian University of Athens, Greece, k.nikoletos@di.uoa.gr; George Papadakis, National and Kapodistrian University of Athens, Greece, gpapadis@di.uoa.gr; Yannis Velegrakis, University of Trento & Utrecht University, The Netherlands, i.velegrakis@uu.nl.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/2-ART65
https://doi.org/10.1145/3709715

id	First name	Last name	City	Zip Code
e1	John	Doe	London	123
e2	Peter	Portman	Dublin	567
e3	Donald	Hirsh	New Jersey	486

(a) Data source D_1



(b) Data source D_2

(e1, e4), (e2, e7), (e2, e5), ..., (e3, e6)

(c) Ordered set of candidate pairs

Fig. 1. Two clean data sources and the candidate pairs generated by a Progressive Entity Matching approach.

addressed through filtering [32]. The second is *Variety*. ER should seamlessly apply to data sources that vary in format, schema and structure. Yet, most ER approaches are crafted either for structured, e.g., relational data or semi-structured, e.g., RDF [29]. The third is *Velocity*. Most ER solutions operate in an offline mode, i.e., they produce results only after having processed all the input data. Unfortunately, there are cases where there are restrictions and cost in computational or temporal resources. An example is the cloud environment. In these cases, partial results are required within a specific time frame [9–11, 45].

To address these challenges, we focus on *Schema-agnostic Progressive Entity Matching*. Velocity is addressed by the progressive functionality, which yields results before processing all input data through a pay-as-you-go functionality. Volume is addressed by Filtering, which restricts the computational cost to the most similar entity profiles, disregarding those dissimilar. Variety is addressed by the schema-agnostic functionality, which represents every entity profile through a concatenation of all attribute values, regardless the respective attribute names.

Consider Figure 1 that illustrates a Record Linkage case, i.e., a case of two data sources D_1 and D_2, each one being duplicate-free, meaning that it contains no matching entity profiles, but there are duplicates across the sources. For instance, e_1 matches to e_4 and e_2 to e_7. The two sources need to be merged, and to do so, the duplicates need to be detected. Furthermore, the two data sources exhibit high variety, varying both in format (structured vs semi-structured) and in schema. Typically, an ER process consists of three steps [3, 13]: Filtering, Entity Matching and Clustering. Yet, due to the small size of this example, Filtering is omitted, while Clustering lies out of the scope of our work. Therefore, we exclusively focus on Entity Matching, where a typical batch algorithm considers all pairs of entities, returning $e_1 \equiv e_4$ and $e_2 \equiv e_7$ after processing the entire data sources, i.e., after examining 12 pairs. In contrast, Progressive Entity Matching defines a processing order, as in Figure 1(c), so as to promote the most likely matches and detect most of them even if the processing is terminated prematurely.

Most existing progressive methods are independent of matching decisions, defining a *static* processing order [34, 45, 54]. Yet, they have some limitations: (i) They disregard recent advances in Filtering: they exclusively consider (meta-)blocking as a pre-processing step for addressing volume, but recent studies show that nearest neighbor search achieves a much higher performance [25, 35, 50]. (ii) They disregard recent advances in natural language processing and the dominance of the semantic entity representations that stem from pre-trained language models in both ER tasks, i.e., blocking and matching. Instead, they exclusively rely on the syntactic representation of entity profiles [25, 49, 50, 52, 56]. (iii) There is no generic framework that unifies the traditional, syntactic- and blocking-based approaches with the latest, semantic-based ones that leverage nearest neighbor search.

To address these shortcomings, we propose a novel framework for Progressive Entity Matching that is generic enough to cover any type of entity representation and of Filtering methods. Our goal is not to combine the outcome of different techniques (e.g., the syntactic- and the semantic-based),

but to propose a set of generic, integrated steps that offer a series of options to researchers and practitioners, facilitating the construction of progressive pipelines that leverage diverse methods in a seamless way. To the best of our knowledge, no other framework offers this in a unifying way.

Our design space consists of four consecutive steps: (i) Filtering restricts the computational cost to the most similar pairs of candidates. (ii) Weighting assigns a similar score to each candidate pair that is proportional to the matching likelihood of the entity profiles comprising it. (iii) Scheduling leverages the similarity scores in order to define the optimal processing order that gives precedence to the duplicate pairs over the non-matching ones. (iv) Matching analytically examines each candidate pair to decide whether its entity profiles are matching or not.

We explain how the filtering step of our framework incorporates all the existing state-of-the-art approaches, from blocking- and sorting-based to those leveraging nearest neighbor search. For each approach, we discuss the corresponding weighting functions for syntactic and semantic representations. For scheduling, we propose four main algorithms based on the concept of the similarity graph [29], which includes a node for each input entity with an edge connecting each candidate pair. We stress that the existing Progressive Entity Matching approaches cover only a small portion of those generated by our framework. We also stress that the matching process lies outside the scope of this work, since it is an orthogonal issue [45, 54], with numerous recent state-of-the-art solutions based on Deep Learning [8, 49].

To test the performance of our framework, we perform a grid search to fine-tune a wide range of end-to-end pipelines over a set of 10 well-established real-world datasets for Record Linkage and of 8 for Deduplication. The experimental results indicate the best combination of filtering and scheduling algorithms in terms of effectiveness. We compare the best configuration of each filtering approach with the state-of-the-art from the literature, i.e., DeepBlocker [50] and Sparkly [35]. The former leverages semantic representations and the latter syntactic ones. This is the first time the two algorithms are applied on Progressive Entity Matching. Our results indicate that DeepBlocker consistently underperforms our semantic-based nearest-neighbor approach both with respect to effectiveness and to time efficiency. Sparkly is much faster than our syntactic-based nearest neighbor approach (due to its Spark-based parallelization), but at the same time, it is significantly less accurate. We also consider an additional state-of-the-art technique as baseline method: I-PES [11].

Overall, we make the following contributions:

(1) We introduce a generic framework for Progressive Entity Matching consisting of four steps, that organize the existing approaches, and giving rise to some new. Among the latter are the first progressive methods leveraging nearest-neighbor search.
(2) We analytically explain the configuration space of each progressive method generated by our framework. Some of the new methods are the first to apply pre-trained language models to Progressive Entity Matching.
(3) We perform an extensive experimental analysis of all Progressive Entity Matching approaches and configurations (using grid search) over 18 well-established datasets. Our experiments give valuable insights into the relative performance of the filtering and scheduling algorithms and demonstrate the superiority of our approaches over the state-of-the-art in the literature.

The implementation of our approaches is available online at: https://github.com/JacobMaciejewski/PER-Design-Space-Exploration.

2 Related Work

Progressive Entity Matching methods can be distinguished in two main categories, the *static* and the *dynamic* [29]. The former generate a processing order that is independent of the matching decisions,

The Battleship Approach to the Low Resource Entity Matching Problem

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Worcester Polytechnic Institute, USA

Entity matching, a core data integration problem, is the task of deciding whether two data tuples refer to the same real-world entity. Recent advances in deep learning methods, using pre-trained language models, were proposed for resolving entity matching. Although demonstrating unprecedented results, these solutions suffer from a major drawback as they require large amounts of labeled data for training, and, as such, are inadequate to be applied to *low resource entity matching* problems. To overcome the challenge of obtaining sufficient labeled data we offer a new *active learning* approach, focusing on a selection mechanism that exploits unique properties of entity matching. We argue that a distributed representation of a tuple pair indicates its informativeness when considered among other pairs. This is used consequently in our approach that iteratively utilizes space-aware considerations. Bringing it all together, we treat the low resource entity matching problem as a Battleship game, hunting indicative samples, focusing on positive ones, through awareness of the latent space along with careful planning of next sampling iterations. An extensive experimental analysis shows that the proposed algorithm outperforms state-of-the-art active learning solutions to low resource entity matching, and although using less samples, can be as successful as state-of-the-art fully trained known algorithms.

CCS Concepts: • **Information systems** → **Entity resolution**; • **Computing methodologies** → **Active learning settings**.

Additional Key Words and Phrases: entity resolution, entity matching, active learning

ACM Reference Format:
Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4 (SIGMOD), Article 224 (December 2023), 25 pages. https://doi.org/10.1145/3626711

1 INTRODUCTION

Entity matching, with variations as entity resolution, record linkage, record deduplication and more, is a core data integration task that plays a crucial role in any data project life cycle [8, 15]. Essentially, entity matching aims to identify dataset tuples that refer to the same real-world entities.

Traditional methods for entity matching and its variations focused on string similarity [20, 21, 27, 31] and probabilistic approaches [10], followed by rule-based methods [58, 59]. Learning-based approaches were also suggested [4, 26] with an increased focus on deep learning in recent years [13, 14, 24, 25, 28, 41, 68], culminating in the use of pre-trained language models, and specifically BERT [9], fine-tuned for entity matching [7, 29, 30, 45].

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il, sbargen@campus.technion.ac.il; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il; Roee Shraga, Worcester Polytechnic Institute, Worcester, Massachusetts, USA, rshraga@wpi.edu.

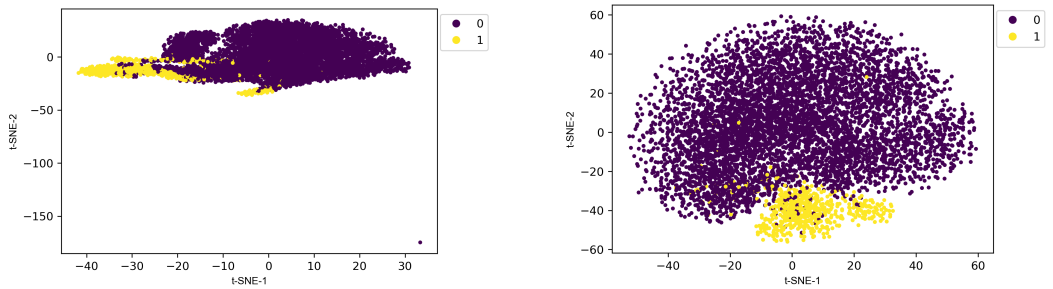
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/12-ART224 $15.00
https://doi.org/10.1145/3626711

Machine learning approaches to entity matching are by-and-large supervised, requiring labeled data for training, and in the case of deep learning, massive amounts of them [34, 46, 60]. Obtaining labeled data is challenging, mostly due to the need for intensive human expert labor that accompanies the labeling process [33, 34, 55]. This, in turn, calls for methods that reduce the labeling process load while maintaining an accurate trained matcher.

Several approaches were suggested over the years to overcome the dependence on large amounts of data, among which, unsupervised [3, 65, 66] and active learning methods [47, 53]. Active learning, the most prevalent approach to the insufficient labeled data problem, offers methods for training using a limited amount of labeled data. This is done by zeroing in on informative samples to be labeled and used for training. An active learning framework uses selection criteria (*e.g.*, variations of uncertainty [25, 38, 48] and centrality [67, 69]) to pinpoint the samples that are expected to improve the model performance the most. These selected samples are then sent to a labeling oracle and added to the available training set in following iterations.

A key tool which we use to quantify both uncertainty and centrality of a sample (a record pair in our case) is its vector-space latent representation. Traditional entity matching approaches constructed pair feature vector using widely used similarity measures [4, 26, 65] or learnable attribute-wise similarity vector using neural networks [24, 41]. Contemporary solutions [7, 29, 45], which demonstrate superior performance and do not require feature engineering, yield an embedding vector as alternative to feature vectors when representing the input (pair) with respect to the task.

In this work we offer a solution to low resource entity matching tasks, where there is a limited budget for labeling. In particular, we attend to the labeled data scarcity problem using active learning, proposing a novel strategy tailored to the specific characteristics of entity matching. Entity matching usually suffers from inherent label imbalance, where the number of negative (non-match) pairs overshadows the number of positive (match) ones [2]. When training with insufficient amount of data, this property is often problematic since the model may struggle to generalize to unseen minority class samples (in this case, matching pairs). To tackle this challenge, we ensure a balanced sampling of positive and negative samples using high-dimensional latent space encoding for tuple pairs. Specifically, we argue that vector-space similarity between pair representations reflects, with high probability, agreement of their labels. We next use this notion to carefully guarantee a balanced sampling, although positive pairs are harder to be found.



(a) Amazon-Google (b) Walmart-Amazon

Fig. 1. Visualization of pair distribution by t-SNE, partitioned into match and non-match pairs. Using representative vectors (with dimension of 768) of a fully trained models we demonstrate that match pairs tend to gather together.

Example 1. *Figure 1 provides an illustration of the latent space of tuple pairs over well known benchmarks using t-SNE [64]. Figure 1a presents the widely used Amazon-Google dataset,*[1] *which consists of 11,460 pairs, with only 1,167 (∼10%) labeled as match samples. To demonstrate the pairs behavior, we trained a DITTO [29] model with the fully available train set (60% of the entire dataset). After training, we extracted the latent representations (dimension of 768) generated for each tuple pair and applied t-SNE to create a human-readable 2-dimensional reduced space, where the yellow dots represent match pairs and the purple dots represent non-match pairs. As illustrated, there is a concentration of match pairs in a few main areas of the latent space, forming a cohesive and clear partition between match and non-match samples. As another example, Figure 1b presents the pairs distribution over the Walmart-Amazon dataset*[2] *(10,242 pairs, 962 matches, with 60% of the entire dataset used as a train set). Again, we see a concentration of match pairs, this time mainly in a single area of the reduced latent space.*

This example suggests that a latent space can identify regions where mostly match or non-match samples reside. We use this observation to tackle a major problem in active learning, namely modeling prediction uncertainty. A standard approach is to use the model prediction confidence score as a proxy to its underlying uncertainty. However, when using transformer-based models this method is likely to fail, as they tend to produce extreme confidence values (close to 0 or 1) which barely reflect the real confidence [17, 22]. We propose a new approach for modeling the uncertainty of a pair, based on its agreement with other pairs in its vicinity, such that when the correspondence between pair's prediction and its surroundings is higher, its uncertainty becomes lower.

1.1 Contributions

In this work we offer a novel active learning approach to entity matching, utilizing an effective space partitioning for diversity, and locality-based measurements for targeted sample selection. Specifically, equipped with the observed phenomenon of concentration of match pairs (Figure 1), we propose a sampling strategy that can be intuitively demonstrated using the popular Battleship game[3] (hence the battleship approach in the manuscript title). Our strategy uses a locality principle, searching in a vicinity of a match (or non-match) pair to create a balanced training set for human labeling. To balance locality with diversity and supporting model generalization we use a constrained version of the well-known k-means clustering algorithm [6], combined with centrality and uncertainty measures that offers an efficient selection mechanism of samples.

Our main contribution can be summarized as follows:

(1) A novel active learning sample selection method, based on match and non-match pair locality to ensure a balanced training set. The proposed solution uses tuple pair representation as a tool for diverse sampling.
(2) A novel uncertainty measure, overcoming the barrier of dichotomous confidence values assigned by transformer pre-trained language model.
(3) A large-scale empirical evaluation showing the effectiveness of our approach.
(4) An open source access to our implementation.[4]

The rest of the paper is organized as follows. We survey the necessary background for our task in Section 2, positioning entity matching in the context of active learning. In Section 3 we propose a new active learning algorithm for entity matching, inspired by the battleship game. We describe the experimental setup in Section 4 and present the empirical evaluation in Section 5. Then, we

[1] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Amazon-Google/exp_data/
[2] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Walmart-Amazon/exp_data/
[3] https://en.wikipedia.org/wiki/Battleship_(game)
[4] https://github.com/BarGenossar/The-Battleship-Approach-to-AL-of-EM-Problem

Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration

JIANHONG TU, Renmin University of China, China
JU FAN*, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
PENG WANG, Renmin University of China, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China
XIAOFENG JIA, Beijing Big Data Centre, China
SONG GAO, Beijing Big Data Centre, China

Data matching – which decides whether two data elements (*e.g.,* string, tuple, column, or knowledge graph entity) are the "same" (*a.k.a.* a match) – is a key concept in data integration, such as entity matching and schema matching. The widely used practice is to build task-specific or even dataset-specific solutions, which are hard to generalize and disable the opportunities of knowledge sharing that can be learned from different datasets and multiple tasks. In this paper, we propose **Unicorn**, a unified model for generally supporting common data matching tasks. **Unicorn** can enable knowledge sharing by learning from multiple tasks and multiple datasets, and can also support zero-shot prediction for new tasks with zero labeled matching/non-matching pairs. However, building such a unified model is challenging due to heterogeneous formats of input data elements and various matching semantics of multiple tasks. To address the challenges, **Unicorn** employs one generic `Encoder` that converts any pair of data elements (a, b) into a learned representation, and uses a `Matcher`, which is a binary classifier, to decide whether a matches b. To align matching semantics of multiple tasks, **Unicorn** adopts a mixture-of-experts model that enhances the learned representation into a better representation. We conduct extensive experiments using 20 datasets on seven well-studied data matching tasks, and find that our unified model can achieve better performance on most tasks and on average, compared with the state-of-the-art specific models trained for ad-hoc tasks and datasets separately. Moreover, **Unicorn** can also well serve new matching tasks with zero-shot learning.

CCS Concepts: • **Information systems → Mediators and data integration**.

Additional Key Words and Phrases: data matching, data integration, multi-task learning

ACM Reference Format:
Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1, Article 84 (May 2023), 26 pages. https://doi.org/10.1145/3588938

*Ju Fan is the corresponding author.

Authors' addresses: Jianhong Tu, Renmin University of China, Beijing, China, tujh@ruc.edu.cn; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Peng Wang, Renmin University of China, Beijing, China, lisa_wang@ruc.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn; Xiaofeng Jia, Beijing Big Data Centre, Beijing, China, jiaxf@jxj.beijing.gov.cn; Song Gao, Beijing Big Data Centre, Beijing, China, gaos@jxj.beijing.gov.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART84 $15.00
https://doi.org/10.1145/3588938

1 INTRODUCTION

Data matching generally refers to the process of deciding whether two data elements are the "same" (*a.k.a.* a match) or not, where each data element could be of different classes such as string, tuple, column, and so on. Data matching is a key concept in data integration [13] and data preparation [9] that includes a wide spectrum of tasks. In this paper, we consider seven common data matching tasks, namely entity matching, entity linking, entity alignment, string matching, column type annotation, schema matching, and ontology matching, as shown in Figure 1.

Due to their importance, almost all the aforementioned matching tasks have been studied for decades, and still remain to be important research topics. With the tremendous successes of deep learning, deep neural networks have been widely used for tackling various matching tasks, such as DeepMatcher [34] and Ditto [30] for entity matching (see a recent tutorial [4]), and TURL [10] and HNN [5] for column type annotation. However, these current deep learning based solutions are task-specific or even dataset-specific, which are referred to as **specific models** in this paper.

There are two main **limitations of specific models**. First, they can only learn knowledge from specific tasks or datasets. That is, the learned knowledge cannot be shared across different models. For example, the knowledge learned by a column type annotation model (*e.g.,* TURL [10]) cannot be shared with an entity matching model (*e.g.,* Ditto [30]) due to different neural network designs, although very likely the two models can help each other. Second, one model has to be trained (or fine-tuned) only on the labeled examples of each task or dataset, which is inefficient and has a high monetary cost. For example, DeepMatcher [34] and Ditto [30] have to be fine-tuned on a new entity matching dataset with at least hundreds of labeled matching/non-matching pairs.

In this paper, we propose **`Unicorn`**, a unified model to support multiple data matching tasks, as shown in Figure 1. Compared with aforementioned specific models, **`Unicorn`** has the following notable advantages.

(1) ***Task unification*** that generalizes task-specific solutions into one unified model, thus achieving lower maintenance complexity and smaller model sizes, compared with specific models.
(2) ***Multi-task learning*** that enables the unified model to learn from multiple tasks and multiple datasets to make full use of *knowledge sharing*, which even outperforms specific models trained only on their own datasets separately.
(3) ***Zero-shot prediction*** that allows the model to make predictions for a new task or a new dataset with *zero* labeled matching/non-matching pairs.

Although several unified models have been recently studied in the NLP and CV communities [3, 41, 42], building such a unified model for supporting multiple data matching tasks is challenging.

(1) Data elements in the matching tasks can take ***heterogeneous formats***, such as tuples and columns in tabular data, entities in knowledge graphs, and plain text, which will increase the difficulty of task unification.
(2) Each data matching task may have its ***unique matching semantics***. For example, entity matching that matches two tuples is different from schema matching that matches two columns. Thus, it is non-trivial to enable knowledge sharing among multiple tasks.

To address the challenges, we develop a general framework **`Unicorn`**, which consists of three key modules: an `Encoder`, a `Mixture-of-Experts` layer, and a `Matcher` (see Figure 1). First, the `Encoder` converts any pair of elements (a, b) with heterogeneous formats into a learned representation. To this end, we propose to serialize any pair of elements into text while still preserving their inherent structure, and employ a unified pre-trained language model for effective encoding. Second, to align matching semantics of various tasks or datasets, the `Mixture-of-Experts` layer enhances the learned representation into a better representation by combining the knowledge from multiple "experts" controlled by a learned gating network. Third, the `Matcher` is a typical binary classifier,

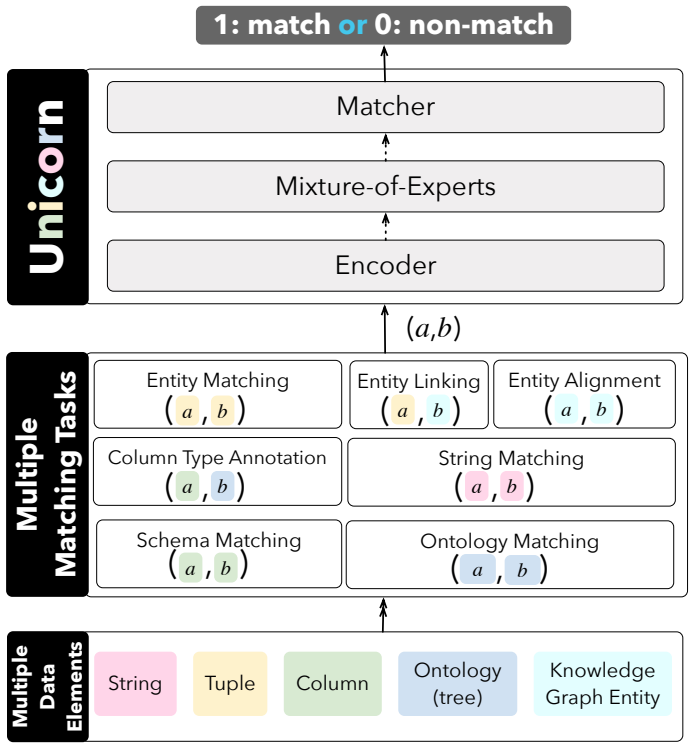


Fig. 1. **Unicorn** is a unified model for "data matching" task in data integration. So far, it supports seven matching tasks in data integration for a pair (a, b) where a or b may be of the same or different types of data elements. **Unicorn** consists of a unified `Encoder`, a `Mixture-of-Experts` layer, and a `Matcher` that will decide whether (a, b) is a match (1) or a non-match (0).

e.g., a multilayer perceptron (MLP), to predict either 1 (for match) or 0 (for non-match) by taking the above representation as input.

Our notable contributions are summarized as follows.

(1) As far as we know, **Unicorn** is the first unified multi-tasking model towards supporting various matching tasks in data integration. We formally define the problem of data matching tasks and introduce an overview of the **Unicorn** framework (Section 2).
(2) We develop effective techniques for two key modules in the **Unicorn** framework. We introduce a unified representation learning method for the `Encoder` module (Section 3) and design effective methods for the `Mixture-of-Experts` layer (Section 4).
(3) We conduct a thorough evaluation on 20 datasets for the seven common data matching tasks shown in Figure 1. Extensive experiments show that **Unicorn**, as a unified model, outperforms the state-of-the-art specific models on most tasks and on average (Section 5 **Exp-3**). In addition, **Unicorn** can well serve new matching tasks with zero-shot learning (Section 5 **Exp-4**).
(4) We make a unified benchmark for multiple data matching tasks available at Github[1] . We standardize the format of datasets from different data matching tasks, making these datasets more convenient to use. We publish the source code of **Unicorn**, and provide the trained **Unicorn**

[1]https://github.com/ruc-datalab/Unicorn

Benchmarking Filtering Techniques for Entity Resolution

George Papadakis
National and Kapodistrian University of Athens, Greece
gpapadis@di.uoa.gr

Marco Fisichella
L3S Research Center, Germany
mfisichella@l3s.de

Franziska Schoger
L3S Research Center, Germany
schoger@l3s.de

George Mandilaras
National and Kapodistrian University of Athens, Greece
gmandi@di.uoa.gr

Nikolaus Augsten
University of Salzburg, Austria
nikolaus.augsten@plus.ac.at

Wolfgang Nejdl
L3S Research Center, Germany
nejdl@l3s.de

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00389

***Abstract*—Entity Resolution is the task of identifying pairs of entity profiles that represent the same real-world object. To avoid checking a quadratic number of entity pairs, various filtering techniques have been proposed that fall into two main categories: (i) *blocking workflows* group together entity profiles with identical or similar signatures, and (ii) *nearest-neighbor methods* convert all entity profiles into vectors and identify the closest ones to every query entity. Unfortunately, the main techniques from these two categories have rarely been compared in the literature and, thus, their relative performance is unknown. We perform the first systematic experimental study that investigates the relative performance of the main representatives per category over numerous established datasets. Comparing techniques from different categories turns out to be a non-trivial task due to the various configuration parameters that are hard to fine-tune, but have a significant impact on performance. We consider a plethora of parameter configurations, optimizing each technique with respect to recall and precision targets. Both schema-agnostic and schema-based settings are evaluated. The experimental results provide novel insights into the effectiveness, the time efficiency and the scalability of the considered techniques.**

***Index Terms*—nearest neighbors, record linkage, deduplication**

I. Introduction

Entity Resolution (ER) is a well-studied problem that aims to identify so-called *duplicates* or *matches*, i.e., different entity profiles that describe the same real-world object [1]. ER constitutes a crucial task in a number of data integration tasks, which range from Link Discovery for interlinking the sources of the Linked Open Data Cloud to data analytics, query answering and object-oriented searching [2]–[4].

Due to its quadratic time complexity, ER typically scales to large data through a Filtering-Verification framework [1], [3]. The first step (*filtering*) constitutes a coarse-grained, rapid phase that restricts the computational cost to the most promising matches, a.k.a. *candidate pairs*. This is followed by the *verification* step, which examines every candidate pair to decide whether it is a duplicate.

Verification is called *matching* in the ER literature [2], [3], [5]. Numerous matching methods have been proposed; most of them rely on similarity functions that compare the textual values of entity profiles [2], [3], [5], [6]. Early attempts were rule-based, comparing similarity values with thresholds, but more recent techniques rely on learning, i.e., they usually model matching as a binary classification task (`match`, `non-match`) [6]. Using a labelled training dataset, supervised, active, and deep learning techniques are adapted to ER [7]. In some cases, a clustering step is subsequently applied on the resulting similarity scores to refine the output [8].

In this work, we are interested in filtering methods, which significantly reduce the search space of ER. The performance of a filtering technique is assessed along three dimensions: recall, precision and time efficiency. A good filter stands out by:

1) *High recall.* The candidate pairs should involve many duplicates to reduce the number of false negatives.
2) *High precision.* The candidate pairs should involve few false positives (i.e., non-matching pairs) to significantly reduce the search space.
3) *Low run-time.* The overhead added by the filtering step to the overall run-time of ER should be low.

Ideally, filtering should also be directly applicable to the input data. For this reason, we exclusively consider methods that require no labelled training instances, which are often not available or expensive to produce [9]. We also focus on techniques for textual entity profiles. These methods are organized into two types [10]:

1) *Blocking workflows* extract one or more signatures from every entity profile and form clusters of profiles with identical or similar signatures [3], [11], [12].
2) *Nearest neighbor (NN) methods* index part or all of the input data and treat every entity profile as a query, to which they return the most similar candidates. They rely on sparse [13], [14] or dense numeric vectors [15].

Although the two types of filtering techniques follow very different approaches, they all receive the same input (the entity profiles) and produce the same output (candidate pairs).

To the best of our knowledge, no previous work systematically examines the relative performance across the two different kinds of methods. The main blocking methods are empirically evaluated in [11], [12], [16]. The studies on sparse [13], [14], [17] and dense vector-based NN methods [15] evaluate run-time and approximation quality, but do not evaluate the performance on ER tasks (cf. Section II). Even in the few cases where blocking is used as baseline for an NN method (e.g., Standard/Token Blocking is compared to DeepBlocker in [9]), the comparison is not fair: the blocking method is treated

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00389

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

as an independent approach, instead of applying it as part of a blocking workflow, which is common in the literature [11].

Comparing techniques from different categories turns out to be a challenging task, due to their fundamentally different functionality and the diversity of configuration parameters that significantly affect their performance. Yet, there is *no* systematic fine-tuning approach that is generic enough to apply to all filtering methods – e.g., the step-by-step configuration optimization in [11] applies to blocking pipelines, but not to the single-stage NN methods.

In this work, we perform the first thorough and systematic experimental study that covers both types of filtering methods. To ensure a fair comparison, every approach is represented by its best performance per dataset, as it is determined after an exhaustive fine-tuning with respect to a common performance target that considers several thousand different parameter configurations. This approach is applied to 5 blocking workflows and 8 NN methods over 10 real-world datasets. For each filter type, we also consider two baseline methods with default parameters. We explore both *schema-agnostic* and *schema-based settings*: the latter focus exclusively on the values of the most informative attribute, while the former take into account all information within an entity, essentially treating it as a long textual value – as a result, they are inherently applicable to datasets with heterogeneous schemata.

Overall, we make the following contributions:

- We perform the first thorough experimental analysis on filtering techniques from different categories. We evaluate 14 state-of-the-art filters and 4 baselines on 10 real-world datasets in both schema-based and schema-agnostic settings.
- Our configuration optimization process enables the meaningful comparison of blocking workflows and NN methods in the context of serial processing (single-core execution).
- We present a qualitative analysis of the filtering techniques based on their *scope* and *internal functionality*.
- We perform a thorough scalability analysis that involves seven synthetic datasets of increasing size.
- Our work provides new insights into the relative performance and scalability of the considered techniques. We show that the blocking workflows and the cardinality-based sparse NN methods consistently excel in performance.
- Two of the tested NN methods, SCANN and kNN-Join, are applied to ER for the first time. SCANN is one of the most scalable techniques, and kNN-Join one of the best performing ones, while sticking out by its intuitive fine-tuning.
- All code and data used in this work are publicly available through a new, open initiative that is called Continuous Benchmark of Filtering methods for ER: https://github.com/gpapadis/ContinuousFilteringBenchmark.

The main part of the paper is structured as follows: Section II discusses the related works, while Section III defines formally the configuration optimization task. We elaborate on the filtering methods in Section IV and present their qualitative and quantitative analyses in Sections V and VI, resp. Section VII concludes with the main findings of our experimental analysis along with directions for future research.

II. Related Work

There has been a plethora of works examining the relative performance of blocking methods. The earliest systematic studies were presented in [3], [12]. They focus exclusively on schema-based settings in combination with several user-defined parameter configurations. They also consider exclusively the first step of blocking workflows: block building.

These studies were extended in [16], which examines the same block building methods and configurations, but applies them to schema-agnostic settings, too. The experimental outcomes suggest that recall raises significantly, when compared to the schema-based settings, while requiring no background knowledge about the given data and the quality of its schema. They also suggest that the sensitivity to parameter configuration is significantly reduced.

Building on these works, the experimental analysis in [11] examines the relative performance of the blocking workflows in schema-agnostic settings. In particular, it considers blocking workflows formed by exactly three steps: block building, block filtering and comparison cleaning. In our work, we extend this analysis by considering the top performing block building methods, all of which cluster together entities that share identical signatures. We combine them with three consecutive, but optional steps: block purging, block filtering and comparison cleaning. These steps give rise to seven different filtering pipelines, out of which only one was examined in [11].

Our work also differs from [11] in that it has only one dataset in common – excluding the scalability ones. Most importantly, [11] optimizes the configuration parameters in a heuristic step-by-step manner: first, the performance of block building is heuristically optimized and then, block filtering is heuristically fine-tuned by receiving as input the output of optimized block building as so on for comparison cleaning. In contrast, we consider a holistic approach to configuration optimization, simultaneously fine-tuning all steps in a blocking workflow. As explained in [18], [19], this approach consistently outperforms the step-by-step fine-tuning, because it is not confined to local maxima per workflow step, while it considers a significantly larger set of possible configurations.

Finally, we also go beyond [3], [11], [11], [12] in two more ways: (i) we systematically fine-tune blocking workflows in the context schema-based settings, and (ii) we compare blocking workflows with sparse and dense NN methods.

The sparse vector-based NN methods essentially correspond to similarity joins. The relative performance of the main methods is examined in [13], [14], [17] with respect to runtime. Recall and precision are not considered, because they are identical across all approaches, i.e., all methods retrieve the same pairs of entities that exceed a similarity threshold. These pairs, which are not necessarily matching, have not been evaluated with respect to the recall and precision of ER. As a result, these studies are not useful in assessing the performance of string similarity joins for ER. Note also that none of the sparse NN methods we consider has been examined in prior experimental analyses. The reason is that the local kNN-Join

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

lies out of the focus of [13], [14], [17], while the range joins for ER involve very low similarity thresholds, unlike the string similarity joins examined in [13], [14], [17].

The dense vector-based NN methods are experimentally compared in [15], but the evaluation measures are restricted to throughput, i.e., executed queries per second, and to recall, i.e., the portion of retrieved vectors that are indeed the nearest ones with respect to a specific distance function (e.g., the Euclidean one). This is different from ER recall, as the closest vectors are not necessarily matching. Nevertheless, we rely on the experimental results of [15] in order to select the top-performing dense NN methods: Cross-polytope and Hyperplane LSH, FAISS and SCANN. We also consider MinHash LSH, a popular filtering technique [10], and DeepBlocker, the most recent learning-based approach, which consistently outperforms all others [9].

III. Preliminaries

We define an entity profile e_i as the set of textual name-value pairs, i.e., $e_i = \{\langle n_j, v_j \rangle\}$, that describes a real-world object [11], [16]. This model covers most established data formats, such as the structured records in relational databases and the semi-structured instance descriptions in RDF data. Two entities, e_i and e_j, that pertain to the same real-world object ($e_i \equiv e_j$) are called *duplicates* or *matches*.

ER is distinguished into two main tasks [3], [5]:

1) *Clean-Clean ER* or *Record Linkage*, which receives as input two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$ that are individually duplicate-free, but overlapping, and
2) *Dirty ER* or *Deduplication*, whose input comprises a single set of entity profiles, $\mathcal{E}$, with duplicates in itself.

In both cases, the output consists of the detected duplicate profiles. In the context of Clean-Clean ER, the filtering methods receive as input $\mathcal{E}_1$ and $\mathcal{E}_2$ and produce a set of candidate pairs $\mathcal{C}$, which are highly likely to be duplicates and should be analytically examined during the verification step. To measure the **effectiveness** of filtering, the following measures are typically used [5], [11], [12], [16], [20]:

1) *Pair Completeness (PC)* expresses *recall*, estimating the portion of the duplicate pairs in $\mathcal{C}$ with respect to those in the groundtruth: $PC(\mathcal{C}, \mathcal{E}_1, \mathcal{E}_2) = |\mathcal{D}(\mathcal{C})|/|\mathcal{D}(\mathcal{E}_1 \times \mathcal{E}_2)|$, where $\mathcal{D}(x)$ denotes the set of duplicates in set x.
2) *Pairs Quality (PQ)* captures *precision*, estimating the portion of comparisons in $\mathcal{C}$ that correspond to real duplicates: $PQ(\mathcal{C}) = |\mathcal{D}(\mathcal{C})|/|\mathcal{C}|$.

All measures result in values in the range $[0, 1]$, with higher values indicating higher effectiveness. Note that there is a trade-off between PC and PQ: the larger C is, the higher PC gets at the cost of lower PQ, and vice versa for a smaller set of candidates. The goal of filtering is to achieve a good balance between these measures.

In this context, we formalize the following configuration optimization task, which enables the comparison of fundamentally different filtering techniques on an equal basis:

Problem 1 (Configuration Optimization): Given two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$, a filter method, and a threshold τ on recall (PC), configuration optimization fine-tunes the parameters of the filtering method such that the resulting set of candidates $\mathcal{C}$ on $\mathcal{E}_1$ and $\mathcal{E}_2$ maximizes PQ for $PC \geq \tau$.

Note that we set a threshold on recall, because ER solutions typically consist of two consecutive steps: Filtering and Matching. The recall of the former step determines the overall recall of ER, since the duplicate pairs that are not included in the resulting set of candidates C cannot be detected by most matching methods. This applies both to Clean-Clean ER, where there is no transitivity, due to the 1-1 matching constraint, and to Dirty ER, where the matching algorithms typically consider local information, e.g., the (deep) learning-based methods that treat matching as a binary classification task [21], [22]. As a result, we chose $\tau = \mathbf{0.9}$ as the PC threshold that ensures high recall for the overall ER process. Preliminary experiments demonstrated that a lower threshold, e.g., 0.85, didn't alter the relative performance of the considered techniques. Besides, in more than 40% of considered cases, the filtering techniques address Problem 1 with $PC \geq 91\%$, i.e., much higher than our threshold.

Regarding **time efficiency**, the *run-time* (RT) measures the time between receiving the set(s) of entity profiles as input and producing the set of candidate pairs as output. RT should be minimized to restrict the overhead of filtering on ER.

IV. Filtering Methods

A. The two paradigms of filtering

Blocking methods first associate every input entity with one or (usually) more signatures and then, they cluster together entities with identical or similar signatures into blocks. Every pair of entities that appears in at least one block is considered a matching candidate. The resulting blocks contain two unnecessary types of candidates, i.e., pairs whose verification lowers precision, without any benefit for recall:

1) The *redundant* candidates are repeated across different blocks, because every entity typically participates in multiple ones and, thus, the blocks are overlapping.
2) The *superfluous* candidates involve non-matching entities.

To eliminate the former and reduce the latter, block and comparison cleaning are applied to the initial blocks, restructuring them based on global patterns [11], [23]. Figure 1 depicts the complete workflow for blocking [10], [24]. Initially, a set of blocks is created by at least one block building method. The initial block collection is then processed by two coarse-grained block cleaning techniques, Block Purging and Block Filtering. Both produce a new, smaller block collection, B' and B'', respectively, but are optional and can be omitted, e.g., in the case of schema-based blocks with low levels of redundant and/or superfluous comparisons. Finally, a comparison cleaning technique is applied, whose fine-grained functionality decides for individual comparisons whether they should be retained or discarded; in this mandatory step, at least the redundant candidates are discarded, but the superfluous ones are also subject to removal.

A fundamentally different approach is followed by NN methods. Instead of extracting signatures from the input entity

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching

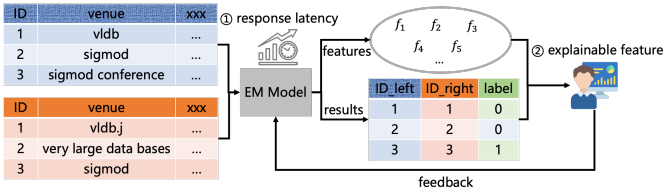
Yafeng Tang[1], Zheng Liang[1], Hongzhi Wang[1,✉], Xiaoou Ding[1], Tianyu Mu[1], Huan Hu[2]
[1] School of Computer Science, Harbin Institute of Technology, Harbin, China
[2] Huawei Cloud Computing Technologies Co., Ltd, Hangzhou, China
tangyf@stu.hit.edu.cn,{lz20, wangzh, dingxiaoou,mutianyu}@hit.edu.cn, huhuan18@huawei.com

***Abstract*—Entity Matching (EM) is a crucial task in data integration. Compared to deep learning-based EM solutions, tree-based machine learning models are more computationally effective and explainable, making them more applicable in real-world EM scenarios. However, Random Forest-based EM methods select features with a static feature engineering rule set for all attributes. Consequently, they suffer model retraining cost to select features, and can hardly customize to different EM tasks. To tackle this problem, we propose Description-Similarity Rules (DSR) for EM feature engineering. DSR introduces diverse attribute value distribution metrics and data-driven thresholds to traditional EM feature engineering rules. Unfortunately, both the DSR search space and its online model retraining costs are exponential. By pushing the model retraining to the offline stage, our DSR set mining algorithm is one order of magnitude faster than the baseline algorithms, taking only seconds for online selection. Empirically, DSR outperforms the feature engineering methods by 3.75% on average F1 score, while reaching the state-of-the-art EM performance on several datasets.**

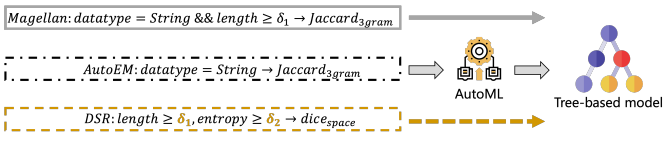
***Index Terms*—Entity Matching, Feature Engineering, Rule Mining**

I. Introduction

Entity matching [1]–[3] is the problem of identifying co-referent manifestations that refer to the same real-world entity from different data sources. For example, attributes of the same person from multiple sources are often complicated [4] in the real world. EM can improve data quality by reducing such duplicates among tables [5].



(a) An interactive EM pipeline



(b) Rule-based EM model

Fig. 1: A typical EM pipeline.

✉ Hongzhi Wang is the corresponding author.

EM has been an open problem for decades. Conventional EM solutions are based on similarity functions, which quantify the similarity between pairwise attribute values of an entity [6]. Rule-based [7]–[10] EM methods use certain thresholds for attribute similarities to make matching decisions, where experts specify the thresholds (e.g., the value of δ_1 in Figure 1(b)). Machine learning-based methods [6] train machine learning models on pairs of labeled entities with similarities as features. They search for the feature combination with the best evaluation metric (e.g., F1 score). Deep Learning methods [1], [3], [11]–[14] incorporate domain knowledge in language-model-based encoders. However, real-world EM tasks pose flexibility requirements beyond the grasp of these methods, which can be specified in the following aspects [15]. (1) **Response latency.** Entity Matching is rarely a one-time job. As Figure 1(a) shows, a typical EM pipeline requires data scientists to go through the cycle of exploring data [10], changing EM the solution, and checking results several times [16]. Also, EM becomes even more time-intensive on multi-source data which need to be split and matched separately. Methods with low response latency are necessary for refining and validating EM solutions in a pipeline as Figure 1. (2) **Explainable features.** To understand and reconfigure a machine learning model in the EM pipeline, one needs to understand how the model is influenced by its input, i.e., the feature value. Even for explainable EM models like Random Forest, system-selected features still need to be traceable from data [17].

To the best of our knowledge, few EM solutions can address the above two issues simultaneously. Rule-based methods like Autofuzzyjoin [18] may not perform well with less structured datasets and long textual data since they rely on limited word level similarity metrics and TF-IDF scores [19]. State-of-the-art deep learning methods like Sudowoodo [13] make it difficult for users to understand why entities are matched or unmatched. They often require a high cost for debugging the model: try numerous configurations at each iteration, and need expert experience to make decisions. Mainstream machine learning-based methods are easier to understand, but their efficiency is limited by searching models and features. Therefore, to meet the flexibility demands mentioned above, lightweight machine learning models such as Random Forest (RF) [20] have been generally adopted for EM [21] in commercial data integration pipelines [6], [15], [22] for their good explainability and satisfactory performance.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00008

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00008

Example 1. *Consider a training process of a Random Forest-based EM model, a data scientist needs to construct features (i.e. similarities) for each attribute pair.*

In Figure 1(b), Magellan [23] simply uses length and data types to recommend similarities, so the selected features result in poor performance (Section V). AutoML-EM [6] relaxes the rule set of Magellan and recommends redundant features, which are subsequently trimmed by auto-sklearn [24]. The general feature selection methods [8], [25]–[29] search for the optimal feature at the cost of exhaustive model retraining. Considering efficiency, in this paper, we focus on rule-based feature selection methods.

Challenges. From Example 1, we can see that the bottleneck of the RF-based EM pipeline is the drawback of rule-based feature selection, which can be summarized as follows.

Exhaustive Rule Candidate Space. In a typical EM pipeline, rule-based similarity selection methods typically cost extensive computational resources as expert knowledge is unavailable. The threshold of descriptions [1] can take any value within a continuous range. Actually, we need to consider rules with conjunction of multiple descriptions as antecedents, and all similarities can be consequent. As a result, the rule candidate space is overwhelming, and searching for each rule candidate requires an EM model retraining. What's worse, an EM pipeline mostly lacks labeled data, so the hyperparameter search algorithms [30] fail to solve this problem.

Rule set is too rigid. The rule set of existing solutions only considers `length` and `datatype` of attribute pair, while the threshold is fixed for all EM tasks. They fail to offer users enough flexibility to give feedback and perform an EM task at hand. Consequently, such rigid rule sets with fixed thresholds recommend too few or insufficient features for complex datasets (as shown in Example 1). Users can barely understand EM data and give constructive feedback to the pipeline in such cases.

Description-Similarity Rule. Rule-based FE methods fall short in practice due to the above limitations. To unlock the flexibility of EM models, we need to design a flexible substitute of feature engineering rules without too much computational overhead. Motivated by this, we propose the Description-Similarity Rule (DSR). DSR helps users in EM pipeline to better understand data with the newly introduced "description". Also, DSR has a flexible threshold, which can be discovered from data without expert knowledge.

Contribution. The contributions of our work can be summarized as follows.

• We propose the Description-Similarity Rule (DSR), a set of data-driven rules for flexible Feature Engineering on Entity Matching models. We design *descriptions* to support common data types including sets, vectors, strings, and boolean data.

• To discover the optimal DSR set for a given EM task, we design a DSR mining algorithm accelerating the baseline approach by one order of magnitude (Section IV). Also, we address the problem of exponential online retraining by sampling historical datasets with a theoretical guarantee on error probability(Section III).

• Empirically, DSR beats the state-of-the-art EM solution by 3.75% on average F1 score. DSR is also statistically comparable to black-box Deep Learning methods (Section V).

[1]Description is a set metric to compute certain attribute value distributions. We will go into details in Definition 2

TABLE I: Frequently used notations.

Symbol	Meaning
T_1, T_2	A pair of tables for entity matching
$A_1, A_2, \ldots$	The attribute set of tables
t_1, t_2	An entity pair from T_1 and T_2
M	The entity matching model
p	The evaluation metric for the EM task
$D = \{d_1, d_2, d_3, d_4\}$	A set of descriptions
D'	A subset of descriptions
$S = \{S_1, S_2, \ldots\}$	A set of similarities
$R = \{r_1, r_2, \ldots\}$	A set of description-similarity rules
δ_i	The threshold for i-th description
θ	The threshold for lowest acceptable performance
ϕ	An operator
$dom(d_i)$	The domain of valid value of description
E	The expectation of a variable
n	The budget of EM model retraining times

II. Overview

In this section, we define essential concepts for Description-Similarity Rules (Section II-A), elaborate the design details of *Descriptions* (Section II-B), present the DSR recommendation problem(Section II-C), and give an overview of our solution (Section II-D).

A. Background

Entity Matching solutions for structured data consist of at least two phases: blocking and matching [31], and often assume that upstream tasks have prepared datasets with aligned attributes [6], [32]. In this paper, we focus on the feature engineering of entity matching for relational data and treat entity blocking as an orthogonal problem.

Definition 1. ***Entity Matching (EM):*** *Given two relational tables T_1 and T_2 with the aligned attribute set $A_1, A_2, ...$ with the same schema, Entity Matching determines whether each pair of records $t_1, t_2 \in T_1 \times T_2$ refers to the same entity in the real world.*

In this paper, we focus on the Random Forest(RF)-based EM solver, as RF is a widely-adopted machine learning model in real-world EM applications [6], [12], [22], [33]. For example, consider the entity pair with $ID = 3$ in Figure 1(a), "sigmod conference" and "sigmod" have a Jaccard similarity of 0.5, which is used by RF-based EM solvers as one of the features to represent this entity pair for RF training. Therefore, the core of *RF-based EM* is **similarity selection** [7], [34], [35], and we follow SOTA RF-based EM methods [6], [23] to exploit similarity selection rules.

A similarity function quantifies the similarity between two strings (or numeric or Boolean values). There are four types of similarity functions: set-based, sequence-based, vector-based,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

and number-based. Next, we will introduce how we design *descriptions* to support the recommendation of these functions.

B. Preliminaries

Our intuition is that general FE methods [36], [37] or intuitive EM methods [6], [23] ignore latent attribute value statistics during similarity recommendation, thus cannot obtain ideal performance on EM task (see Section V for details). Therefore, we propose a new concept called *description* to overcome their drawbacks. For each *description*, we show their relationship with similarity functions.

Definition 2. ***Description (D):*** *Let $T_1.A_i$ and $T_2.A_i$ denote the set of the attribute value A_i of all entities in T_1 and T_2, respectively, the description $d \in D, D = \{d_1, d_2, d_3, d_4\}$ on attribute $d(A_i)$ can be computed as follows:*[2]

- `length`*: For sequence-based similarities such as Jaccard and Jaro [36], we present* `length` *as the length of the string in attribute A.*
- `AvgWordNum(AWN)`*: For set-based similarities, we present* `AWN` *to describe strings including separators.* `AWN` *is associated with similarity functions such as the dice-coefficient [38] and Jaccard Index [39].*

$d_2(AvgWordNum)$: Suppose N is a tokenizer. $N(A)$ denotes the tokenized wordBag after employing N on attribute A,

$$d_2(A_i) = |N(A_i)|$$

A tokenizer can be a publicly available tool [40], or any common delimiter such as spaces, commas, semicolons, etc.

- `Entropy`*: For vector-based similarities [41], we design* `entropy` *to convert strings into document vectors [42], especially for long texts.*

$d_3(entropy)$: For each attribute A, we treat A as a document, and $v(A_i)$ represents the vocabulary of A_i. The `entropy` *of A is calculated as follows:*

$$d_3(A_i) = -\sum_{j=1}^{|v(A_i)|} p_{ij} \cdot \log_2(p_{ij}) \tag{1}$$

where p_{ij} is the probability of the j-th word occurring in document A_i.

- `Index of Coincidence(IoC)`*: For number-based similarities, we present* `IoC` *[43] to identify the distribution of characters,* ***especially where numerical and textual values are mixed****.*

$d_4(IoC)$: We denote $v(A_i)$ as the vocabulary of A_i, and f_c as the frequency of character c in A_i. The `IoC` *of A_i is as follows:*

$$d_4(A_i) = \sum_{c \in v(A_i)} \frac{f_c(f_c - 1)}{|A_i|(|A_i| - 1)}$$

We explain the design of descriptions with Example 2. In our technical report [45], we also provide a formal basis to show that descriptions are actually building blocks of the equation of all four types of similarities.

[2]notations in Definition 2 will not be presented in Table I.

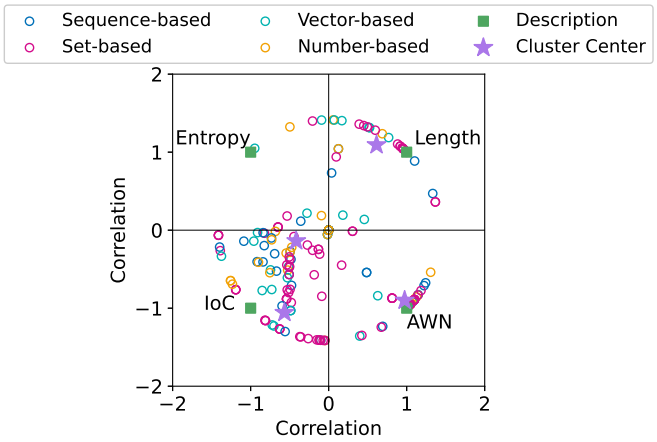


Fig. 2: The correlation diagram between four types of similarity functions (circles) and our descriptions (squares) on the Fodors-Zagats dataset [23] (better viewed in color). The purple stars represent the cluster centers of similarity functions, derived from the K-Means algorithm [44].

Example 2. *In Figure 2, we visualize the correlation between similarities and descriptions on the Fodors-Zagats dataset [23].First, we fix our four descriptions (square) in the center of each quadrant. Then, for each similarity, we calculate its Pearson correlation coefficient with each of the four descriptions and find a point (circle) whose Euclidean distance to each description is proportional to the corresponding coefficient. Note that every quadrant contains quite a few scattered circles. In addition, we apply K-Means clustering for the circles (similarities) and mark the center of the cluster with stars. Another interesting discovery is that three out of four stars land somewhere near a square. That means each description "attracts" considerable amount of similarities.*

As shown in Figure 2, descriptions can serve as the effective precomputation for all similarities. This empirical observation motivates us to explore the idea of recommending similarities based on four descriptions for a given EM task.

Under these definitions, we formally introduce DSR.

Definition 3. ***Description-Similarity Rule:*** *Given a Description set D and a Similarity Function set $S = \{S_1, S_2, \ldots\}$, the Description-Similarity-Rule (DSR) r is:*

$$r: \bigwedge_{d_i \in D' \subseteq D} d_i \geq \delta_i \rightarrow S_j \tag{2}$$

For example, a valid DSR can be written as:

$$(length \geq 10) \wedge (entropy \geq 2.92) \rightarrow Levenshtein$$

Given an entity pair (t_1, t_2), for each attribute A_i, DSR first checks whether its `length` ≥ 10, and then checks whether its `entropy` ≥ 2.92. If all conditions are met, DSR computes the Levenshtein Similarity between $t_1[A_i]$ and $t_2[A_i]$ as the feature. Rules in the DSR set will be traversed, and all similarity functions that meet the *description* constraints will be added to the feature set for the current attribute.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.



Enriching Relations with Additional Attributes for ER

Mengyi Yan
Beihang University, China
yanmy@act.buaa.edu.cn

Wenfei Fan
Shenzhen Institute of Computing Sciences, China
University of Edinburgh
United Kingdom
Beihang University, China
wenfei@inf.ed.ac.uk

Yaoshu Wang
Shenzhen Institute of Computing Sciences, China
yaoshuw@sics.ac.cn

Min Xie*
Shenzhen Institute of Computing Sciences, China
xiemin@sics.ac.cn

ABSTRACT

This paper studies a new problem of relation enrichment. Given a relation D of schema R and a knowledge graph G with overlapping information, it is to identify a small number of relevant features from G, and extend schema R with the additional attributes, to maximally improve the accuracy of resolving entities represented by the tuples of D. We formulate the enrichment problem and show its intractability. Nonetheless, we propose a method to extract features from G that are diverse from the existing attributes of R, minimize null values, and moreover, reduce false positives and false negatives of entity resolution (ER) models. The method links tuples and vertices that refer to the same entity, learns a robust policy to extract attributes via reinforcement learning, and jointly trains the policy and ER models. Moreover, we develop algorithms for (incrementally) enriching D. Using real-life data, we experimentally verify that relation enrichment improves the accuracy of ER above 15.4% (percentage points) by adding 5 attributes, up to 33%.

PVLDB Reference Format:
Mengyi Yan, Wenfei Fan, Yaoshu Wang, and Min Xie. Enriching Relations with Additional Attributes for ER. PVLDB, 17(11): 3109 - 3123, 2024.
doi:10.14778/3681954.3681987

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/SICS-Fundamental-Research-Center/Enrichment.

1 INTRODUCTION

When we talk about incomplete information, we typically refer to (null) values and tuples missing from a relation D of schema R. However, for an application at hand, schema R may be incomplete; it "may not have all attributes required for analysis" [131]. As a consequence, the tuples in D do not have enough information for the application, *e.g.,* causal inference [131], aggregate SQL queries [84, 130] and movie popularity classification [33]. As exemplified in [131], a data analyst in the WHO organization aimed to estimate the effect of a mask policy on the coronavirus mortality rate, but found critical attributes (*e.g.,* weather that affects people's willingness to wear masks) not included in the data [131].

*Corresponding author
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3681987

Table 1: An Example of Person **Table**

tid	name	gender	email	address	city	age	spouse_name	eid
t_1	James Davis	M	james@example.com	18 Elmwood Rd	Chicago	45	Ava Davis	e_1
t_2	John Wilson	M	john@example.com	18 Maple Avenue	Houston	null	Ava Wilson	e_2
t_3	Ava Davis	F	ava@example.com	12 Pine Lane	Boston	36	John Wilson	e_3
t_4	Ava Wilson	F	a.Wat@example.com	18 Maple Avenue	Houston	36	John Wilson	e_3
t_5	Ava Davis	F	ad123@example.com	18 Elmwood Rd	Chicago	42	James Davis	e_4

Attributes are missing from schema R for several reasons [91, 114]. (1) People may lack full knowledge of desired functionality in large-scale applications. (2) The application world constantly evolves, necessitating enhancements. (3) The scale of tasks often requires incremental design and commissioning. Regardless of the reasons, relation D often misses features needed for our application.

In this paper, we focus on entity resolution (ER) as our target application, which has been commonly used in, *e.g.,* e-commerce and financial institutions. As an example, the insurance industry employs ER as a routine operation in detecting identity fraud, which was responsible for over $17 billion stolen from U.S. consumers in 2017 [1]. As reported in [3], fraudsters forge fake identities. Such a fraud is hard to detect, since it can use information stolen from a real person, and a mix of real and synthetic data [6]. There are usually no unique identities as references, since collecting them is "time-consuming, adds friction to the customer journey, and is also an easy check to bypass" [12]. Similar problems also exist in card fraud (*e.g.,* duplicate applications for gift/credit cards) or money laundering (*e.g.,* move money from one account into another) [5, 111].

To detect fraud, the insurance industry has been advocating "to incorporate external information into the identity verification process" [6]. This has been practiced by, *e.g.,* SEON [13] and SIFT [14].

Example 1: Consider Table 1 with 5 tuples t_1-t_5 for 4 persons e_1-e_4, who are applying for promotional gift cards, where each person is limited to one card (*e.g.,* eGift of Starbucks [16]). The tuples have a schema with attributes name, gender, email, address and city; the store allows one person to have, *e.g.,* multiple different emails.

A not-so-sophisticated ER method $\mathcal{A}_{\mathsf{ER}}$ may predict that t_3 and t_5 are the same person (a false positive, FN), since they have exactly the same name. It may also miss the true match of t_3 and t_4 (a false negative, FN), due to different names, emails and addresses, although Ava Wilson was named Ava Davis before her marriage, and she issued a duplicate application using her old identity.

Fortunately, additional attributes can help us reduce FPs/FNs of ER. (a) As remarked in [131], age or ethnicity are often missing in person datasets. If additional attribute age is available, it provides a strong evidence for $\mathcal{A}_{\mathsf{ER}}$ to tell that t_3 and t_5 are mismatched, since they have different ages. (b) If we further enrich the schema with attribute spouse_name, (*e.g.,* by social networks [12]), $\mathcal{A}_{\mathsf{ER}}$ can identify t_3 and t_4, since they are married to the same person. □

Missing attributes are as damaging as missing data, but it has not received much attention. While there has been a host of work on missing data imputation [25, 26, 29, 40, 49, 50, 54, 55, 66, 77, 100, 105, 116, 128, 129], no prior work has systematically studied how to enrich incomplete schema in order to improve ER accuracy. They either do not target downstream applications or is not developed for ER and thus, miss distinguishing attributes for ER (see Section7).

To enrich dataset D of schema R for ER, we can extract information from external sources *e.g.,* text [63, 64], information space [43], XML [126], data warehouse [22] and Web [59]. In fact, knowledge enrichment has been practiced in medicine [112], network [133], e-commerce [15], recommendation [113] and text generation [132]. Among them, knowledge graphs (KGs) are particularly promising for ER. KGs link related entities and disambiguate entities with similar names [17]; moreover, their attributes are already reconciled and typically highly informative [130, 131], making it feasible to improve the ER accuracy. In particular, financial institutions have been using KGs to detect various types of fraud [5]; indeed, "using a KG for ER, the company can easily detect clusters of fake claims" [3].

In light of this, we study relation enrichment for ER by referencing a KG G. We consider *reliable* G, *i.e.,* a KG that is relatively clean and complete. Several popular KGs are in place, *e.g.,* general-purpose Wikidata [9] and domain-specific DRKG [71]. The issue is highly nontrivial. It requires us to link tuples in D with vertices in G for enrichment. Worse still, a KG typically maintains all sorts of properties of entities and their links to provide a comprehensive picture. If we enrich schema R with all such properties, it may hamper the accuracy of ER. As evidenced by [34], only *relevant* attributes contribute positively to identifying true positives, and "attributes containing null values may affect negatively the ER result". This motivates us to enrich schema with bounded relevant features.

Moreover, real-life datasets and KGs constantly change, *e.g.,* Wikidata publishes hundreds of live updates every minute [8] and the IMDB we used is refreshed daily [11]. In particular, financial institutions often require real-time detection of, *e.g.,* fraud in the online payment of credit cards [18]. It is too costly to conduct relation enrichment starting from scratch in response to the updates. These motivate the need for incremental enrichment.

To make the idea work, several questions have to be answered. What distinguishing features should we add to R to best improve the ER accuracy? For a tuple t in D, where can we find the additional attributes from KG G to complement t? How can we incrementally maintain the enriched D in response to updates to D and G?

Contributions & Organization. This paper studies relation enrichment for improving the accuracy of ER models. Consider a relation D of schema R and assume a reliable knowledge graph G.

(1) An enrichment scheme (Section 3). We formulate *the problem of relation enrichment for ER*. Given a black-box ER model $\mathcal{A}_{\mathsf{ER}}$ and a parameter m, it is to extract at most m features from knowledge graph G and extend schema R with the features as additional attributes, in order to maximize the accuracy of $\mathcal{A}_{\mathsf{ER}}$. We separate schema enrichment from data enrichment, and show that the former is NP-complete and the latter is in PTIME. This said, we propose a scheme ENRICH for enriching both schema R and relation D.

(2) Schema enrichment (Section 4). We propose a method for enriching schema R for ER under ENRICH. We develop a method for heterogeneous entity resolution (HER) to identify top-ranked matches (tuples and vertices) across relation D and KG G. We learn a policy via reinforcement learning to extract at most m features that extends R to schema R_G. Each feature is fetched by a path in G. We pick features that are as diverse from the existing attributes of R as possible, yield as few null values as possible, and maximumly improve the accuracy of ER. To make the policy robust to different data distributions, we jointly train the policy and the model $\mathcal{A}_{\mathsf{ER}}$.

(3) Data enrichment (Section 5). Under ENRICH, we develop algorithms for enriching the relation D to an instance D_G of schema R_G. We support both a batch mode and an incremental mode. In the batch mode, we extend each tuple t of D by identifying vertices v in G that refer to the same entity as t via HER, traversing paths from v to extract the additional features, and adding the features to t. In the incremental mode, we dynamically maintain D_G in response to updates to both relation D and graph G. To scale with large G and D, we parallelize the algorithms and show their parallel scalability, *i.e.,* they guarantee to reduce runtime when more resources are used [81]. We defer the parallelization to [10] for the lack of space.

(4) Experimental study (Section 6). Using real-life data and benchmarks, we empirically find the following. (a) Relation enrichment improves the accuracy of ER models by 15.4% on average, up to 33%, by adding 5 attributes. (b) It is on average 5.2% (resp. 14.6%) more accurate than ML models for feature augmentation (resp. feature selection), up to 18.2% (resp. 32.6%). (c) Batch enrichment is 5.94X faster than the baselines on IMDB on average. (4) The incremental method beats the batch one when updates to D and G are up to 20%, and is 6.28X faster when updates $|\Delta G| = 5\%|G|$.

We discuss related work in Section 7 and future work in Section 8.

2 PRELIMINARIES

In this section, we review basic notations, ER and HER.

Relations. Consider a relation schema $R = (\mathsf{id}, A_1, \ldots, A_n)$, where A_i is an attribute ($i \in [1, n]$), and id is an entity id as introduced by Codd [39], such that each tuple of R represents an entity of type τ with identity id. A relation D of R is a set of tuples of schema R.

Knowledge graphs. Following [68], we represent a knowledge graph as $G = (V, E, L)$. Here (a) V is a finite set of vertices representing entities, (b) $E \subseteq V \times V$ consists of edges representing relationships between entities; and (c) for each vertex $v \in V$, $L(v)$ is its feature or value, and for each edge $e \in E$, $L(e)$ is its label. Between a pair (v, v') in V, there are possibly multiple edges carrying distinct labels.

A *path* ρ *from a vertex* v_0 in graph G is $\rho = (v_0, v_1, \ldots, v_l)$ such that (v_{i-1}, v_i) is an edge in E for $i \in [1, l]$. The *length of* ρ is the number l of edges on ρ. A path is *simple* if each vertex appears on ρ at most once. We consider simple paths, simply referred to as paths.

Entity resolution. Given a relation D, entity resolution (ER) is to identify all pairs of tuples in D that refer to the same real-life entity. It returns a set of pairs (t_1, t_2) of tuples of D that are identified as *matches*. If t_1 does not match t_2, (t_1, t_2) is referred to as a *mismatch*.

A number of ER methods have been developed, based on ML [23, 45, 74, 86, 95, 102, 134], logic rules [24, 27, 32, 48, 61, 79, 127] and hybrid of the two [28, 41, 56]. We focus on ML-based ER models, which take quadratic-time (after the relevant ML models are trained).

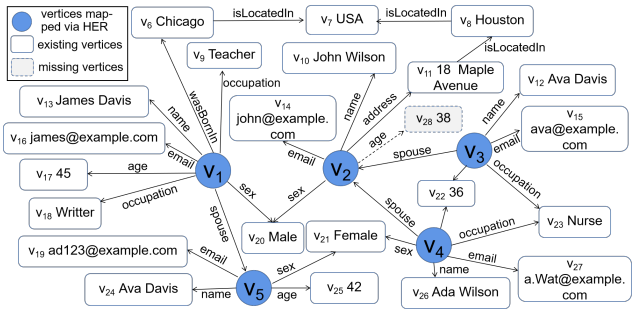


Figure 1: An example knowledge graph

*Heterogeneous entity resolution (*HER*).* HER is to identify entities across a relation and a graph. It is defined as a *mapping* f_{HER} that given a graph G and a set D of tuples of schema R, computes a set:

$$f_{\mathsf{HER}}(D, G) = \{(t, v) \mid t \in D, v \in V \text{ in } G, t \Rightarrow v\}.$$

Here $t \Rightarrow v$ denotes that tuple t and vertex v make a *match*, *i.e.,* t and v refer to the same entity. For each $t \in D$, there can be multiple vertices v in G that match t. We refer to f_{HER} as the HER *mapping*.

Several methods are in place for HER, *e.g.,* rule-based JedAI [98], parametric simulation [51], and ML-based Silk [72], MAGNN [58] and EMBLOOKUP [19]. In particular, it is in $O(|D||G|)$ time to compute all HER matches across D and G by parametric simulation [51].

We assume that schema R has enough information for a well-designed HER mapping f_{HER} to map tuples of R to entities in G.

Example 2: Consider a KG G in Figure 1 for tuples in Table 1 (*e.g.,* derived from social networks [65]), where t_i matches v_i ($i \in [1, 5]$).

HER differs from ER. (a) It identifies tuples and vertices across a relation and a graph, while ER matches tuples in a relation; (b) an attribute in t may map to a path in G, *e.g.,* city of t_2 vs. $\rho = (v_2, v_{11}, v_8)$ in G; (c) not every attribute in t can find a matching path in G, and vice versa, *e.g.,* v_4 has occupation "Nurse", which finds no corresponding attribute in Table 1; and (d) t and v often have different descriptions for the same property, *e.g.,* gender of t_2 vs. sex of v_2. □

3 A SCHEME FOR RELATION ENRICHMENT

In this section, we first formulate the enrichment problem and incremental enrichment problem for ER (Section 3.1). We then settle the complexity of the enrichment problems (Section 3.2). After this, we propose enrichment scheme ENRICH (Section 3.3).

3.1 Relation Enrichment Problem

Given a relation schema $R = (\bar{A})$, where $\bar{A}$ is a set $(\mathsf{id}, A_1, \ldots, A_n)$ of attributes, consider a relation D of R, a KG G, and an ER model $\mathcal{A}_{\mathsf{ER}}$.

Accuracy. We first present how to measure the accuracy improvement on ER models, in terms of Precision, Recall and F_1.

Following Codd [39], consider tuples for representing a (countably infinite) set $\mathcal{E}$ of real-world entities. For tuples t in instance D of schema $R = (\bar{A})$, there exists a mapping f from each tuple ID in D to $\mathcal{E}$ such that $f(t.\mathsf{id}) = e$, *i.e.,* for each tuple t in D, $f(t.\mathsf{id})$ is the entity represented by t (such mapping is usually implicitly assumed in ER). Then the accuracy of $\mathcal{A}_{\mathsf{ER}}$ on D is traditionally measured in terms of $\mathsf{F}_1 = \frac{2 \cdot \mathsf{Precision} \cdot \mathsf{Recall}}{\mathsf{Precision} + \mathsf{Recall}}$. Here Precision is the ratio of *pairs of distinct tuples* that are correctly identified to all identified tuple pairs, *i.e.,* $\mathsf{Precision} = \frac{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true},\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true}\}|}$ for distinct t and s, and Recall is the ratio of correctly identified tuple pairs to all tuple pairs that refer to the same real-world entity,

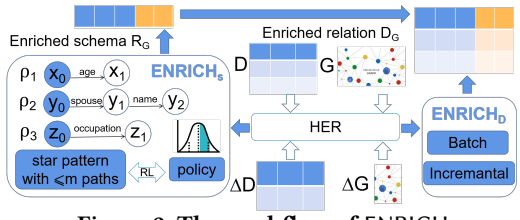


Figure 2: The workflow of ENRICH

i.e., $\mathsf{Recall} = \frac{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true},\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}{|\{(t,s) \mid t,s \in D,\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}$.

Example 3: Consider Table 1 of schema $R = (\bar{A}) = (\mathsf{name}, \mathsf{gender}, \mathsf{email}, \mathsf{address}, \mathsf{city})$, where D has 5 tuples, $t_1[\bar{A}]$-$t_5[\bar{A}]$, and the mapping f is shown in the table. Here assume that $\mathcal{A}_{\mathsf{ER}}$ makes an FP prediction and an FN prediction as stated in Example 1. The precision of $\mathcal{A}_{\mathsf{ER}}$ on D is $\mathsf{Precision} = \frac{0}{1}$ since the only distinct pair predicted true by $\mathcal{A}_{\mathsf{ER}}$ is (t_3, t_5) (which is a FP). Similarly, $\mathsf{Recall} = \frac{0}{1}$ since the only true match (t_3, t_4) is not identified (due to the FN). □

As shown above, $\mathcal{A}_{\mathsf{ER}}$ on D is not accurate, for the lack of attributes. To improve it, we aim to enrich schema $R = (\bar{A})$ to $R_G = (\bar{A}, \bar{B})$, where $\bar{A}$ copies the attributes of R, $\bar{B}$ is a set of at most m attributes extracted from graph G, and m is the "budget" for extending schema R. Intuitively, we want to extend D and create an instance D_G of schema R_G, such that for each t in D, we have exactly one *enriched tuple* $t_G \in D_G$, where $t_G.\mathsf{id} = t.\mathsf{id}$, $t_G[\bar{A}] = t[\bar{A}]$ and $t_G[\bar{B}]$ is the partial tuple extracted from G. We refer to R_G as the *enriched schema* of R with G, and to D_G as the *enriched relation* of D with G.

Example 4: Assume that $R_G = (\bar{A}, \bar{B})$ where $\bar{B} = (\mathsf{spouse_name})$. After enriching R to R_G with an additional attribute spouse_name, the FN (*i.e.,* (t_3, t_4)) is reduced, as stated in Example 1, improving Precision of $\mathcal{A}_{\mathsf{ER}}$ on D_G to $\frac{1}{2}$, since $\mathcal{A}_{\mathsf{ER}}$ predicts true for both (t_3, t_5) (which is the FP) and (t_3, t_4), where only the latter one is correctly identified. Similarly, Recall is also improved to $\frac{1}{1}$ since the only true match (t_3, t_4) in Table 1 is correctly identified. □

We use the difference between the F_1 of $\mathcal{A}_{\mathsf{ER}}$ on D_G and on D, denoted by $\Delta\mathsf{F}_1$, as *improvement of* $\mathcal{A}_{\mathsf{ER}}$ *on* D *via* D_G. The difference of Precision/Recall can also be used, depending on application needs.

Problems. We now state the *enrichment problem* for ER model $\mathcal{A}_{\mathsf{ER}}$.

- *Input*: $R = (\bar{A})$, D and G as above, and a positive integer m.
- *Output*: (a) An enriched schema $R_G = (\bar{A}, \bar{B})$ of R with G such that R is extended with at most m attributes $\bar{B}$ extracted from G; and (b) an enriched relation D_G of D with G.
- *Objective*: To maximize the improvement of $\mathcal{A}_{\mathsf{ER}}$ on D via D_G.

Here ER is conducted by the same $\mathcal{A}_{\mathsf{ER}}$ on both D and D_G.

The enrichment problem can be sub-divided into two problems: (1) *schema enrichment*, to deduce enriched schema R_G, and (2) *data enrichment*, to compute enriched relation D_G after R_G is in place.

Incremental enrichment problem. Real-life data is constantly changed by small updates. Consider updates to D and G. Updates to relation D consist of deleted/inserted tuples, denoted by ΔD; note that modifications to a tuple t can be regarded as deleting t followed by inserting a tuple with the changed values. Graph updates, denoted by ΔG, consist of edge insertions/deletions. Note that vertex updates are a dual [80] and can be handled similarly; and the change to an edge label can be seen as the deletion of an existing edge, followed by the insertion of a new one with the updated label. We use $G \oplus \Delta G$ to denote graph G updated by ΔG; similarly for $D \oplus \Delta D$.



Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching

Nima Shahbazi
University of Illinois Chicago
nshahb3@uic.edu

Nikola Danevski
University of Rochester
ndanevsk@u.rochester.edu

Fatemeh Nargesian
University of Rochester
fnargesian@rochester.edu

Abolfazl Asudeh
University of Illinois Chicago
asudeh@uic.edu

Divesh Srivastava
AT&T Chief Data Office
divesh@research.att.com

ABSTRACT

Entity matching (EM) is a challenging problem studied by different communities for over half a century. Algorithmic fairness has also become a timely topic to address machine bias and its societal impacts. Despite extensive research on these two topics, little attention has been paid to the fairness of entity matching.

Towards addressing this gap, we perform an extensive experimental evaluation of a variety of EM techniques in this paper. We generated two social datasets from publicly available datasets for the purpose of auditing EM through the lens of fairness. Our findings underscore potential unfairness under two common conditions in real-world societies: (i) when some demographic groups are over-represented, and (ii) when names are more similar in some groups compared to others. Among our many findings, it is noteworthy to mention that while various fairness definitions are valuable for different settings, due to EM's class imbalance nature, measures such as positive predictive value parity and true positive rate parity are, in general, more capable of revealing EM unfairness.

PVLDB Reference Format:
Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. PVLDB, 16(11): 3279 - 3292, 2023.
doi:10.14778/3611479.3611525

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/UIC-InDeXLab/fair_entity_matching.

1 INTRODUCTION

Entity matching (EM) seeks to match pairs of entity records from (the same or different) data sources that refer to the same real-world entity. EM is very useful in many applications domains, including (a) healthcare, where matching of patient records from different healthcare facilities (e.g., emergency rooms, hospitals, etc.) can be used to determine if they refer to the same real-world person; (b) airline security, where airline passenger records are matched against no-fly list records to identify people who should be prevented from boarding flights or should undergo additional screening; (c) e-commerce, where product records from different retailers' websites can be matched to identify popular products and fraudulent knockoffs; and so on.

EM is a challenging problem that has been extensively investigated for over half a century by different communities, e.g., statistics, databases (DB), natural language processing (NLP), and machine learning (ML), resulting in a variety of techniques proposed in the literature for addressing this problem. These challenges arise because entities in autonomous data sources can be represented in a variety of ways (e.g., highly structured records versus textual descriptions), using different conventions (e.g., the many ways in which person names and postal addresses are represented), data quality issues (e.g., misspellings, missing values), and so on. A consequence is that, despite significant advances in recent years (especially with recent neural techniques like Ditto [36]), EM techniques still result in both false positives (non-matching record pairs that are declared as matches) and false negatives (matching record pairs that are declared as non-matches). These errors can have serious consequences in practice, as seen in the following examples.

Example 1: (No-fly list) Consider the airline security application, which aims to identify passengers that are likely to be dangerous (e.g. terrorists) for screening and potentially preventing them from boarding the flights. Using a dataset of criminal records called the no-fly list, passenger names (and other information) are matched against the no-fly list for this purpose. False positives in airline security can lead to significantly inconveniencing passengers. On the other hand, false negatives can result in known terrorists being permitted to board flights with undesirable consequences. Due to historical biases, the no-fly list datasets could over-represent some minority groups in comparison to society's population distribution. This, as we shall evaluate in our experiments, can result in higher false positive rates for those demographic groups. Another potential issue is that some demographic groups have more similar names. Hence, passengers from those groups may have a higher chance of having the same or similar information to those of known terrorists, which in turn will cause higher false positive rates for them. □

In Example 1, getting (falsely) matched is harmful. We next show an example where not getting matched can be harmful.

Example 2: (High-value customers list) Upselling to potentially high-value customers is critical for many businesses such as the fashion industry, airlines, and tourism. Suppose business A acquires

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611525

a list L of names of high-value customers from other businesses. Business A uses EM techniques to match its own customers against list L and sends exclusive offers to upsell to the matching cases in order to prioritize them. Unlike the previous example, in this case, over-representation in the high-value customer's list L is beneficial. However, unlike the no-fly list, the privileged group(s) are likely to be over-represented and minority groups under-represented in the list L. As a result, a lower (true and false) positive match rate is expected for minority groups and they end up receiving fewer exclusive offers, resulting in biased advertising [4, 9, 40]. □

When such disparities (e.g., false positives) occur in a systematic way for some demographic (sub-)groups, thereby disadvantaging them over others, concerns about the *fairness* of EM techniques arise. While the fairness of ML models has been the topic of much recent work in the literature [12, 23, 25, 25, 28, 34, 61, 62, 64], not much attention has been paid to the fairness of EM techniques.

In this paper, we seek to address this gap in the literature and perform an extensive experimental evaluation and analysis of a variety of EM techniques on a range of datasets through the *fairness lens*. Traditionally, blocking may precede matching to reduce the space of possible matching candidates from quadratic to sub-quadratic, e.g., linear. A rich body of research focuses on blocking algorithms [19, 35, 46, 47]. In this paper, our goal is to audit off-the-shelf entity matching systems used in practice. As such, our evaluation and analysis are performed on end-to-end matching systems which may include their own built-in blocking algorithms.

Summary of Contributions: In summary, we make the following technical contributions in this paper:

- Given the pairwise nature of EM, we propose the use of *single fairness* and *pairwise fairness* to evaluate entity matchers. We adopt 11 popular fairness measures from the literature for this task and analyze their suitability for EM.
- We select a suite of 13 EM techniques (including 1 declarative rule-based technique, 7 non-neural ML techniques, and 5 neural ML techniques) and 6 benchmark datasets (including 2 structured datasets, 2 textual datasets, and 2 dirty datasets) that have been used in prior work on entity matching for fairness evaluation. Using publicly available individual-level data, we also created and used two semi-synthetic matching *social datasets* for fairness evaluation. These datasets are shared publicly as benchmarks for auditing the fairness of matchers.
- We evaluated all combinations of EM techniques, datasets, and fairness measures and analyzed the outcomes. We classified the results into four cases based on whether an (EM technique, dataset, or fairness measure) yielded (i) accurate or inaccurate matching results, and (ii) fair or unfair matching results.
  Some of our findings in this study are as follows:
  - Our results on social data confirm matching unfairness when (i) there are higher similarities among records of a certain group, (ii) the representation of demographic groups in data is biased.
  - Our results underscore that responsible EM requires training data that is representative of different possibilities from various (demographic) groups.
  - While different fairness measures are valuable for different settings, due to the class-imbalance property of EM, measures such as *positive predictive value parity* and *true positive rate parity* are more capable of revealing EM unfairness.
  - Significantly relying on proxy attributes such as name, can cause unfairness in non-neural models. On the other hand, relying on pre-trained language models and embeddings, or not fully considering the dataset structure can cause unfairness in neural matchers.

  We use our findings and lessons learned to put together a set of rules of thumb for responsible entity matching (Table 8).

2 RELATED WORK

Fairness in entity resolution (ER) has briefly been studied in the literature. In [24], a constraint-based formulation for fairness is proposed to mitigate bias in ER tasks by ensuring that all (sub-)groups have the same opportunity to be resolved. Furthermore, [38] proposes a (sub-)group-based training for different ethnicities in order to increase both accuracy and fairness in SVM-based ER which is consistent with our suggestion to use ensemble learning for EM. Finally, in a parallel work [43], the authors propose an AUC-based fairness definition for EM and ER tasks and try to resolve the bias issues through a data augmentation solution. To the best of our knowledge, we are the first to comprehensively audit off-the-shelf entity matching models for fairness and propose proper measures, datasets, and comparison angles fitting the problem settings given the inherent differences with typical machine learning tasks.

3 FAIRNESS EVALUATION FRAMEWORK

3.1 Background

Given two sets of records A and B, the EM problem is to identify all correspondences between record pairs in $A \times B$ that correspond to the same real-world entity. A correspondence $c = (e_i, e_j, s)$ interrelates two records e_i and e_j with a confidence value $s \in [0, 1]$ that indicates the similarity of e_i and e_j or the confidence of a matcher about e_i and e_j referring to the same entity [33]. To decide whether the record pair of $c = (e_i, e_j, s)$ is a *match* or *non-match*, matchers often apply a threshold on s [11, 60]. We decouple the choice of a threshold from the outcome of the matching and consider the outcome of an EM task as pairs of matching and non-matching records. Formally, we consider the following EM problem:

Definition 1 (Entity Matching Problem). *Consider two sets of records A and B. For every pair of records $(e_i, e_j) \in A \times B$, let y_{ij} be the ground-truth label indicating if e_i and e_j refer to the same entity. Given all pairs $(e_i, e_j) \in A \times B$, the EM problem is to predict y_{ij} with a label h_{ij}. That is, h_{ij} refers to the decision of the matcher about the label of e_i and e_j (match or non-match).*

In a fairness-sensitive setting, records are accompanied by sensitive attributes (e.g. `gender`, `country`, `race`, etc.). Let $\mathcal{A} = \{A_1, \ldots, A_n\}$ be the sensitive attributes, $dom(A_i)$ be the domain of A_i, and $\mathcal{G} = \{g_1, \ldots, g_m\}$ be the set of all groups of interest, i.e. $\mathcal{G} = \bigcup_{A_i \in \mathcal{A}} dom(A_i)$. The mapping $L(e_i)$ relates a record to its associated groups $G_i \subseteq \mathcal{G}$. In other words, G_i is the group that e_i belongs to. Given two sets of records A and B and the set $[(e_i, e_j, G_i, G_j, h_{ij}, y_{ij})]_{\forall (e_i, e_j) \in A \times B}$, we would like to audit the fairness of a matcher with respect to groups.

3.2 Single and Pairwise Lens

3.2.1 Group Selection. The first step in auditing an entity matcher for fairness is identifying meaningful (sub-)groups in sensitive attributes. An input dataset to a matcher $\mathcal{M}$ includes record ids, the value ((sub-)group) of each record for sensitive attributes, the decisions of $\mathcal{M}$, as well as true labels for the record pairs. Depending on the type, cardinality, and the number of sensitive attributes, multiple fairness cases may happen that are presented in Table 1.

The space of groups for a single attribute with binary or multiple values is the domain of the corresponding attribute. In multiple-attribute settings, we can define intersectional subgroups, as the cartesian product of group values. More details about intersectional subgroups are provided in the technical report [50].

3.2.2 Single and Pairwise Fairness Evaluation. Given the pairwise nature of EM tasks, there are two ways to audit entity matchers:

- *Single Fairness:* The performance of a matcher is evaluated for one group s against either record in a pair. Given a correspondence $c = (e_i, e_j, h, y)$ and a group s of interest, c is legitimate, if either e_i or e_j belong to group s.
- *Pairwise Fairness:* The performance of a matcher is evaluated for a pair of groups s, s' against both records in a pair. Given a correspondence $c = (e_i, e_j, h, y)$ and a pair of groups (s, s') of interest, c is legitimate, if e_i belongs to s and e_j belongs to s', or vice versa. From an encoding perspective, we concatenate the encodings of groups s and s' into a vector c and the encodings (explained in the technical report [50]) of e_i and e_j into a vector e and validates vector e belongs to c with both directions of $\langle s, s' \rangle$ and $\langle s', s \rangle$.

We consider the EM task to be symmetric in single and pairwise fairness definitions. We remark that these definitions can be extended to ordered single and ordered pairwise fairness where the groups are defined on left or right records. In this paper, we focus on non-directional single and pairwise fairness.

3.3 Correctness

The correctness of a matcher measures how well its matching predictions conform with the ground-truth. Given a test dataset with correspondences of $t = (e_i, e_j, h, y)$, where h is a binary variable indicating the result of EM (*match* or *non-match*) for records with encodings e_i and e_j, and y is a binary variable indicating the ground-truth for matching, we profile predictions of h using the numbers of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), respectively. Unlike a classification task, in the confusion matrix of a matching task, the result is counted both for the group(s) of e_i and the group(s) of e_j. For further explanations, please refer to the technical report [50].

3.4 Fairness Measures

Similar to many of the existing works on algorithmic fairness and responsible data science [5, 15, 16, 25, 28, 42, 52, 54], we use the terms bias and unfairness interchangeably to refer to when a matcher is not fair. At a high level, fairness definitions can be viewed from three perspectives: group, subgroup, and individual fairness [12]. The most granular notion of fairness is individual fairness that requires similar outcomes for similar individuals [23]. The more popular perspective of fairness, (sub)group fairness, requires similar treatment for different (sub)groups. A model/algorithm satisfies some fairness constraints if it has equal or similar performance (according to some fairness measure) on different (sub)groups. *The focus of this paper is on (sub)group fairness.* Most of the group fairness measures belong to one of the following four categories [6, 12]. (1) *Independence* requires independence of analysis outcome from demographic groups. (2) *Separation* requires independence of the outcome from demographic groups conditioned on the target variable. (3) *Sufficiency* requires independence of the target variable from demographic groups conditioned on the outcome. (4) *Causation* requires that in a counterfactual world, the decision would not change had the individual belonged to a different demographic group. We do not consider Causal fairness in our audit. In Table 2, we present our suite of fairness measures, adapted from the notions of fairness in classification [12], for auditing an entity matcher $\mathcal{M}$ for a set $\mathcal{G}$ of (sub)groups.

3.5 Selecting Fairness Measures for EM

Depending on the context of an EM task at hand, proper fairness measures should be employed. Besides, a major difference between EM and regular classification tasks is that the input to EM tasks is a pair of records. Due to its pairwise matching nature, *class imbalance* is a distinguishing property of EM, compared to regular classification tasks. To better explain this, let us consider a toy example, where two data sources D and D' contain exactly the same set of n records. Each pair of records $e \in D$ and $e' \in D'$ is passed as an input to an entity matcher. In this setting, only n of the n^2 pairs are matches, and the others are non-matches. In other words, the probability a random pair is a match is as low as $\frac{1}{n}$. Class imbalance is also a challenge in some of classification problems but the degree of imbalance is often constant while in EM tasks, it is as high as $O(n)$. Indeed, blocking techniques [46] can help in reducing the extreme class imbalance. Even after blocking, a high class imbalance is expected for EM tasks.

Guide for Practitioners: Let us consider the fairness measures in Table 2. Which measures to choose depends on the downstream task and the problem context. Therefore, practitioners should choose measures depending on the importance of TPs (true match), FPs, FNs, and TNs in the problem context. For example, among the fairness measures, *statistical parity* does not consider the ground-truth labels and requires equal match ratios from different groups, independent of whether they really are a match or not. As a result, this measure does not seem reasonable for deduplication tasks using EM. On the other hand, it may be useful to ensure equal representation of different groups when using EM for joining tables.

When the input to the EM task is imbalanced and most of the pairs are non-matches, some measures are more capable of revealing the unfairness of matchers. First, note that even a matcher that marks all pairs as non-matches has high accuracy in this setting. Subsequently, accuracy parity may not reveal the unfairness. Similarly, measures such as FPRP and TNRP may fail to reveal unfairness in detecting true matches. In these settings, the fairness measure for successfully discovering these events is **Positive Predictive Value Parity (PPVP)**. Another important measure in this context is **True Positive Rate Parity (TPRP)**, a.k.a **Equal Opportunity**, which focuses on correct match predictions among the (rare) true



Causal Data Integration

Brit Youngmann
CSAIL MIT
brity@mit.edu

Michael Cafarella
CSAIL, MIT
michjc@csail.mit.edu

Babak Salimi
University of California, San Diego
bsalimi@ucsd.edu

Anna Zeng
CSAIL, MIT
annazeng@mit.edu

ABSTRACT

Causal inference is fundamental to empirical scientific discoveries in natural and social sciences; however, in the process of conducting causal inference, data management problems can lead to false discoveries. Two such problems are (i) not having all attributes required for analysis, and (ii) misidentifying which attributes are to be included in the analysis. Analysts often only have access to partial data, and they critically rely on (often unavailable or incomplete) domain knowledge to identify attributes to include for analysis, which is often given in the form of a causal DAG. We argue that data management techniques can surmount both of these challenges. In this work, we introduce the Causal Data Integration (CDI) problem, in which unobserved attributes are mined from external sources and a corresponding causal DAG is automatically built. We identify key challenges and research opportunities in designing a CDI system, and present a system architecture for solving the CDI problem. Our preliminary experimental results demonstrate that solving CDI is achievable and pave the way for future research.

PVLDB Reference Format:
Brit Youngmann, Michael Cafarella, Babak Salimi, and Anna Zeng. Causal Data Integration. PVLDB, 16(10): 2659 - 2665, 2023.
doi:10.14778/3603581.3603602

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/brityoungmann/CDI/blob/main/README.md.

1 INTRODUCTION

Causal inference lies in the heart of empirical research in natural and social sciences and is commonly used in multiple disciplines, including sociology, medicine, and economics [26, 33]. It aims to answer causal queries, such as, "Does being overweight cause coronary heart disease – independent of cholesterol, and diabetes?" or "Do tobacco advertisements entice adolescents to buy more cigarettes regardless of whether their parents smoke?" Causal inference enables analysts to answer causal questions about attributes from a dataset, ultimately enabling them to make real-world discoveries. Pearl's framework [40], which we adapt in this work, provides a principled way to causal inference using structural causal models.

Cause-effect questions are designed to determine whether an *exposure* variable (a pain reliever) causes or affects an *outcome* variable (pain). To correctly estimate causal effects, one must consider *confounding variables*—variables that influence both the exposure and outcome and thus might distort the association between them. Otherwise, one might draw perplexing conclusions due to *confounding bias* [41] that can lead to overestimation or underestimation of the association between exposure and outcome. The internal validity of a study heavily relies on how well confounding bias has been addressed. However, to determine a sufficient set of confounding variables to account for confounding bias, *background knowledge* about the data generative process is required. This knowledge is often given in the form of a *causal DAG* depicting causal relationships between the attributes [40] (and is wildly used in econometric and social sciences [26, 33]). Pearl [40] presented sufficient and necessary conditions for identifying[1] the *adjustment set* of variables to include in the analysis, which can be checked against a causal DAG. However, analysts often lack such a DAG [22]. While associational assumptions are testable from data, causal relations cannot, in general, be fully recovered from data [41].

Table 1: Example Input Dataset.

State	Mask Policy	Confirmed Cases	New Cases	Recovered	Death cases
MA	yes	121046	2740	4980	109
FL	yes	640978	24349	25140	55
CA	no	735235	31150	42170	34
SD	no	15300	1791	2083	49

We identify two fundamental challenges for conducting valid causal inference: First, critical confounding variables may not be included in the data. Second, even if they are included, the background knowledge required to identify the correct adjustment set of variables might be missing. We argue that data management techniques and ideas can surmount both of these challenges. We illustrate that via the following example:

Mary, a data analyst in the WHO organization, aims to estimate the (treatment) effect of a mask policy on the coronavirus mortality rate. To do this, she examines a dataset containing Covid-19-related facts in multiple states in the US (an illustration of this dataset is given in Table 1), and uses a Causal analysis tool (e.g., [49, 53]). To correctly estimate the effectiveness of face masks, Mary must consider confounding variables. However, there are confounding variables that are not included in her data. For example, the weather affects people's willingness to wear masks and Covid-19 death rate [47]. Moreover, to determine a sufficient set of confounding variables, Mary must use background knowledge of causal relationships she may lack.

In this example, variables missing from the dataset are critical for the analysis to avoid confounding bias. Thus, Mary must *integrate her data* with external data sources. This is a laborious task, typically done using data discovery and integration tools (e.g., [36, 59]) by skilled programmers—which is often not the case for social or

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603602

[1] In causal inference, identifiability typically refers to the conditions that permit measuring causal effect from observed data. Here, we discuss the ability to identify a sufficient adjustment set of variables for accurate causal inference.

natural scientists. We argue this is the case for many real-life scenarios. After augmenting the data to include unobserved variables, the next step is determining a sufficient adjustment set of variables to account for confounding bias. However, Mary does not have the required background knowledge to do that.

Our Vision: Our goal is to mine unobserved confounding variables and missing background knowledge needed to answer causal questions. To this end, we introduce the Causal Data Integration (CDI) problem. CDI is an effort to augment an input dataset with unobserved variables and background knowledge required for causal analysis. To address the challenge of finding unobserved variables, the system mines variables from external sources. To overcome the challenge of identifying the right adjustment set of confounders, the system then builds a corresponding causal DAG, ensuring the adjustment set is sufficient for controlling the confounding bias. A CDI system may greatly reduce the manual effort devoted to performing causal analysis, allowing analysts to get all required data and background knowledge quickly and effectively.

CDI may also be beneficial for the data management community. Causal inference has been shown to be useful for various data management tasks, including query result and classifier explanation [8, 42, 56], and hypothetical reasoning [21]. However, some of these studies make strong assumptions about the underlying causal DAG that may not hold in practice [26, 33]. For example, [21] assumes that the DAG is known, and [42] assumes that all confounding variables are present in the input dataset. To address these issues, a CDI system can be utilized to construct a causal DAG for a given dataset while integrating relevant missing attributes. This not only addresses the limitations of existing studies but also enables the use of causal inference in more complex and realistic scenarios.

In this work, we show how a CDI system can be accomplished.

Measure of Success: Distinct from the classical data integration task, where success is defined by its ability to combine data from different sources and provide users with a unified materialized view of them, the success of a CDI system is defined by the ability to discover all relevant unobserved variables and recover the correct set of confounders for a causal question. The CDI task is potentially easier than data integration. However, its impact is greater, as it enables analysts to conduct causal analysis more easily. A CDI system enables analysts to include all confounding variables in the analysis; failing to do so may lead to false discoveries, incorrect medical diagnoses, and erroneous conclusions [26, 33]. In our example, a CDI system mines, among others, attributes describing the weather and population density per state, and infers causal relations among the attributes in the augmented dataset. A successful CDI system would assist Mary in identifying the right adjustment set of attributes to account for confounding bias, allowing her to accurately estimate the effect of an enforced mask policy on the mortality rate.

Challenges & Opportunities: The first challenge is to augment an input dataset with unobserved variables, ensuring the *output causal DAG is complete* (i.e., there are no unobserved confounding variables). Data discovery methods have been extensively researched to identify relevant data sources that can be integrated with an input dataset for attribute extraction [12, 14]. However, these methods are not specifically designed to discover unobserved attributes that are relevant for cause-effect estimation. This presents a unique opportunity to develop dedicated data discovery tools tailored for identifying unobserved confounding variables.

Extracted attributes may contain data quality issues, such as outliers and missing values. These issues may lead to erroneous conclusions, compromising the reliability of causal inference. In particular, missing values pose a significant challenge in causal inference as they may result in *selection bias* [10], where certain samples are preferentially excluded from the data. Other data quality issues, such as inconsistent data formats and data duplication, can also impact the accuracy of causal inference analysis. Therefore, in addition to handling missing values, it is crucial to ensure that the system is robust to various data quality issues to ensure the validity of causal inference results.

The challenge of identifying the correct adjustment set and constructing a causal DAG is particularly difficult when dealing with hundreds of extracted attributes. A human-driven causal DAG construction (from domain expertise) is infeasible in this scenario, and automatic methods for building a full causal DAG may not be feasible either [22]. To this end, we propose the construction of a cluster causal DAG (C-DAG) [9], which groups related attributes and only specifies causal relationships between clusters of attributes. By doing so, the correct adjustment set of attributes can still be identifiable from this C-DAG. This approach reduces cognitive overhead for analysts and makes causal analysis possible for high-dimensional data, which is currently not feasible for existing tools (e.g., [49, 53]). However, a main challenge is how to maintain the necessary level of granularity to ensure accurate causal inference. The limitations of existing solutions and challenges in constructing the desired C-DAG are discussed, and a limited prototype implementation for solving the CDI problem is presented with preliminary experimental results demonstrating that solving CDI is achievable.

2 THE CDI PROBLEM

2.1 Data Model and Background

The input dataset $\mathcal{D}$ contains the exposure (T) and outcome (O) attributes. A *causal DAG* for $\mathcal{D}$ is a DAG whose nodes are the attributes and whose edges capture all causal relationships between the attributes [40]. Generally, because causes must precede effects, the literature considers causal DAGs rather than graphs [40]. Causal DAGs provide a simple way of graphically representing causal relationships and are particularly helpful in understanding potential sources of bias in causal estimations. It is well-known that background knowledge is required to determine a causal DAG for a given dataset [41]. A causal DAG can only be as good as the background information used to create it [38]; a DAG is complete and therefore has a causal interpretation only if it contains all common causes of any two variables. In Section 3.3, we review existing approaches for building a causal DAG for a given dataset and discuss their limitations. We assume the analyst is interested in investigating a causal relationship between T and O. She may wish to estimate, e.g., *the direct, indirect, or total effect* [40]. The total effect of T on O is the total extent to which O is changed by the T. It is equal to the sum of the direct and indirect effects. Causal questions about total and (in)direct effect could be answered by identifying the right set of *confounding* (that influence both T and O) and *mediating* (that

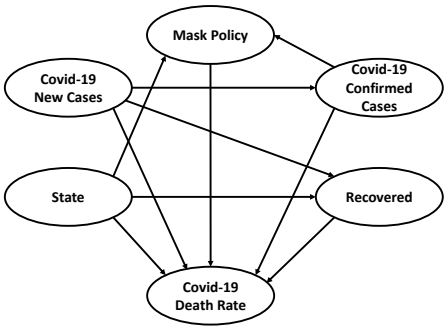


Figure 1: Example causal DAG for Table 1

mediate the relationship between T and O) variables. Such variables could be identified using graphical criteria (e.g., the backdoor criterion [40]) that can be checked against a causal DAG.

Example 2.1. In our example, T is `mask policy`, and O is `death cases`. A corresponding causal DAG is shown in Figure 1. To obtain a reliable estimate of the total effect of T on O, it is crucial to adjust for all relevant confounding variables. However, mistakenly assuming that the DAG contains all relevant information, the adjustment set only includes the attributes `state` and `confirmed cases`. As is common in causal inference, missing confounders can lead to biased estimates. Thus, it is necessary to consider external data to incorporate missing variables.

2.2 Problem Formulation & Challenges

Given an input dataset, the system mines unobserved confounding attributes relevant for a given causal estimation, yielding an augmented dataset $\widehat{\mathcal{D}}$. It then maps the attributes in $\widehat{\mathcal{D}}$ into a causal DAG, to enable analysts to determine the adjustment set. We next outline key challenges in designing such a system:

(1) Completeness: Unobserved variables can dramatically affect the quality of causal analysis [32, 41]. Thus, the first challenge is to *ensure the generated DAG is complete*, i.e., there are no unobserved confounding variables w.r.t. the provided data sources. Our goal is to ensure that all relevant information can be automatically extracted from the provided sources. However, completeness, in general, cannot be guaranteed since relevant variables may exist outside of the provided data sources.

(2) Robustness: The robustness of a CDI system is determined by its ability to handle various data quality issues that may arise during the integration process. As in general data integration, there are multiple data quality issues that a CDI system has to face, including missing values and outliers. The impact of such issues on causal inference is particularly significant. Unlike in machine learning, where data quality issues may only result in a slight drop in accuracy, these issues can lead to false discoveries in causal inference if not handled properly. This is because if these issues are not randomly distributed across the exposure groups, they can significantly impact the validity of the resulting conclusions. One common issue that can significantly impact causal inference is missing data. Extracted attributes may contain missing values, which can lead to selection bias and erroneous conclusions. Thus, a key challenge is to ensure the system is robust to various data quality issues.

Table 2: Extracted attributes from different sources.

Extracted Attributes from US Open Data

State	Population size	Population density
MA	6,981,974	901
FL	22,244,823	402
CA	39,029,342	254
SD	909,824	12

Extracted Attributes from DBPedia

State	Governor	snow inch	Avg temp.	Min temp.
MA	Maura Healey	51.05	48.14	23
FL	Ron DeSantis	-	71.8	70
CA	Gavin Newsom	-	61.17	54
SD	Kristi Noem	37.43	45.54	34

(3) Conciseness: We can potentially extract hundreds of attributes from given data sources, which can lead to a complex, high-dimensional causal DAG. To overcome this, the CDI system should aim to reduce users' cognitive overhead in interpreting the DAG. It is also crucial to ensure the scalability of causal analysis, as existing tools are not feasible for high-dimensional data [49, 53]. One approach is to ensure that related attributes are grouped into clusters while still ensuring that the correct set of confounders is identifiable. Thus, a third challenge is to ensure the output DAG is concise. The CDI system should be able to construct a causal DAG that effectively captures the causal relationships between attributes while maintaining the necessary level of granularity to ensure accurate causal inference. This will enable analysts to efficiently handle high-dimensional data while facilitating information elicitation about the validity of the assumptions used for drawing causal inferences.

3 THE CDI SYSTEM

Our system operates as a 3-steps pipeline. First, the *Knowledge Extractor*, extracts candidate attributes to handle the *completeness* challenge. The *Data Organizer* then handles data quality issues to account for the *robustness* challenge. Last, the *Causal DAG Builder* builds a clustered casual DAG to handle the *conciseness* challenge. We next present the key challenges and research opportunities for each system component.

3.1 The Knowledge Extractor

This module receives a table $\mathcal{D}$ (containing T and O) and sources of knowledge. From each source, it extracts attributes representing additional properties of entities from $\mathcal{D}$. A successful Knowledge Extractor mines all unobserved variables that should be included in the analysis. The primary goal of the *Knowledge Extractor* is to extract all relevant variables from the provided data sources to enable reliable causal inference. In causal inference, the *unconfoundedness assumption* [39] (the assumption that all variables affecting both the exposure T and the outcome Y are observed and can be controlled for) is a key consideration. This assumption is similar to the closed-world assumption in database in that it assumes that the database is complete. However, since complete knowledge of all variables affecting both T and O is often unattainable, the focus should be on extracting all relevant variables from the provided data sources. The module is designed to ensure that as much relevant information as possible can be automatically extracted from the provided sources, and its capabilities can be expanded over time to incorporate new sources or refine the extraction and integration process. We next provide an example of how it works with specific data sources.

Data Lakes: There has been unprecedented growth in the volume of publicly available data (e.g., the US Federal Open Data [6] and the



Discovering Similarity Inclusion Dependencies

YOURI KAMINSKY*, Hasso Plattner Institute, University of Potsdam, Germany
EDUARDO H. M. PENA, Federal University of Technology – Paraná, Brazil
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany

Inclusion dependencies (INDs) are a well-known type of data dependency, specifying that the values of one column are contained in those of another column. INDs can be used for various purposes, such as foreign-key candidate selection or join partner discovery. The traditional notion of INDs is based on clean data, where the dependencies hold without exceptions. Unfortunately, data often contain errors, preventing otherwise valid INDs from being discovered. A typical response to this problem is to relax the dependency definition using a similarity measure to account for minor data errors, such as typos or different formatting. While this relaxation is known for functional dependencies, for inclusion dependencies no such relaxation has been defined.

We formally introduce *similarity inclusion dependencies*, which relax the inclusion by demanding the existence only of sufficiently similar values. Similarity inclusion dependencies can fulfill traditional IND use cases, such as foreign-key candidate discovery, even in the presence of dirty data. We present Sawfish, the first algorithm to discover all similarity inclusion dependencies in a given dataset efficiently. Our algorithm combines approaches for the discovery of traditional INDs and string similarity joins with a novel sliding-window approach and lazy candidate validation. Our experimental evaluation shows that Sawfish can outperform a baseline by a factor of up to 6.5.

CCS Concepts: • **Information systems** → **Information integration**; *Data mining*.

Additional Key Words and Phrases: data profiling, data lake, joinability

ACM Reference Format:
Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1, Article 75 (May 2023), 24 pages. https://doi.org/10.1145/3588929

1 INCLUSION DEPENDENCY

Data profiling is the process of extracting metadata from datasets. Data dependencies are an important type of metadata and, thus, have a crucial role in data profiling. There are different forms of data dependencies, e.g., functional dependencies (FDs) and inclusion dependencies (INDs). In particular, INDs express that the tuples of one column-combination are contained in the tuples of another column-combination. We call an IND unary, if it holds between two individual columns. INDs help data practitioners to understand and structure unknown data, in particular, in discovering foreign key candidates and joinable partners [20]. Moreover, INDs have assisted schema design in [15] and can improve query execution [12].

*Corresponding author (youri.kaminsky@hpi.de)

Authors' addresses: Youri Kaminsky, Hasso Plattner Institute, University of Potsdam, Germany, youri.kaminsky@hpi.de; Eduardo H. M. Pena, Federal University of Technology – Paraná, Campo Mourão, Paraná, Brazil, eduardopena@utfpr.edu.br; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART75 $15.00
https://doi.org/10.1145/3588929

Traditional IND discovery assumes clean data: *all* tuples of the dependent column-combination must be exactly *equal* in the referenced column-combination. However, the ever-increasing volume of data also leads to more "dirty" data [18]. Thus, relaxed dependencies have been introduced to deal with erroneous data [4].

One example of relaxed dependencies is matching dependencies (MDs), which generalize the concept of functional dependencies (FDs) [23]. For a traditional FD, the tuples on the dependent side must be equal for all equal tuples on the determinant side. In contrast, MDs use similarity measures instead of the strict equality constraint. In other words, an MD holds if, for all *similar* tuples on the determinant side, all tuples on the dependent side are also *similar*. MDs allow data practitioners to address typical use cases of FDs, such as schema normalization, even in the presence of dirty data. Moreover, MDs can also be used for duplicate detection [23]. A feature of MDs is that they support arbitrary similarity measures and a configurable similarity threshold to balance the error tolerance and over generalization.

There is no corresponding notion yet for INDs to allow for similar values. Thus, we introduce similarity inclusion dependencies (sINDs). In contrast to traditional INDs, sINDs use a similarity measure to define inclusion. An sIND holds if, for all dependent values, there exists a referenced value that is at least *similar*. Like MDs, sINDs support arbitrary similarity measures and configurable similarity thresholds. For this work, we consider representatives of both edit-based and token-based similarity measures: the edit distance and the Jaccard similarity.

sINDs are a natural way to handle dirty data. Besides traditional IND use cases, sINDs can also identify sources of possibly erroneous data in data lakes. If a candidate IND does not hold, but its respective sIND holds, the data can be analyzed and used to identify erroneous relations. Either only one of the relations contains errors, so we can use the other relation to fix these, or both relations contain errors, and the data needs to be cleaned more thoroughly.

To illustrate the usefulness of sINDs, we present an example in Table 1. The tables are for a fictitious soccer tournament. While one table shows the results after all teams played against each other, the other table presents an aggregation of the participation forms of all goalkeepers in the tournament. We would assume that the values of the *club* column of Table 1b are contained in the values of the *name* column of Table 1a. However, multiple goalkeepers made minor spelling mistakes in their club name. Despite these mistakes, we want to discover the dependency. On the one hand, we are interested in joining these columns to know which goalkeeper played for the most successful team. On the other hand, we could perform a data cleaning task, because all mistakes are only in the *club* column of Table 1b. Thus, there is a matching counterpart in the other column to automatically correct the errors.

Table 1. Example relations of a soccer tournament

(a) Final Results

results	
name	points
SpVgg Beelitz	4
Potsdamer SC	9
SpVgg Bernau	4
VfL Potsdam	0

(b) Participating Goalkeepers

goalie	
p_id	club
202	SpVgg Beelitz
216	SpVgg Beelittz
469	Potsdamer SC
617	SpVgg Be_nau
692	ViL Potsdam
853	Potsdamer SC

There are other known extensions to INDs that can deal with dirty data. One example is partial INDs [2], which allow a certain portion of the dependent tuples to not be present in any form in the referenced tuples. Therefore, partial INDs are especially useful when dealing with incomplete columns. However, partial INDs provide no insight into what kind of matching failures occur. If a significant portion of tuples have a similar, but not equal counterpart, no partial INDs would be found. Since half of the goalkeepers in our example have misspelled their club names, we would need to set the error threshold above 50% to find a partial IND. In a typical database, this threshold would lead to many spurious dependencies. A special form of partial dependencies are conditional INDs (cINDs), which hold only for tuples that fulfil a certain condition [3, 1].

Another example are approximate INDs [13]. They are discovered on a sample of the complete data and show that an IND holds only with a certain probability. This relaxation is used to increase the discovery speed, but not the generality of the found IND. Moreover, approximate INDs also provide no insight into data errors. In our example, typical sampling strategies would lead to different value sets for the columns. Therefore, we would not find a dependency in our samples.

Apart from introducing the concept of similarity inclusion dependencies, we present **Sawfish**, the first approach to efficiently discover sINDs in a given dataset. In particular, we make the following contributions:

(1) We introduce the formal concept of similarity inclusion dependencies.
(2) We propose Sawfish, an efficient approach to discover all unary sINDs in a given dataset using the edit-distance and the Jaccard similarity measure.
(3) We offer an in-depth evaluation of Sawfish, comparing our approach to the state-of-the-art IND discovery algorithm Binder [22] and a naive baseline. We show how Sawfish scales with the size of the input data and different algorithm configurations.
(4) We present a case study that explores the usefulness of the discovered sINDs. We have manually built a ground-truth with over 1000 sINDs and publicly provide the annotations.
(5) We integrate Sawfish on the Metanome data profiling platform [21], so it can be easily used with a variety of datasets and compared to other data profiling algorithms. All code is made publicly available.

The remainder of this work is structured as follows. In Section 2 we present related research on inclusion dependencies, other relaxed dependencies and string similarity joins. We formally define sINDs in Section 3. Section 4 shows how to efficiently discover sINDs and presents the general principle of Sawfish. We present an exhaustive evaluation of Sawfish in Section 5. Finally, Section 6 draws a conclusion and gives insights into the limitations of Sawfish and provides an outlook on future work.

2 RELATED WORK

Although there has been no research on similarity inclusion dependencies yet, three research areas overlap with this work: traditional inclusion dependency discovery, other relaxations of dependencies, and string similarity joins.

Inclusion dependencies (INDs) are a well-known type of data dependency and have several algorithms to discover them from data— see Dürsch et al. for an overview of approaches [9]. There are approaches for both unary and n-ary IND discovery. De Marchi et al. presented an early algorithm to discover unary INDs by intersecting the attribute sets for each value [6]. Many also discovers unary INDs, but focuses on the problem of finding INDs in millions of tables [24]. A prominent representative for n-ary, i.e., inclusion between tuples of attribute lists, IND discovery is Zigzag [7]. It combines both up- and downwards pruning to discover maximal n-ary INDs efficiently.



Effective Entity Augmentation By Querying External Data Sources

Christopher Buss
Oregon State University
bussch@oregonstate.edu

Jasmin Mousavi
Oregon State University
mousavij@oregonstate.edu

Mikhail Tokarev
Oregon State University
tokarevm@oregonstate.edu

Arash Termehchy
Oregon State University
termehca@oregonstate.edu

David Maier
Portland State University
maier@pdx.edu

Stefan Lee
Oregon State University
leestef@oregonstate.edu

ABSTRACT

Users often want to augment and enrich entities in their datasets with relevant information from external data sources. As many external sources are accessible only via keyword-search interfaces, a user usually has to manually formulate a keyword query that extract relevant information for each entity. This approach is challenging as many data sources contain numerous tuples, only a small fraction of which may contain entity-relevant information. Furthermore, different datasets may represent the same information in distinct forms and under different terms (e.g., different data source may use different names to refer to the same person). In such cases, it is difficult to formulate a query that precisely retrieves information relevant to an entity. Current methods for information enrichment mainly rely on lengthy and resource-intensive manual effort to formulate queries to discover relevant information. However, in increasingly many settings, it is important for users to get initial answers quickly and without substantial investment in resources (such as human attention). We propose a progressive approach to discovering entity-relevant information from external sources with minimal expert intervention. It leverages end users' feedback to progressively learn how to retrieve information relevant to each entity in a dataset from external data sources. Our empirical evaluation shows that our approach learns accurate strategies to deliver relevant information quickly.

PVLDB Reference Format:
Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. Effective Entity Augmentation By Querying External Data Sources. PVLDB, 16(11): 3404 - 3417, 2023.
doi:10.14778/3611479.3611535

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/bussch/QueryBasedEntityAugmentation.

1 INTRODUCTION

There is a recognized need to collect and connect information from a variety of data sources [14, 18, 23]. As an example, we have recently worked in a large-scale NIH-funded project to augment the information of biomedical entities by querying other biomedical data sources [49]. The main focus of this project is to *repurpose* current drugs to treat or mitigate the symptoms of diseases for which there is insufficient time or resources to develop effective treatments (e.g., new or rare diseases) [2]. Biomedical researchers often have some local dataset of available drugs (e.g., a dataset of *FDA-approved uses* of drugs). Given a drug in the local dataset, a researcher usually needs to query external data sources to find additional information about the drug (e.g., its *off-label uses*).

Due to a lack of access or resources, external information often must be retrieved through querying [14, 47]. Many data sources are only accessible via query interfaces or APIs. Even with access, it may require too much of a resource (e.g., storage space, time) to download and maintain an up-to-date copy of the external dataset. Thus, information relevant to some local entity must often be gathered on a as-needed basis by querying external data sources. For example, as many biomedical data sources are available only via query APIs, the users of the aforementioned drug repurposing data collection system must often query the information relevant to their current drug of interest through query APIs.

However, formulating a query that extracts specific information can be troublesome. Different data sources often represent the same concept in distinct forms [12, 15] such that one needs to tailor their query to specific external data sources. Figure 1 illustrates a case where users have a local dataset of FDA-approved uses of drugs, named *FDA-Approved Uses*, and would like to query an external data source that contains the off-label uses of those drugs, named *Off-Label Uses*. A drug that is identified by one of its brand names (e.g., *Zoloft*) in *FDA-Approved Uses* is referred to by its generic name (e.g., *Sertraline*) in *Off-Label Uses*. Because of heterogeneities, one may not know how to query for a specific external entity prior to investigating the content and structure of the data in the external source. Consider a biomedical researcher who seeks additional information about the drug *Zoloft* in their local dataset. Since they are only aware of the structure and content of their local dataset, they query the external data source for *Zoloft*, but that elicits no results. They try again using a much more general description of *Zoloft* (i.e., being a *serotonin reuptake inhibitor*). However, their under-specified query produces many results, most of which are irrelevant (i.e., contain information about drugs that are not *Zoloft*). After additional trial-and-error, they find a query that retrieves *Sertraline*. More work is required to then merge the local and external entities into one coherent representation.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611535

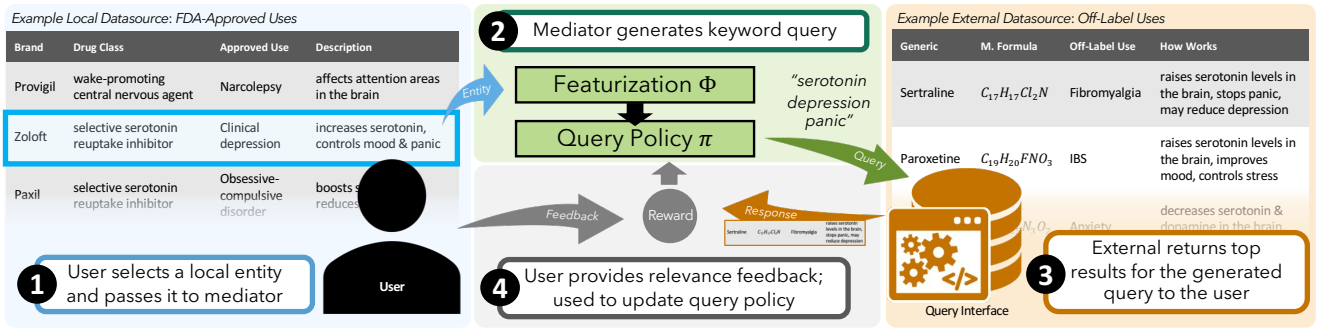


Figure 1: An example of our framework for a single user and single external data source. The user selects (by query, GUI, etc,.) the local entity *Zoloft*. The mediator uses its learned query policy to extract the relevant entity (*Sertraline*) from the external source. The user provides relevance feedback on the results, which is then used to further refine the mediator's querying policy.

Manually querying for specific external entities takes too much time and financial resources. Continuing our example, if the researcher needs additional information for another drug in their local dataset, they will need to repeat the entire process. Moreover, if they need information from additional external data sources, then the work required to query for each drug is greatly exacerbated. Furthermore, any other researcher with a similar information need must *repeat the same such work themselves.*

To alleviate the burden, one can use a **shared system that automates query formulation**. This **mediator system** acts as a go-between for users and external data sources: a user specifies a local entity (e.g., *Zoloft*) perhaps through a query or a graphical user interface, and the mediator maps the local entity to queries that retrieve the relevant external entities (e.g., *Sertraline*) from their respective external sources.

To the best of our knowledge, such mediators are currently created by *manually writing programs* that generate queries for specific external sources to retrieve relevant records to a given local entity. Each program implements a set of manually written rules specific to its external source. These rules *cannot* necessarily be reused across data sources. Thus, the mediator requires a significant amount of labor and expert attention to build and maintain. Instead of conducting their own research, biomedical researchers in our NIH-funded project spend most of their time writing these programs and investigating the content and structure of every external source to ensure that the programs formulate appropriate queries.

In this paper, we examine methods for *learning the mediator's query policy online* through user interaction. As illustrated in Figure 1, after the user specifies a local entity, the mediator formulates a query to retrieve records from an external source according to its *query policy* and shows the returned external records to the user. The user then provides feedback on the relevance of the returned records to the local entity. Our mediator learns to revise its query policy and improves its performance using the user's feedback.

An alternative to this online learning paradigm is to use offline training data to learn query formulation but collecting and labeling such data still requires considerable manual effort from domain experts [14]. Particularly, it is challenging to gather useful training data from external sources. The data collection/labeling might need to be repeated as the external datasets evolve. In many domains (e.g., drug repurposing for emerging viral diseases), users cannot wait long to prepare offline training data.

Of course, online learning of query policies comes with its own set of challenges. First, the mediator should learn to formulate reasonably accurate queries over external sources early on. We assume the mediator must be effective in the *short run* so users will continue to provide feedback. It is particularly difficult to meet this goal over large local or external datasets as the amount of required feedback for accurate learning generally grows with the number of entities. Second, the mediator should improve its querying policy and increase the effectiveness of its results in the *long run*. Online learning literature indicates that a policy that is effective in the *short run* (i.e., meets the first challenge) might not be accurate in the *long run* as it might become biased to early observations or decisions that do not deliver accurate results in the *long run* [42]. Third, due to lack of prior knowledge about the precise content and structure of the relevant external information, the number of candidate queries for a local entity might be enormous. This *large search space* makes finding effective queries difficult.

Due to the wide-spread use of keyword query interfaces over external sources, we develop online learning methods for formulating keyword queries. There are systems for automatic keyword query formulation, but they assume returned results are always relevant, which is not usually true and is the challenge that we address [47] (see Section 8). Our contributions are as follows:

- We present a framework for on-demand collection of relevant external entities only accessible via query interfaces (Section 2).
- We define the problem of online query-policy learning within the context of the aforementioned framework (Section 3).
- We present a method that learns a separate query policy for each individual local entity. We show that this approach does not scale to large local datasets as it might require a great deal of user feedback (Section 4).
- We propose an entity-conditional method that learns a query policy jointly over all local entities. This approach significantly reduces the amount of user feedback required to learn effective query policies. To overcome representational heterogeneity across the local and external sources, we propose techniques to use features and keywords from the external results in our model and queries, respectively (Section 5).
- If the local dataset contains many diverse entities, it might not be possible to learn effective queries for many entities using a shared model. Hence, we propose an approach that gradually

replaces a shared model with entity-specific ones based on the effectiveness of the shared model. The resulting models will retain the desirable properties of the shared model in the *short run* and learn more effective queries in the *long run* (Section 6).

- We explore whether the broad language understanding capabilities of state-of-the-art large language models can improve query generation. We train a neural network over features extracted by large-scale pretrained Longformer [5] and LLaMA [44] models to serve as our query policy (Section 6).
- We perform extensive empirical studies using six pairs of real-world datasets from different domains, including biology, products, and news. Our studies indicate that our proposed methods learn reasonably effective queries quickly and improve their accuracy in the long run over large datasets (Section 7).

2 GENERAL FRAMEWORK

The **mediator** wraps the local dataset and the query interface over the external data source. We assume the mediator has full access to the local dataset, but can only access external datasets through their query interfaces. Given a user-specified entity from the local dataset, the mediator must devise and submit a query to the interface to extract external entities relevant to the given local entity. This framework is not tied to a particular method by which a user specifies the local entity (e.g., through query or GUI).

Local Dataset. To simplify our exposition, we assume the local dataset is a single relational table where each tuple stores information about a distinct entity. One may extend our approach to multi-relational datasets by defining an entity as the join of its related tuples. We denote the set of local dataset entities as $\mathcal{E}$.

External Dataset. Like the local dataset, we model the external dataset as a set of entities (i.e., tuples). Given local entity e and external dataset D, $X(e) \in D$ represents the external entity that is relevant to the local one. The definition of "relevant entity" depends on the domain. For example, a clinical trial is relevant to the drug that it concerns. For notational convenience, we assume only one relevant external entity exists for each local entity, however, in the case of more than one, we can easily extend $X(e)$ to be the set of all relevant entities. If no relevant entities exist, then extracting $X(e)$ is impossible regardless of the method used. Thus, to accurately evaluate our methods, we assume that $X(e)$ always exists.

Example 1. Figure 1 shows excerpts of a local (left) and an external (right) dataset. $\mathcal{E}$ consists of all drugs in *FDA-Approved Uses*. If e is *Zoloft* then the relevant tuple $X(e)$ in *Off-Label Uses* is *Sertraline*. We show the content of $X(e)$ for explanation's sake. In a real setting, the content of $X(e)$ would not be known a priori.

Querying Policy. We call the queries submitted by the mediator to the external data source *mediator queries*. We denote the set of *all possible* mediator queries as $\mathcal{Q}$. $\mathcal{Q}$ is a subset of the queries accepted by the external query interface. The precise definition of $\mathcal{Q}$ varies based on the characteristics and capabilities of the external query interfaces. A *querying policy* (*policy*) is a mapping $\pi : \mathcal{E} \rightarrow \mathcal{Q}$. To our knowledge, policies are traditionally written manually.

Example 2. Given e = *Zoloft*, the mediator must devise a keyword query to extract $X(e)$ = *Sertraline*. One policy is to use the content of the input entity (*Zoloft*) within the output mediator query. However, the content in `Brand` and `Approved Use` are likely unique to the local dataset. Given this observation, assume the mediator's policy ignores terms from `Brand` and `Approved Use` and prefers terms from `Drug Class` and `Description`. Thus, this policy maps e (*Zoloft*) to the keyword query *"serotonin depression panic"*.

Query Result. External query interfaces usually return results of query q as a list of entities inverse sorted based on the degree by which the query interface deems the entities relevant to q. More precisely, the *result* of query $q \in \mathcal{Q}$, $D[q]$, is a list of entities in D.

Query Effectiveness. Ideally, we would like the mediator query q submitted for local entity e to return the external entity relevant to e, i.e., $X(e)$ is placed in a relatively high position in $D[q]$. Given mediator query q for local entity e, we define the effectiveness of q over external dataset D as a real-valued function $f(X(e), D[q])$ whose range is in $[0, 1]$. The precise mechanics of f depends on the domain. For instance, there are standard metrics in information retrieval and data management to measure how effectively queries achieve this goal given their returned results [37]. For example, *precision@k* is the fraction of relevant answers in the top-k returned results. Another frequently used metric is *reciprocal rank* (RR) $\frac{1}{r}$ where r is the position of the first relevant answer. One metric may be more appropriate than another for a specific setting. For instance, reciprocal rank may be a better indication of effectiveness than precision@k if there are at most a couple relevant answers to the query. One can choose f based on the domain. In this paper, we use reciprocal rank.

Example 3. The mediator submits q = *"serotonin depression panic"* to the query interface over the external dataset in Figure 1, which returns the ranked results $D[q]$ = (*Paroxetine*, *Sertraline*). Since $X(e)$ = *Sertraline*, the reciprocal rank of these results would be $\frac{1}{2}$.

Effectiveness of Policy. A mediator's policy is evaluated based on the effectiveness of the queries it produces. More formally, the effectiveness of policy π for local dataset $\mathcal{E}$ and external dataset D is $F(\mathcal{E}, D, \pi) = \sum_{e \in \mathcal{E}} P(e)\ f(X(e), D[\pi(e)])$ where $P(.)$ is the prior probability of choosing local entities for augmentation by users. Unless otherwise noted, we assume that P(.) is uniform.

Optimal Policy. Optimal policy delivers the maximum effectiveness across the entire local dataset. More precisely, the optimal policy π^* for local dataset $\mathcal{E}$ and external dataset D is $\operatorname{argmax}_{\pi} F(\mathcal{E}, D, \pi)$.

Problem Statement. Given local dataset $\mathcal{E}$ and external dataset D, the mediator seeks to find the optimal policy π^* from the set of all possible policies. Finding the optimal policy requires a sufficient understanding of both the representation of entities in $\mathcal{E}$ and how query interface over D answers queries.

Merging Local and External Information. One might have to merge local data with its relevant external data by performing other steps of data integration, such as schema matching [14]. However, it takes more than one paper to investigate all steps of data integration. Thus, we assume that in these settings, users leverage existing data integration tools to create the final dataset and focus on the task of collecting information from external sources effectively.

Sudowoodo: Contrastive Self-supervised Learning for Multi-purpose Data Integration and Preparation

Runhui Wang[§]
Computer Science Department
Rutgers University
New Brunswick, United States
runhui.wang@rutgers.edu

Yuliang Li
Megagon Labs
Mountain View, United States
yuliang@megagon.ai

Jin Wang
Megagon Labs
Mountain View, United States
jin@megagon.ai

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00391

***Abstract*—Machine learning (ML) is playing an increasingly important role in data management tasks, particularly in Data Integration and Preparation (DI&P). The success of ML-based approaches, however, heavily relies on the availability of large-scale, high-quality labeled datasets for different tasks. Moreover, the wide variety of DI&P tasks and pipelines oftentimes requires customizing ML solutions at a significant cost for model engineering and experimentation. These factors inevitably hold back the adoption of ML-based approaches to new domains and tasks.**

In this paper, we propose Sudowoodo**, a multi-purpose DI&P framework based on contrastive representation learning.** Sudowoodo **features a unified, matching-based problem definition capturing a wide range of DI&P tasks including Entity Matching (EM) in data integration, error correction in data cleaning, semantic type detection in data discovery, and more. Contrastive learning enables** Sudowoodo **to learn similarity-aware data representations from a large corpus of data items (e.g., entity entries, table columns) without using any labels. The learned representations can later be either directly used or facilitate fine-tuning with only a few labels to support different DI&P tasks. Our experiment results show that** Sudowoodo **achieves multiple state-of-the-art results on different levels of supervision and outperforms previous best specialized blocking or matching solutions for EM.** Sudowoodo **also achieves promising results in data cleaning and column matching tasks showing its versatility in DI&P applications.**

I. Introduction

Machine learning, particularly deep learning (DL), is playing an increasingly important role in almost all fields in computer science including data management [1], [2], [3], [4]. More recently, this trend has extended to Data Integration and Preparation (DI&P) [5], [6], [7], [8] with learning-based solutions making promising progress and advancing the state-of-the-art in multiple tasks, including entity matching [9], [10], data cleaning [11], [12], [13], [14], and table annotation [15], [16], [17]. For example, Entity Matching (EM) solutions based on pre-trained language models such as BERT [18] achieve new state-of-the-art results across multiple matching tasks [19], [10]. While shown to be effective, as we describe next, existing learning-based solutions for DI&P suffer from two major challenges making them less attractive in practice.

Challenge 1: Label Requirement. The success of ML-based approaches comes at the cost of creating large-scale, high-quality annotated datasets. For example, in Entity Matching, learning-based methods typically require around 10k or more labeled entity pairs to achieve the best pairwise matching performance [9], [10]. Creating high-quality datasets of such cardinality can be quite expensive considering the required human efforts, which make ML-based solutions less accessible to new DI&P tasks and domains.

Challenge 2: Task Variety. There is a wide scope of DI&P tasks requiring ML models for different problem formulations. This means that oftentimes, practitioners have to build a specialized ML solution for each task, resulting in extra costs of model engineering. For example, Entity Matching typically consists of two tasks, *blocking* and *matching* [20], with the goal of (1) filtering out candidate matching pairs and (2) performing the pairwise comparison, respectively. The two tasks are closely related, but because of their different problem formulations, existing studies typically propose specialized solutions [21], [9], [19], [22], [10] that require separate modeling, data annotation, and experimentation efforts. for each task. This has also been the case for other DI&P tasks in data cleaning and data discovery [11], [12], [13], [14], [15], [16], [17].

To address these two challenges, we proposed Sudowoodo, a multi-purpose DI&P framework based on *contrastive representation learning* [23]. Sudowoodo features a generic DI&P problem definition of data matching: given a collection of data items (e.g., entity entries, table columns, cell values), identify all pairs of compatible items. This generic definition allows Sudowoodo to support a wide range of tasks in data integration, cleaning, and discovery. As Figure 1 illustrates, EM can be formulated as a task of matching entity entries; error correction in data cleaning [12], [13], [14] can be formulated as matching erroneous cell values with candidate corrections; and column matching, which is widely used in data discovery [24], [15], can be formulated as matching table columns that have the same semantic type. The matching criteria are also customizable w.r.t the specific application. In this work, we consider any criteria that can be cast into a general binary relation, such as whether two entity entries refer to the same real-world entity for EM.

Sudowoodo addresses the label requirement challenge by

[§]This work was done during the internship at Megagon Labs

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00391

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

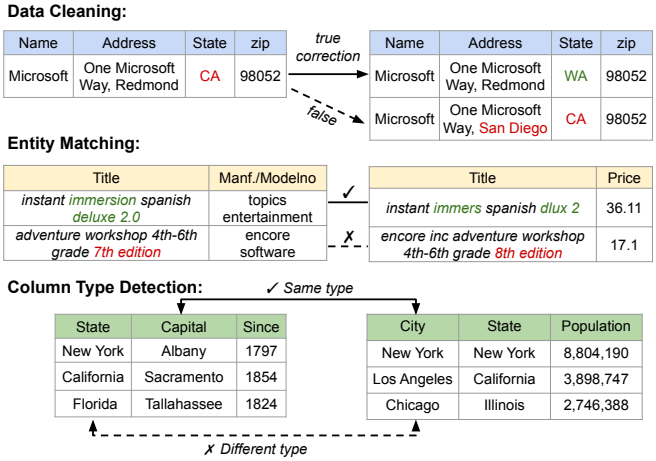


Fig. 1: Sudowoodo supports any DI&P tasks that can be formulated as a general form of matching task.

leveraging contrastive learning to learn a data representation model from a large collection of unlabeled data items, such as corpora of entity entries or table columns. Unlike that of language models such as BERT [18], the effectiveness of Sudowoodo's pre-training process relies on its contrastive learning objective instead of learning linguistic characteristics. The contrastive objective allows Sudowoodo to learn how to distinguish pairs of similar data items from dissimilar ones that are likely to be distinct. In this way, without using any labels, Sudowoodo learns meaningful representations where similar data items are close in the representation space while pairs of dissimilar data items are far apart.

After the pre-training process, the learned data representation model is accessible to downstream tasks either directly in an unsupervised manner or by *fine-tuning* using a few labels. Unlike that in language models, fine-tuning is not always necessary if the downstream task is similarity-based for which Sudowoodo already provides a reliable metrics space. When fine-tuning is needed, Sudowoodo provides an optional *pseudo labeling* step that extracts training signals with high confidence from the learned representations useful for further boosting the fine-tuning performance. As we will illustrate, pseudo labeling together with a customized fine-tuning architecture significantly improves Sudowoodo's performance on DI&P tasks in low-resourced settings.

Example 1: We illustrate the Sudowoodo pipeline in Figure 2 using Entity Matching (EM), a typical application of data integration. The upper part of the figure shows a standard EM pipeline consisting of the blocking and matching sub-tasks. Given two collections of entity entries (Tables A and B), the goal of blocking is to select a candidate set of matching pairs from the cross product to avoid the quadratic-size pairwise comparison. Next, it trains a matching model on a labeled dataset sampled from the candidate set, before applying the model to all the candidate pairs for the final matching results.

Sudowoodo starts with contrastive learning to learn the similarity-aware data representations of all entity entries from the two input tables. Because its model already captures entity similarity, we can solve the blocking task by applying

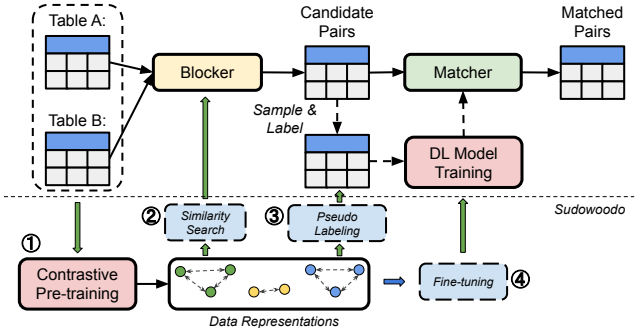


Fig. 2: The Sudowoodo pipeline for EM (blocking+matching).

nearest neighbor search to identify the top-k most similar entity entries for all input entries as the candidate set. For the matching task, practitioners can then fine-tune the pre-trained representation model on a labeled training set. For this step, Sudowoodo's pseudo labeling module allows automatic creation of additional high-quality labels by extracting match/unmatch pairs with high confidence from the learned representations. By doing so, Sudowoodo significantly reduces the label requirement for training the matching model.

Further optimizations. Based on the standard contrastive learning framework, Sudowoodo features several optimizations to further boost the pre-training effectiveness. First, the quality of learned representations heavily depends on the generated pairs of data items that are semantic-preserving. Sudowoodo obtain such pairs via data transformation such as synonym replacement or span deletion. To obtain more fine-grained transformations, Sudowoodo supports the *cutoff* data augmentation (DA) technique [25] that transforms input data items at the word-embedding level. Combining cutoff with a package of DI&P data augmentation operators, Sudowoodo obtains more diverse views for contrastive pre-training.

Another challenge for applying contrastive learning to DI&P tasks is how to obtain more effective negative examples. Negative examples allow the representation model to learn how to separate dissimilar data items. The standard contrastive learning approach uses uniformly sampled pairs as negative examples. This simple approach can be insufficient when random pairs fail to provide strong enough negative examples because they are easily separable. To address this issue, Sudowoodo splits the unlabeled data items into clusters and samples negative pairs within each cluster. By doing so, Sudowoodo allows the representation model to learn "harder" by distinguishing syntactically similar items.

In summary, this paper makes the following contributions:

- We propose a generic matching problem definition that captures a wide variety of DI&P tasks including (1) EM in data integration, (2) Error correction in data cleaning, (3) Semantic type detection in data discovery, and more.
- We propose Sudowoodo, a contrastive representation learning framework for the supported DI&P tasks. Sudowoodo pre-trains a representation model from a large collection of unlabeled data items. Practitioners can then apply the representations to downstream tasks either directly or by

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

fine-tuning with few labels.

- We introduce 3 optimizations to Sudowoodo. Specifically, we (1) augment the pre-training data using a novel cutoff operator, (2) introduce "harder" negative samples via clustering, and (3) combine the standard contrastive loss with a non-contrastive learning objective through redundancy regularization [26].
- We evaluate Sudowoodo on real-world datasets for EM, data cleaning, and semantic type detection. Sudowoodo achieves a series of new SOTA results under different label budgets, highlighting its capability of holistically providing solutions for multiple DI&P tasks.

The rest of the paper is organized as follows. Section II defines the problem and introduces the overall architecture. We introduce the contrastive learning algorithms in Section III and their optimizations in Section IV. Section V extends Sudowoodo to data cleaning and column matching. Section VI summarizes the experiment results. We discuss related work in Section VII and conclude in Section VIII.

II. Background and System Overview

In this section, we first define a general matching task for capturing a broad set of DI&P tasks including Entity Matching (EM), data cleaning, and column type detection. We then review the baseline method of pre-trained language models and fine-tuning for solving the task. We also provide an overview of Sudowoodo's architecture.

A. Task definition

In Sudowoodo, we consider DI&P tasks that can be formulated as a general problem of *matching related data items*. These data items can be of diverse formats across different DI&P tasks. For example, as Figure 1 illustrates, these data items can be entity entries, table columns, cell values, etc. The matching criteria can also be customizable binary relations w.r.t the specific DI&P applications, such as: whether two entity entries refer to the same real-world entity for EM; whether a row is a correct candidate correction to an erroneous row for data cleaning; or whether two table columns have the same semantic type for data discovery.

It is commonly seen that within a DI&P task, the same matching problem is formulated in different ways to address different application needs. For example, in an EM task, the blocking stage requires quickly getting a small candidate set that contains filtered matching pairs with high recall while the matching stage requires both high recall and precision. To facilitate their different needs, in Sudowoodo, we consider two flavors of the matching tasks: *embedding* and *pairwise matching*. The embedding task aims at generating a vector representation for each data item so that related data items are close in the vector space. The generated vectors can then be indexed and queried to support fast similarity search [27] or clustering [28]. The goal of the pairwise matching task is to perform binary classification for determining whether a pair of data items are related or not. We formally define these two tasks as follows:

Definition 1 (Embedding): Given a collection D of data items, a d-dimensional embedding model M_{emb} takes every data item $x \in D$ as input and outputs a real vector $M_{\mathsf{emb}}(x) \in \mathbb{R}^d$. Given a similarity function sim, e.g., cosine, for every pair of data items (x, x'), the value of $\mathsf{sim}(x, x')$ is large if and only if (x, x') matches.

For simplicity, we assume all output vectors are normalized, i.e. the L-2 norm $\|M_{\mathsf{emb}}(x)\|_2 = 1$ for all data item x.

Definition 2 (Pairwise Matching): Given a collection D of data items, a pairwise matching model M_{pm} takes a pair (x, y) of data items as input and outputs $M_{\mathsf{pm}}(x, y) \in \{0, 1\}$ where $M_{\mathsf{pm}}(x, y) = 1$ if (x, y) is a real match or 0 otherwise.

Note that we can also abuse the notation $M_{\mathsf{pm}}(x, y)$ as a 2-d real vector where each dimension indicates the predicted probability of non-match / match denoted as $M_{\mathsf{pm}}(x, y)_0$ and $M_{\mathsf{pm}}(x, y)_1$ respectively.

Entity Matching. With the help of Sudowoodo, it is rather easy to build an effective EM pipeline. The embedding model provides the similarity measure for the blocking stage of EM while the matching stage can directly use the pairwise matching model that Sudowoodo provides. For the rest of the paper, we will take EM as the main example when introducing the Sudowoodo framework for clarity. We will describe how to extend Sudowoodo to the data cleaning and column type detection tasks later in Section V.

B. Fine-tuning language models

The goal of Sudowoodo is to (1) train high-quality embedding models without labels and (2) train the pairwise matching models using no or just a few labels. Pre-trained language models (LMs), such as BERT, provide a good baseline solution which has recently achieved promising results for EM [10], [29], [22], [30]. These models typically consist of 12 or more Transformer layers [31] pre-trained on large unlabeled text corpora such as Wikipedia. In the process of pre-training, the LMs are trained in a self-supervised manner to perform auxiliary tasks such as missing token and next-sentence prediction. In this way, LMs effectively capture lexical or semantic meanings of the input text sequences. Formally, a LM M encodes an input token sequence x into a vector representation $M(x)$.

To apply pre-trained LMs in Sudowoodo, we first need to serialize the input data items into sequences of tokens. For EM, we follow the serialization method in Ditto [10] which converts an entity entry into the format:

[COL] Title [VAL] instant immers ... [COL] Price [VAL] 36.11

where [COL] and [VAL] are special tokens indicating the start of an attribute or value. We denote by $\mathsf{serialize}(x)$ the serialization of a data item x. We omit "serialize" when the context is clear. After that, we can obtain a baseline embedding model that first serializes an input data item and encodes it using the pre-trained LM.

Next, one can obtain a baseline pairwise matching model via *fine-tuning* the pre-trained model on a downstream task-specific training set. Pre-trained LMs like BERT support

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.